

# Theoretische Informatik

---

Prof. Dr. Sebastian Ritterbusch

Studiengang  
Wirtschaftsinformatik

Studienakademie  
Mannheim



# Inhaltsverzeichnis

|                                                                             |     |
|-----------------------------------------------------------------------------|-----|
| Kapitel 1. Formale Sprachen                                                 | 5   |
| 1.1. Alphabete und Wörter                                                   | 5   |
| 1.2. Sprachen und Abzählbarkeit                                             | 6   |
| 1.3. Verknüpfungen von Sprachen und Entscheidbarkeit                        | 10  |
| 1.4. Aufgaben                                                               | 14  |
| Kapitel 2. Grammatiken                                                      | 17  |
| 2.1. Grammatiken                                                            | 17  |
| 2.2. Die Chomsky-Hierarchie                                                 | 20  |
| 2.3. Grammatiken allgemeiner (Typ-0) und kontextsensitiver (Typ-1) Sprachen | 21  |
| 2.4. Grammatiken kontextfreier (Typ-2) Sprachen                             | 23  |
| 2.5. Grammatiken regulärer (Typ-3) Sprachen                                 | 31  |
| 2.6. Aufgaben                                                               | 35  |
| Kapitel 3. Rechnermodelle                                                   | 41  |
| 3.1. Endliche Automaten                                                     | 41  |
| 3.2. Kellerautomaten                                                        | 59  |
| 3.3. Turingmaschinen                                                        | 66  |
| 3.4. Aufgaben                                                               | 70  |
| Kapitel 4. Berechenbarkeit und Komplexität                                  | 75  |
| 4.1. Berechenbarkeit                                                        | 75  |
| 4.2. Komplexitätsklassen                                                    | 82  |
| 4.3. Algorithmische Komplexität                                             | 89  |
| 4.4. Aufgaben                                                               | 92  |
| Kapitel 5. Lösungen                                                         | 95  |
| Literaturverzeichnis                                                        | 121 |
| Index                                                                       | 123 |



## Formale Sprachen

Formale Sprachen sind ein zentraler Aspekt der theoretischen Informatik. Sie können nicht nur ein vielseitiges für Menschen und Computer verständliches Nutzungsinterface darstellen, sondern haben sich auch als Programmiersprachen zur Entwicklung der Software für Computer etabliert. Die Erkennung unterschiedlicher Klassen von formalen Sprachen kann durch unterschiedliche Computermodelle realisiert werden, und durch Sprachen können neben Betrachtungen von Komplexitäten viele zentrale Probleme der Informatik erfasst und analysiert werden.

### 1.1. Alphabete und Wörter

**DEFINITION 1.1.** Ein **Alphabet**  $\Sigma = \{\alpha_1, \alpha_2, \dots, \alpha_n\}$  ist eine endliche Menge von **Zeichen** oder **Symbolen**.

**BEISPIEL 1.2.** Das Alphabet  $\Sigma_{\text{lat}} = \{\mathbf{a}, \mathbf{b}, \mathbf{c}, \dots, \mathbf{z}\}$  kleiner lateinischer Buchstaben ist eine abzählbare Menge, genauso bilden auch die Zahlen  $\Sigma_{\text{zahl}} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$  oder der Unicode  $\Sigma_{\text{Unicode}} = \{x \mid x \text{ ist ein Zeichen im Unicode}\}$  ein Alphabet. Boolesche Ausdrücke haben auch ein entsprechendes Alphabet  $\Sigma_{\text{logik}} = \{0, 1, (, ), \wedge, \vee, \neg, (, )\} \cup \Sigma_{\text{lat}}$ .

**DEFINITION 1.3.** Ein **kartesisches Produkt** wie  $A \times B$  oder  $A^n$  für  $n \in \mathbb{N}$  von Mengen oder Alphabeten bezeichnet die Menge der **Tupel**  $(a, b)$  oder  $(a_1, \dots, a_n)$  von Elementen der Mengen:

$$A \times B := \{(a, b) \mid a \in A, b \in B\}$$

$$A^n := \underbrace{A \times \dots \times A}_n = \{(a_1, \dots, a_n) \mid a_1, \dots, a_n \in A\}$$

$n$  Faktoren

**BEISPIEL 1.4.** Die möglichen Zustände eines Laptops und Programms können so beschrieben werden:

$$\{\text{netz, akku, leer}\} \times \{\text{saved, notsaved}\} = \{(\text{netz, saved}), (\text{akku, saved}), (\text{leer, saved}),$$

$$(\text{netz, notsaved}), (\text{akku, notsaved}),$$

$$(\text{leer, notsaved})\}$$

**DEFINITION 1.5.** Ein **Wort**  $\omega$  ist ein endliches oder leeres Tupel  $(w_1, w_2, \dots, w_n) \in \Sigma^n$  von Zeichen  $w_k \in \Sigma$  eines Alphabets mit **Länge**  $|\omega| = n$  der Anzahl der Zeichen. Wörter werden im Folgenden meist ohne die Klammern und Kommata der Tupel, also als  $w_1 w_2 \dots$  geschrieben. Dabei bezeichnet  $\varepsilon$  ein **leeres Wort**  $()$  ohne Zeichen. Die **Wortmengen**

$$\Sigma^0 := \{\varepsilon\}, \quad \Sigma^* := \bigcup_{n \geq 0} \Sigma^n, \quad \Sigma^+ := \bigcup_{n > 0} \Sigma^n$$

bezeichnen die Menge der Worte  $\Sigma^0$  über dem Alphabet  $\Sigma$  mit keinem Zeichen, die Menge der Worte  $\Sigma^*$  mit beliebig vielen Zeichen und die Menge der Worte  $\Sigma^+$  mit mindestens einem Zeichen. Die Operationen  $M^*$  und  $M^+$  auf einer Menge  $M$  wird auch **Kleene-\* -Abschluss** oder **Kleene-+ -Abschluss** bezeichnet.

BEISPIEL 1.6. Ist  $M = \{01, 2\}$ , so ergeben sich u.a. diese Wortmengen:

$$M^0 = \{\varepsilon\}$$

$$M^1 = \{01, 2\}$$

$$M^2 = \{0101, 012, 201, 22\}$$

$$M^3 = \{010101, 01012, 01201, 0122, 20101, 2012, 2201, 222\}$$

...

$$M^+ = M^1 \cup M^2 \cup \dots = \{01, 2, 0101, 012, 201, 22, 010101, 01012, \dots\}$$

$$M^* = M^0 \cup M^+ = \{\varepsilon, 01, 2, 0101, 012, 201, 22, 010101, 01012, \dots\}$$

Insbesondere hängt die Wortlänge  $|w|$  für ein  $w \in L^*$  von der Definition des Alphabets  $L$  ab. So ist in diesem Beispiel  $|222| = 3$  während  $|0101| = 2$  ist.

**DEFINITION 1.7.** Für zwei Wörter  $u = (u_1, \dots, u_n)$  und  $w = (w_1, \dots, w_m)$  ist das **Produkt** oder **Konkatenation**

$$u \cdot w = uw = (u_1, \dots, u_n, w_1, \dots, w_m) = u_1 \dots u_n w_1 \dots w_m.$$

Die  $n$ -te **Potenz** eines Wortes  $w$  ist

$$w^n = \underbrace{w \cdot \dots \cdot w}_{n \text{ Faktoren}} \text{ für } n > 0,$$

oder das leere Wort  $w^0 = \varepsilon$  für  $n = 0$ .

BEISPIEL 1.8. Sei  $\Sigma = \{a, e, n, r\}$ , sowie  $w = na \in \Sigma^*$  und  $v = er \in \Sigma^*$ . Dann ist:

$$w^2 = nana$$

$$vw = erna$$

$$wv^2w = naererna$$

**BEMERKUNG 1.9.** Der Begriff **Abschluss** in Definition 1.5 bedeutet, dass auf einer Menge mit einer Verknüpfung, hier einer Sprache und dem Produkt, jede Anwendung der Operation mit Elementen aus einer Kleene-Abschluss-Menge wieder ein Element aus der Menge ergibt. Beispielsweise ist die Subtraktion auf den natürlichen Zahlen nicht abgeschlossen, der Abschluss der natürlichen Zahlen bezüglich der Subtraktion sind die ganzen Zahlen.

Die Kleene-Abschlüsse und Multiplikationen werden auch verkürzt auf Wörtern verwendet, damit ist dann der Abschluss oder das kartesische Produkt der Menge mit genau diesem Wort gemeint, wie beispielsweise:

$$(ab)^+ = \{ab\}^+ = \{ab, abab, ababab, \dots\}$$

$$cd^*e = \{c\} \times \{d\}^* \times \{e\} = \{ce, cde, cdde, cddde, \dots\}$$

## 1.2. Sprachen und Abzählbarkeit

**DEFINITION 1.10.** Jede Teilmenge  $L \subseteq \Sigma^*$  ist eine **formale Sprache** über dem Alphabet  $\Sigma$ .

BEISPIEL 1.11. Auf dem Alphabet  $\Sigma = \{0, 1, 2\}$  ist  $\Sigma^*$  die Menge oder Sprache von Wörtern aus Ziffern 0, 1 oder 2 beliebiger Länge wie 101 oder auch 0001. Weitere formale Sprachen sind hier beispielsweise:

(1) Die Menge  $M_3 \subset \Sigma^*$  der binären Zahlen ohne führende Nullen:

$$M_3 = \{0\} \cup \{1\} \times \{0, 1\}^* = \{0, 1, 10, 11, 100, 101, 110, 111, 1000, 1001, \dots\}$$

(2) Die Menge  $M_2 \subset \Sigma^*$  von einer gleichen Anzahl von 0 und 1 in dieser Reihenfolge:

$$M_2 = \{0^n 1^n \mid n \in \mathbb{N}\} = \{01, 0011, 000111, 00001111, 0000011111, \dots\}$$

(3) Die Menge  $M_1 \subset \Sigma^*$  von einer gleichen Anzahl von 0, 1 und 2 in dieser Reihenfolge:

$$M_1 = \{0^n 1^n 2^n \mid n \in \mathbb{N}\} = \{012, 001122, 000111222, 000011112222, \dots\}$$

(4) Die Menge  $M_0 \subset \Sigma^*$  mit Wörtern der Länge von Zweierpotenzen:

$$M_0 = \{w \in \Sigma^* \mid |w| = 2^n, n \in \mathbb{N}\} = \{0, 1, 2, 00, 01, \dots, 21, 22, 0000, 0001, \dots\}$$

Diese Beispiele werden dann in Beispielen 2.4 und 2.9 mit unterschiedlichen Grammatik-Typen erzeugt.

Die Beispiele zeigen, dass mit endlichen Alphabeten formale Sprachen unendlich groß sein können.

**DEFINITION 1.12.** Eine Menge  $M$  ist **abzählbar**, wenn die einzelnen Elemente abzählbar sind, es also eine bijektive Funktion  $f : N \rightarrow M$  von den natürlichen Zahlen  $N = \mathbb{N}$  oder einer Teilmenge der natürlichen Zahlen  $N \subset \mathbb{N}$  gibt. Wenn es keine solche Funktion geben kann, so ist die Menge **überabzählbar unendlich**.

**SATZ 1.13.** Jede endliche Menge  $M$  ist abzählbar.

**BEWEIS.** Eine endliche Menge  $M$  hat eine endliche Anzahl  $n = |M|$  von Elementen. Wird nun beginnend von  $M_0 = M$  und  $k = 1$  in  $n$  Schritten jeweils ein Element  $m_k$  der Menge  $M_{k-1}$  entnommen mit  $M_k = M_{k-1} \setminus \{m_k\}$ , so ist induktiv  $|M_k| = |M_{k-1}| - 1 = n - k$  und es ist  $M_n = \emptyset$ . Die Bijektion lautet dann  $f : N \rightarrow M$  mit  $f(k) = m_k$  mit  $N = \{1, \dots, n\}$ .  $\square$

**SATZ 1.14.** Jede Teilmenge  $M \subseteq N$  einer abzählbaren Menge  $N = \{n_1, n_2, \dots\}$  ist abzählbar.

**BEWEIS.** Sei  $f(k) = n_k$  die Abzählung der Menge  $N$ . Sei  $R = \{k \in \mathbb{N} \mid n_k \in M\}$  die Menge der Indizes der Elemente aus  $N$ , die in  $M$  sind. Dann ist die Einschränkung  $f|_R : R \rightarrow M$  von  $f$  genau die Abzählung, die die Abzählbarkeit von  $M$  beweist.  $\square$

**BEISPIEL 1.15.** Einfache Beispiele für abzählbar unendliche Mengen sind zum Beispiel die geraden Zahlen  $\{2n \mid n \in \mathbb{N}\}$  oder die Quadratzahlen  $\{n^2 \mid n \in \mathbb{N}\}$  oder auch die Menge der Fakultäten  $\{n! \mid n \in \mathbb{N}\}$ . Ebenso sind die ganzen Zahlen  $\mathbb{Z}$  mit der Funktion

$$f(n) = \begin{cases} \frac{n}{2}, & \text{wenn } n \text{ gerade} \\ -\frac{n-1}{2}, & \text{wenn } n \text{ ungerade} \end{cases}$$

abzählbar:  $f(1) = 0, f(2) = 1, f(3) = -1, f(4) = 2, \dots$  Auch die rationalen Zahlen  $\mathbb{Q} = \{\frac{z}{n} \mid z \in \mathbb{Z}, n \in \mathbb{N}\}$  sind abzählbar, das **Diagonalisierungsverfahren von Cantor** wird dazu mit dieser Tabelle für die positiven rationalen Zahlen veranschaulicht:

$$\begin{array}{ccccccc} \frac{1}{1} & & \frac{2}{1} & \rightarrow & \frac{3}{1} & & \frac{4}{1} \rightarrow \frac{5}{1} \dots \\ \downarrow & \nearrow & & \nwarrow & & \nearrow & \downarrow \\ \frac{1}{2} & & \frac{2}{2} & & \frac{3}{2} & & \frac{4}{2} \rightarrow \frac{5}{2} \dots \\ & \nwarrow & \nearrow & & \nwarrow & \nearrow & \\ \frac{1}{3} & & \frac{2}{3} & & \frac{3}{3} & & \frac{4}{3} \rightarrow \frac{5}{3} \dots \\ \downarrow & \nearrow & & \nwarrow & & \nearrow & \downarrow \\ \frac{1}{4} & & \frac{2}{4} & & \frac{3}{4} & & \frac{4}{4} \rightarrow \frac{5}{4} \dots \\ \vdots & & \vdots & & \vdots & & \vdots \end{array}$$

Die 0 und alle negativen Brüche können wieder wie bei den ganzen Zahlen eingeschoben werden. Damit sind auch alle rationalen Vektoren  $\mathbb{Q}^n$  in beliebiger Dimension  $n \in \mathbb{N}$  abzählbar, da jeweils eine Dimension mit dem Diagonalisierungsverfahren heruntergezählt werden kann.

SATZ 1.16. Für jede endliche Menge oder jedes Alphabet  $\Sigma$  ist deren Kleene-Abschluss  $\Sigma^*$  abzählbar.

BEWEIS. Ist das Alphabet  $\Sigma$  leer, so ist auch  $\Sigma^*$  leer, und damit für  $N = \emptyset$  trivial abzählbar. In allen anderen Fällen besitzt  $\Sigma$  mit Größe  $n = |\Sigma|$  nach Satz 1.13 eine Aufzählung  $m_k$  mit  $k = 1, \dots, n$ . Jedes Wort kann dann im **Stellenwertsystem** zur Basis  $n + 1$  dargestellt werden: Dem Wort  $w$  aus  $l = |w|$  Symbolen  $m_{k_1} m_{k_2} \dots m_{k_l}$  wird die Zahl

$$1 + k_1 \cdot (n + 1)^{l-1} + k_2(n + 1)^{l-2} + \dots + k_l(n + 1)^0$$

also der Zahl  $1 + (k_1 k_2 \dots k_l)_{(n+1)}$  zugeordnet. Die Abbildung  $f : N \rightarrow \Sigma^*$  mit  $N \subseteq \mathbb{N}$  ergibt für  $f(x)$  aus der Stellenwertdarstellung von  $x - 1 > 0$  zur Basis  $n + 1$  beginnend mit der höchstwertigen Ziffer  $k_1$  bis zur letzten Stelle  $k_l$ . Das Bild  $f(x)$  ist dann das Wort  $m_{k_1} m_{k_2} \dots m_{k_l}$ . Das leere Wort  $\varepsilon$  wird von 1 abgebildet und entsprechend ist  $f(1) = \varepsilon$ , und daher ist auch die Addition von 1 in der Berechnung des Werts für alle Worte erforderlich.  $\square$

BEISPIEL 1.17. Auf dem Alphabet  $\Sigma = \{\mathbf{e}, \mathbf{i}, \mathbf{rn}, \mathbf{st}\}$  mit der Aufzählung  $m_1 = \mathbf{e}$ ,  $m_2 = \mathbf{i}$ ,  $m_3 = \mathbf{rn}$ ,  $m_4 = \mathbf{st}$  des Alphabets haben die folgenden Wörter diese Abzählung nach dem Verfahren aus Satz 1.16 für  $f : N \rightarrow \Sigma^*$ :

| $x$    | 1             | 2                    | 3                    | 4                    | 5                    | 6                                 | 7                                 | 8                                 |         |
|--------|---------------|----------------------|----------------------|----------------------|----------------------|-----------------------------------|-----------------------------------|-----------------------------------|---------|
|        | 1             | $1 + 1$<br>$1 + 1_5$ | $1 + 2$<br>$1 + 2_5$ | $1 + 3$<br>$1 + 3_5$ | $1 + 4$<br>$1 + 4_5$ | $1 + 1 \cdot 5 + 0$<br>$1 + 10_5$ | $1 + 1 \cdot 5 + 1$<br>$1 + 11_5$ | $1 + 1 \cdot 5 + 2$<br>$1 + 12_5$ | $\dots$ |
| $f(x)$ | $\varepsilon$ | $\mathbf{e}$         | $\mathbf{i}$         | $\mathbf{rn}$        | $\mathbf{st}$        | ---                               | $\mathbf{ee}$                     | $\mathbf{ei}$                     |         |

| $x$    | 45                                              |         | 109                                             |         | 114                                             |         |
|--------|-------------------------------------------------|---------|-------------------------------------------------|---------|-------------------------------------------------|---------|
|        | $1 + 1 \cdot 25 + 3 \cdot 5 + 4$<br>$1 + 134_5$ | $\dots$ | $1 + 4 \cdot 25 + 1 \cdot 5 + 3$<br>$1 + 413_5$ | $\dots$ | $1 + 4 \cdot 25 + 2 \cdot 5 + 3$<br>$1 + 423_5$ | $\dots$ |
| $f(x)$ | $\mathbf{ernst}$                                |         | $\mathbf{stern}$                                |         | $\mathbf{stirn}$                                |         |

In dieser Abzählung bleiben (nur) die Zahlen unbesetzt, die eine 0 in der Stellenwertdarstellung besitzen.

THEOREM 1.18. Jede formale Sprache ist abzählbar.

BEWEIS. Da jede formale Sprache  $L$  über einem endlichen Alphabet  $\Sigma$  definiert ist, ist das eine direkte Folge aus Satz 1.16, dass  $\Sigma^*$  abzählbar ist, und damit nach Satz 1.14 die Teilmenge  $L \subseteq \Sigma^*$  abzählbar ist.  $\square$

Die oben eingeführte Abzählung mit Hilfe eines Stellenwertsystems ist nur für Abschlüsse von endliche Mengen realisierbar. Damit kann diese Methode nicht auf aus Wörtern bestehende Sätze, wie später Programme, angewendet werden, wenn die zu Grunde liegende Menge der möglichen Wörter einer Sprache zwar abzählbar, doch aber unendlich ist. Eine allgemeinere Art der Abzählung, die für abzählbar unendliche Ursprungsmengen funktioniert, ist die Berechnung der Gödelnummer:

DEFINITION 1.19. Sei  $(p_n)$  die Folge der Primzahlen:

$$p_1 = 2, p_2 = 3, p_3 = 5, p_4 = 7, p_5 = 11, p_6 = 13, \dots$$

Für eine abzählbare Menge  $M = \{m_1, m_2, \dots\}$  ist die **Gödelnummer**  $c_M : M^* \rightarrow \mathbb{N}$  des Tupels  $w = (m_{k_1}, m_{k_2}, \dots, m_{k_l})$  gegeben durch

$$c_M(w) = p_1^{k_1} \cdot p_2^{k_2} \cdot \dots \cdot p_l^{k_l} = \prod_{i=1}^l p_i^{k_i}.$$

BEISPIEL 1.20. Auf dem Alphabet  $\Sigma = \{\mathbf{e}, \mathbf{i}, \mathbf{rn}, \mathbf{st}\}$  mit der Aufzählung  $m_1 = \mathbf{e}$ ,  $m_2 = \mathbf{i}$ ,  $m_3 = \mathbf{rn}$ ,  $m_4 = \mathbf{st}$  des Alphabets haben die folgenden Wörter diese Abzählung  $f : N \rightarrow \Sigma^*$  mit der Gödelnummer aus Definition 1.19:



|                   |               |       |           |       |               |           |                   |       |           |               |         |
|-------------------|---------------|-------|-----------|-------|---------------|-----------|-------------------|-------|-----------|---------------|---------|
| $x = c_\Sigma(w)$ | 1             | 2     | 3         | 4     | 5             | 6         | 7                 | 8     | 9         | 10            |         |
|                   | $2^0$         | $2^1$ | $2^0 3^1$ | $2^2$ | $2^0 3^0 5^1$ | $2^1 3^1$ | $2^0 3^0 5^0 7^1$ | $2^3$ | $2^0 3^2$ | $2^1 3^0 5^1$ | $\dots$ |
| $f(x) = w$        | $\varepsilon$ | e     | ---       | i     | ---           | ee        | ---               | rn    | ---       | ---           |         |

|                   |       |                      |           |         |               |         |               |         |               |         |
|-------------------|-------|----------------------|-----------|---------|---------------|---------|---------------|---------|---------------|---------|
| $x = c_\Sigma(w)$ | 16    | 17                   | 18        |         | 6000          |         | 18000         |         | 33750         |         |
|                   | $2^4$ | $2^0 3^0 \dots 17^1$ | $2^1 3^2$ | $\dots$ | $2^4 3^1 5^3$ | $\dots$ | $2^4 3^2 5^3$ | $\dots$ | $2^1 3^3 5^4$ | $\dots$ |
|                   | 4     |                      | 1, 2      | $\dots$ | 4, 1, 3       | $\dots$ | 4, 2, 3       | $\dots$ | 1, 3, 4       | $\dots$ |
| $f(x) = w$        | st    | ---                  | ei        |         | stern         |         | stirn         |         | ernst         |         |

Die Zuordnung von Gödelnummern ist durch die Eindeutigkeit der Primfaktorzerlegung eindeutig und es bleiben die Zahlen unbesetzt, die bis zum höchsten Primzahlfaktor davor eine Primzahlpotenz von 0 haben.

Eine Abzählung mit Gödelnummern geht weiter als eine Abzählbarkeit der Menge von Vektoren  $\mathbb{Q}^n$  mit fester Länge  $n$ , da damit alle Elemente von  $M^*$  über einer abzählbar unendlichen Menge  $M$  beliebige Längen haben können. Dies bedeutet, dass jedes in einer formalen Sprache formulierbares Programm abzählbar ist:

**SATZ 1.21.** Die Menge von endliche Folgen  $P = \{p = (w_1, w_2, \dots, w_n) \mid w_k \in L, n \in \mathbb{N}\}$  aus Wörtern einer formalen Sprache  $L \subseteq \Sigma^*$  über einem Alphabet  $\Sigma$  ist abzählbar.

**BEWEIS.** Jeder Abschluss  $\Sigma^*$  ist nach Satz 1.16 abzählbar. Damit ist jede Sprache  $L \subseteq \Sigma^*$  nach Satz 1.14 abzählbar. Damit kann nach Definition 1.19 für jede Folge  $p \in P$  injektiv ein Gödelnummer  $c_L(p)$  über  $L$  bestimmt werden. Auf der Menge  $N = \{x = c_L(p) \mid p \in P\}$  kann die Umkehrung  $f : N \rightarrow P$  von  $c_L$  auf  $P$  eingeschränkten bijektiven Funktion  $c_{L|P} : P \rightarrow N$  bestimmt werden, und damit ist  $P$  abzählbar.  $\square$

Angenommen jedes in einer formalen Sprache geschriebenes Programm löst ein Problem, was als Berechnung einer Lösung interpretiert werden kann, so sind dies verschwindend wenige lösbare Probleme, verglichen schon mit der Reichhaltigkeit der reellen Zahlen im Intervall  $(0, 1)$ :

**SATZ 1.22.** Die Menge der reellen Zahlen  $r \in (0, 1) \subset \mathbb{R}$  ist überabzählbar unendlich.

**BEWEIS.** Angenommen die reellen Zahlen im Intervall können abgezählt werden, und die Zahlen werden als Binärbrüche der Reihe nach tabellarisch aufgeschrieben:

$$\begin{array}{l}
 r_1 = 0, x_{11} x_{12} x_{13} x_{14} x_{15} x_{16} \dots \\
 r_2 = 0, x_{21} x_{22} x_{23} x_{24} x_{25} x_{26} \dots \\
 r_3 = 0, x_{31} x_{32} x_{33} x_{34} x_{35} x_{36} \dots \\
 r_4 = 0, x_{41} x_{42} x_{43} x_{44} x_{45} x_{46} \dots \\
 \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \vdots \quad \ddots
 \end{array}$$

Dann kann die diagonal gelesene reelle Zahl  $r = 0, \overline{x_{11} x_{22} x_{33} x_{44}} \dots \in (0, 1)$ , wo jeweils die Ziffern zwischen 0 und 1 gewechselt werden, nicht in der Abzählung vorhanden sein, da sie sich mit jeder Zahl  $r_k$  an  $k$ -ter Stelle hinter dem Komma unterscheidet. Damit kann eine Abzählung nie vollständig sein, und somit ist die Annahme falsch. Daher ist die Menge der reellen Zahlen nach diesem **Diagonalargument von Cantor** nicht abzählbar also überabzählbar unendlich.  $\square$

Soweit davon auszugehen ist, dass die Teilmenge der in der Realität tatsächlich relevanten reellen Zahlen tatsächlich auch überabzählbar sind, wird es nie möglich sein, für alle Fragestellungen über solche Zahlen Lösungen in der Form von Programmen über einer gegebenen formalen Sprache zu formulieren. Gleichzeitig ist aber auch die Anzahl der formalen Sprachen sehr groß:

**KOROLLAR 1.23.** Die Menge der formalen Sprachen ist überabzählbar unendlich.

**BEWEIS.** Für jede reelle Zahl  $x \in \mathbb{R}$  mit Nachkommastellen  $r_1 r_2 \dots$  gibt es eine formale Sprache  $L_x$  über  $\Sigma_{\text{Zahl}}$ :

$$L_x = \{r_1 r_2 \dots r_n \in \Sigma_{\text{Zahl}}^* \mid x \text{ hat die ersten } n \text{ Nachkommastellen } r_1 \dots r_n\}$$

Beispielsweise ist  $L_\pi = \{1, 14, 141, 1415, 14159, 141592, 1415926, \dots\}$ . Damit ist die Anzahl der formalen Sprachen mindestens so groß, wie die Anzahl reeller Zahlen im Intervall  $(0, 1)$ , also aller möglichen Nachkommastellen in  $\mathbb{R}$ , zuzüglich der 0, und damit nach Satz 1.22 überabzählbar unendlich.  $\square$

### 1.3. Verknüpfungen von Sprachen und Entscheidbarkeit

Da Sprachen über Mengen definiert werden, sind die Mengenoperationen auch für Sprachen sehr relevant. Besonders wichtig sind die Vereinigung, der Durchschnitt und das Mengenkomplement, aber auch das gerade eingeführte Produkt und Kleenscher Abschluss.

**SATZ 1.24.** Sind  $L_1$  und  $L_2$  zwei formale Sprachen über den Alphabeten  $\Sigma_1$  und  $\Sigma_2$ , so gilt:

- (1) Die Vereinigung  $L_\cup = L_1 \cup L_2$  ist eine formale Sprache über dem Alphabet  $\Sigma_1 \cup \Sigma_2$ .
- (2) Der Schnitt  $L_\cap = L_1 \cap L_2$  ist eine formale Sprache über dem Alphabet  $\Sigma_1 \cup \Sigma_2$ .
- (3) Das Komplement  $\overline{L_k} = \Sigma_k^* \setminus L_k$ ,  $k = 1, 2$  ist formale Sprache über dem Alphabet  $\Sigma_k$ .
- (4) Das Produkt  $L_1 L_2 = \{w_1 w_2 \mid w_1 \in L_1, w_2 \in L_2\}$  ist formale Sprache über  $\Sigma_1 \cup \Sigma_2$ .
- (5) Kleensche Abschlüsse  $L_k^*$  und  $L_k^+$ ,  $k = 1, 2$  sind formale Sprachen über  $\Sigma_k$ .

**BEWEIS.** Zu (1) und (2): Die Vereinigung der Alphabeten  $\Sigma = \Sigma_1 \cup \Sigma_2$ , also zweier endlicher Mengen, ist wieder eine endliche Menge und damit ein Alphabet. Da sowohl  $\Sigma_k \subseteq \Sigma$  für  $k = 1, 2$ , sind  $L_1$  und  $L_2$  auch Sprachen über  $\Sigma$  und es gilt  $L_k \subseteq \Sigma_k^* \subseteq \Sigma^*$ , da die Teilmengenbeziehung in jeder Mengengenpotenz und damit auch in deren Vereinigung gilt. Damit sind auch  $L_1 \cup L_2 \subseteq \Sigma^*$  und  $L_1 \cap L_2 \subseteq \Sigma^*$  und damit Sprachen über  $\Sigma = \Sigma_1 \cup \Sigma_2$ .

Zu (3): Da  $\Sigma_k^* \setminus L_k \subseteq \Sigma_k^*$  nach Definition der Mengendifferenz, ist  $\overline{L_k} \subseteq \Sigma_k^*$  und so Sprache über  $\Sigma_k$ .

Zu (4): Für  $L = L_1 \cup L_2$  ist  $L_1 \subseteq L$  und  $L_2 \subseteq L$ .  $L$  ist Sprache über  $\Sigma = \Sigma_1 \cup \Sigma_2$  nach Punkt 1. Damit ist jedes Wort  $w \in L \subseteq \Sigma^*$  in einem  $w \in \Sigma^k$  für ein bestimmtes  $k$  enthalten. Ebenso ist damit  $ww \in \Sigma^k \Sigma^k = \Sigma^{2k} \subseteq \Sigma^*$ . Damit ist  $LL \subseteq \Sigma^*$  und damit  $L_1 L_2 \subseteq LL \subseteq \Sigma^*$  Sprache über  $\Sigma$ .

Zu (5): Zunächst ist  $\varepsilon \in \Sigma_k^*$ , somit reicht es für  $L_k^+$  zu argumentieren. Jedes Wort  $w \in L_k^+$  ist in  $w \in L_k^n$  für ein bestimmtes  $n$ . Damit gibt es ein Teilwort  $m^n = w$  mit  $m \in L_k$ . Da  $L_k \subseteq \Sigma_k^*$  gibt es ein  $p$ , so dass  $m \in \Sigma_k^p$ . Damit ergibt sich, dass  $m^n \in (\Sigma_k^p)^n$  liegt und damit in  $\Sigma_k^{np} \subseteq \Sigma_k^*$ . Damit dies für alle Worte in  $L_k^+$  gilt, ist  $L_k^+ \subseteq \Sigma_k^*$  und damit Sprache über  $\Sigma_k$ .  $\square$

Es ist ersichtlich, dass eine Abzählbarkeit von  $L_1$  und  $L_2$  auch mit entsprechenden Anpassungen der Abzählung auch für Vereinigung, Schnitt und Produkt gilt. Die Abzählbarkeit des Abschlusses wurde in Satz 1.21 bewiesen, nur dass hier Programme dann wieder als formale Sprachen über dem ursprünglichen Alphabet interpretiert werden. Deutlich schwieriger ist die Frage nach der Herleitung der Abzählbarkeit des Komplements zu beantworten, doch brauchen wir uns diese Fragen eigentlich überhaupt nicht mehr zu stellen, da jede formale Sprache über einem endlichen Alphabet nach Satz 1.18 abzählbar ist.

Tatsächlich ist diese Frage nur unbefriedigend beantwortet, da die reine Existenz einer Abzählbarkeit nichts über die tatsächliche Methode der Abzählung sagt. Die Erkenntnisse sind damit nicht konstruktiv. Es gibt viele Fragestellungen, die bisher nicht konstruktiv zu beantworten sind, wie die Folgende:

BEISPIEL 1.25. Gibt es zwei irrationale Zahlen  $x, y \in \mathbb{R} \setminus \mathbb{Q}$ , deren Potenz rational ist, also  $x^y \in \mathbb{Q}$  ist? Wenn die Potenz  $a^b$  der irrationalen Zahlen  $a = \sqrt{2}$  und  $b = \sqrt{2}$  rational ist, so ist die Frage mit  $x = a$  und  $y = b$  beantwortet. Ist die Potenz  $c = a^b$  irrational, so ist aber

$$c^b = (a^b)^b = a^{b \cdot b} = a^{\sqrt{2} \cdot \sqrt{2}} = a^2 = (\sqrt{2})^2 = 2 \in \mathbb{Q},$$

und damit wäre dann die Frage mit  $x = c$  und  $y = b$  beantwortet. Da aber noch nicht bekannt ist, ob  $a^b = \sqrt{2}^{\sqrt{2}}$  rational ist, ist nicht klar, ob nun die Lösung  $a^b$  oder  $c^b$  lautet. Wir können aus der logischen Regel **tertium non datur** nur schließen, dass die Aussage stimmt, weil eine der beiden Kombinationen die Lösung ist, jedoch ist unbekannt, welche der beiden die Lösung ist. Damit wird durch Anwendung der Regel die **konstruktivistische Logik** verlassen. Dies ist auch vorab ein Beispiel für ein **nicht entscheidbares Problem**, das in Definition 1.29 eingeführt wird.

Aus diesem Grund wird die einfache und nicht konstruktivistische Erkenntnis einer Abzählbarkeit von einer konstruktiven und praktischen Aufzählbarkeit unterschieden:

DEFINITION 1.26. Eine Menge oder Sprache  $M$  ist **aufzählbar** oder **rekursiv aufzählbar**, wenn eine surjektive Abbildung  $f : \mathbb{N} \rightarrow M$  bekannt ist, die nach endlichen Schritten für jedes  $n \in \mathbb{N}$  die Berechnung von  $f(n)$  ermöglicht, falls  $M \neq \emptyset$ . Daraus ergibt sich eine **Aufzählung** von  $M$  durch die Folge  $(f(1), f(2), \dots)$ .

BEMERKUNG 1.27. Die genaue Bedeutung der Berechenbarkeit in Definition 1.26 wird im Laufe der Vorlesung in Definition 4.1 im Sinne eines formulierbaren und endlich ausführbaren Programmes noch genauer erklärt. Die Verwendung der Bezeichnung „aufzählbar“ legt den Schwerpunkt auf die Existenz der Aufzählung als berechenbare Funktion, während die Bezeichnung „rekursiv aufzählbar“ den Schwerpunkt auf die Existenz eines Programms legt, was aber genau äquivalent ist. Zwischen den Bezeichnungen „aufzählbar“ zu „abzählbar“ ist der relevante Unterschied in der konstruktiven Kenntnis der Aufzählbarkeit im Gegensatz von der nicht konstruktiven Gewissheit der Abzählbarkeit.

SATZ 1.28. Sei  $\Sigma$  ein Alphabet, dann ist  $\Sigma^*$  aufzählbar.

BEWEIS. Die Konstruktion aus Satz 1.16 zur Abzählbarkeit von  $\Sigma^*$  ist schon eine konstruktive Aufzählung von  $\Sigma^*$ . Die nicht zugeordneten natürlichen Zahlen werden beispielsweise auf das jeweils zuletzt zugeordneten Wort abgebildet.  $\square$

Die Übertragung der Eigenschaft der Aufzählbarkeit muss im Gegensatz bei den Verknüpfungen mit Angabe eines ausführbaren Algorithmus erfolgen. So kann wie in Beispiel 1.15 bei der Vereinigung abwechselnd die eine oder die andere Aufzählung verwendet werden, wie bei der Aufzählung der ganzen Zahlen und beim Produkt die Aufzählung beispielsweise mit dem Diagonalisierungsverfahren von Cantor wie bei den rationalen Zahlen erfolgen.

Es ist aber kein Verfahren bekannt, wie aus einer allgemeinen Aufzählung einer Sprache konstruktiv eine Aufzählung des Komplements oder bei zwei aufgezählten Sprachen für deren Schnitt abgeleitet werden kann. Dies hat eine unmittelbare Folge auf das Entscheidungsproblem des Wortproblems einer Sprache.

DEFINITION 1.29. Das **Entscheidungsproblem** bezeichnet die Frage, ob für ein Problem ein ausführbares Verfahren angegeben werden kann, mit dem in endlich vielen Schritten eine Entscheidung für das Problem bestimmt wird. Existiert so ein Verfahren, so ist das Problem **entscheidbar**, kann es so ein Verfahren nicht geben, so ist es **nicht entscheidbar**. Gibt es ein Verfahren, das nach endlich vielen Schritten die Entscheidung für eine Klasse von möglichen Antworten bestimmt, so wird das Entscheidungsproblem als **semi-entscheidbar** bezeichnet.

Es geht jetzt also um die Entscheidbarkeit des folgenden Wortproblems:

**DEFINITION 1.30.** Sei  $L$  eine Sprache über  $\Sigma$  und  $w \in \Sigma^*$ . Das **Wortproblem** bezeichnet die Frage, ob  $w$  Teil der Sprache ist, also entweder  $w \in L$  oder  $w \notin L$  gilt.

Liegt eine Aufzählbarkeit einer Sprache und deren Komplement vor, kann die Entscheidbarkeit des Wortproblems direkt beantwortet werden:

**SATZ 1.31.** Sind  $L$  und  $\bar{L}$  aufzählbare Sprachen über dem Alphabet  $\Sigma$ , so ist das Wortproblem  $w \stackrel{?}{\in} L$  für ein  $w \in \Sigma^*$  entscheidbar. Dann werden  $L$  und  $\bar{L}$  als **entscheidbare Sprachen** oder **rekursive Sprachen** bezeichnet.

**BEWEIS.** Es seien  $f_L : \mathbb{N} \rightarrow L$  und  $f_{\bar{L}} : \mathbb{N} \rightarrow \bar{L}$  die Aufzählungen von  $L$  und  $\bar{L}$ . Abwechselnd wird aufsteigend die beginnend von  $k = 1$  das Wort  $w$  mit  $f_L(k)$  und  $f_{\bar{L}}(k)$  verglichen. Nach endlicher Anzahl von Schritten ist  $f_L(k) = w$ , dann ist  $w \in L$ , oder  $f_{\bar{L}}(k) = w$ , dann ist  $w \notin L$ .  $\square$

Ist jedoch für eine aufzählbare Sprache  $L$  keine Aufzählung für das Komplement  $\bar{L}$  bekannt, so kann das gerade beschriebene Verfahren nur für die Aufzählung von  $L$  verwendet werden. Damit endet es nach endlich vielen Schritte genau dann, wenn  $w \in L$ . Ist hingegen  $w \notin L$ , so endet das Verfahren nicht. Daher kann das Wortproblem aufzählbarer Sprachen als semi-entscheidbar klassifiziert werden. Sind sowohl das Wortproblem  $w \in L$  und das Wortproblem  $w \in \bar{L}$  semi-entscheidbar, so sind  $w \in L$  und  $w \in \bar{L}$  entscheidbar.

**SATZ 1.32.** Jede entscheidbare Sprache ist aufzählbar.

**BEWEIS.** Jede formale Sprache  $L$  basiert auf einem Alphabet  $\Sigma_L$ . Nach Satz 1.28 ist der Abschluss  $\Sigma_L^*$  mit  $f_{\Sigma_L^*}$  aufzählbar und nach Definition 1.10 ist  $L \subseteq \Sigma_L^*$ . Entweder ist die Sprache  $L$  leer, oder es gibt ein Wort  $w_0 \in L$ . Wenn  $L$  entscheidbar ist, so kann für jedes  $n \in \mathbb{N}$  in endlichen Schritten bestimmt werden, ob  $f_{\Sigma_L^*}(n) \in L$  ist. Wenn ja, so ist  $f_L(n) = f_{\Sigma_L^*}(n)$ , und sonst  $f_L(n) = w_0$ .  $\square$

Damit ist jede rekursive Sprache eine rekursiv aufzählbare Sprache. Umgekehrt führt eine Aufzählbarkeit aber nicht zu einer Entscheidbarkeit, sondern ist nur eine Semi-Entscheidbarkeit, da nicht klar ist, ob eine Lücke in beispielsweise einer lexikographischen Ordnung beziehungsweise einer Ordnung nach Stellenwert später noch aufgefüllt wird. Eine weitere thematisch nahe liegende und hilfreiche Illustration zwischen Entscheidbarkeit und Semi-Entscheidbarkeit liefert das folgende Problem:

**DEFINITION 1.33.** Die Collatz-Funktion  $f(n) : \mathbb{N} \rightarrow \mathbb{N}$  lautet:

$$f(n) = \begin{cases} \frac{n}{2}, & \text{wenn } n \text{ gerade} \\ 3n + 1, & \text{sonst.} \end{cases}$$

Das **Collatz-Problem** ist dann die Frage, ob es für ein gegebenes  $n \in \mathbb{N}$  eine Konstante  $k \in \mathbb{N}_0$  gibt, so dass die  $k$ -malig wiederholte Ausführung der Collatz-Funktion  $f$

$$f^k(n) = \underbrace{f(f(\dots f(n) \dots))}_{k \text{ Ausführungen}} = 1$$

erstmal die Zahl 1 ergibt.

**BEISPIEL 1.34.** Beginnend von  $n = 6$  ergeben sich beispielsweise diese Auswertungen:

$$6 \xrightarrow{f} 3 \xrightarrow{f} 10 \xrightarrow{f} 5 \xrightarrow{f} 16 \xrightarrow{f} 8 \xrightarrow{f} 4 \xrightarrow{f} 2 \xrightarrow{f} 1 \xrightarrow{f} 4 \xrightarrow{f} 2 \xrightarrow{f} 1 \xrightarrow{f} \dots$$

Wenn einmal die Zahl 1 erreicht wurde, so wiederholen sich die Werte 1, 4, 2 immer wieder. Für  $n = 6$  wird also die Zahl 1 zum ersten Mal nach  $k = 8$  Auswertungen erreicht.

SATZ 1.35. *Das Collatz-Problem ist semi-entscheidbar.*

BEWEIS. Die Definition des Collatz-Problems entspricht exakt der Anforderung an die Semi-Entscheidbarkeit: Wenn nach endlichen  $k$  Schritten die Zahl 1 erreicht wird, so ist das Problem positiv beantwortet.  $\square$

Das Collatz-Problem kann direkt in eine Collatz-Sprache über  $\Sigma_{\text{Zahl}}$  übertragen werden:

$$L_{\text{Collatz}} = \{n \in \mathbb{N} \mid \exists k \in \mathbb{N}_0 : f^k(n) = 1\}$$

Das Wortproblem auf dieser Sprache ist damit hier nach Definition des Problems statt einer Aufzählung ebenso mindestens semi-entscheidbar. Ob das Problem auch entscheidbar ist, konnte bisher niemand beantworten. Die naive Methode des Ausprobierens, ob es überhaupt ein  $w \in \mathbb{N}$  mit  $w \notin L_{\text{Collatz}}$  gibt, hat trotz intensiver Suche bisher nicht geendet. Eine Antwort auf das sogenannte **Leerheitsproblem** der Sprache, was meint, ob bewiesen  $\bar{L}_{\text{Collatz}} = \emptyset$  gilt, wie es allgemein vermutet wird, verspricht für das Collatz-Problem viel Ruhm– und ein hohes Preisgeld.

DEFINITION 1.36. Das **Halteproblem** ist die Fragestellung, ob die Ausführung eines Programms  $p$  bei gegebenen Eingabedaten  $x$  nach endlichen Schritten terminiert.

Das Halteproblem ist die verallgemeinerte Fragestellung zum Collatz-Problem, das für beliebige formuliert wird. Entsprechend ist die äquivalente Sprache

$$L_{\text{Halteproblem}} = \{(p, x) \in \Sigma_{\text{Unicode}}^* \times \Sigma_{\text{Unicode}}^* \mid p(x) \text{ terminiert nach endlichen Schritten}\}$$

nur semi-entscheidbar, da durch Ausführung nur  $(p, x) \in L_{\text{Halteproblem}}$  gezeigt werden kann, hingegen konnte Alan Turin beweisen, dass es keinen Algorithmus gibt, der die Entscheidung  $(p, x) \notin L_{\text{Halteproblem}}$  für beliebiger  $p$  und  $x$  in endlicher Zeit beantworten kann. Im Gegensatz zur ungeklärten Frage, ob  $\bar{L}_{\text{Collatz}}$  überhaupt ein Element besitzt, kennen wir viele Programme, die nicht terminieren, und daher ist sicher  $\bar{L}_{\text{Halteproblem}} \neq \emptyset$ .

### 1.4. Aufgaben

**AUFGABE 1.** Gegeben sei das Alphabet  $\Sigma = \{a, el, en, g, l, ste\}$ . Welche Worte liegen in  $\Sigma^4$ ?

$w_1 = galgen, w_2 = stelle, w_3 = sagen, w_4 = lagen, w_5 = allen, w_6 = aalen$

**AUFGABE 2.** Gegeben sei das Alphabet  $\Sigma = \{be, che, ei, ler, ne, r\}$ . Welche Worte liegen in  $\Sigma^4$ ?

$w_1 = reicher, w_2 = reiben, w_3 = leiner, w_4 = reiber, w_5 = reiher, w_6 = reiner$

**AUFGABE 3.** Gegeben sei das Alphabet  $\Sigma = \{e, en, in, r, t, u\}$ . Welche Worte liegen in  $\Sigma^5$ ?

$w_1 = reiner, w_2 = teurer, w_3 = treuer, w_4 = teuren, w_5 = retten, w_6 = teuer$

**AUFGABE 4.** Gegeben sei das Alphabet  $\Sigma = \{e, el, g, i, ne, st\}$ . Welche Worte liegen in  $\Sigma^5$ ?

$w_1 = geiste, w_2 = steige, w_3 = geigen, w_4 = leiste, w_5 = stege, w_6 = eigen$

**AUFGABE 5.** Gegeben sei das Alphabet  $\Sigma = \{e, g, in, l, s, ter\}$ . Welche Worte liegen in  $\Sigma^*$ ?

$w_1 = tester, w_2 = seile, w_3 = lines, w_4 = segel, w_5 = seinen, w_6 = erster$

**AUFGABE 6.** Gegeben sei das Alphabet  $\Sigma = \{ber, e, he, in, sch, u\}$ . Welche Worte liegen in  $\Sigma^*$ ?

$w_1 = herber, w_2 = heine, w_3 = scheune, w_4 = heber, w_5 = scheu, w_6 = ueber$

**AUFGABE 7.** Bestimmen Sie die Wörter dieser Sprache:

$$L = \{acx^m(zq)^n \mid n \in \{0, 1\}, m \in \{1, 2\}\}$$

**AUFGABE 8.** Bestimmen Sie die Wörter dieser Sprache:

$$L = \{(ar)^2 r^k (yb)^m \mid m \in \{0, 1\}, k \in \{0, 1, 2\}\}$$

**AUFGABE 9.** Bestimmen Sie die Wörter dieser Sprache:

$$L = \{(b^m a)^l za \mid m \in \{0, 1\}, l \in \{1, 2, 3\}\}$$

**AUFGABE 10.** Bestimmen Sie die Wörter dieser Sprache:

$$L = \{(b^l q)^n yqy^k y \mid l \in \{0, 1\}, n \in \{0, 1\}, k \in \{0, 1\}\}$$

AUFGABE 11. Bestimmen Sie die Wörter dieser Sprache:

$$L = \left\{ \left( y (r^2)^m \right)^m x^n z a \mid m \in \{0, 1\}, n \in \{0, 2, 3\} \right\}$$

AUFGABE 12. Bestimmen Sie die Wörter dieser Sprache:

$$L = \left\{ (x a^n b)^l y y \mid n \in \{0, 1\}, l \in \{0, 1, 2\} \right\}$$

AUFGABE 13. Gegeben sei das Alphabet  $\Sigma = \{a, \text{gen}, i, \text{re}\}$  mit Aufzählung in dieser Reihenfolge. Bestimmen Sie die Zahlen  $n$  nach Stellenwert mit Bild  $f(n)$  der Wörter **regen**, **aare** und die Worte mit Stellenwert 15, 118.

AUFGABE 14. Gegeben sei das Alphabet  $\Sigma = \{\text{ber}, \text{ern}, \text{t}, \text{te}\}$  mit Aufzählung in dieser Reihenfolge. Bestimmen Sie die Zahlen  $n$  nach Stellenwert mit Bild  $f(n)$  der Wörter **berber**, **ernte** und die Worte mit Stellenwert 9, 74.

AUFGABE 15. Gegeben sei das Alphabet  $\Sigma = \{e, h, r, \text{ste}\}$  mit Aufzählung in dieser Reihenfolge. Bestimmen Sie die Zahlen  $n$  nach Stellenwert mit Bild  $f(n)$  der Wörter **steh**, **rehe** und die Worte mit Stellenwert 45, 1417.

AUFGABE 16. Gegeben sei das Alphabet  $\Sigma = \{h, \text{he}, \text{re}, \text{ste}\}$  mit Aufzählung in dieser Reihenfolge. Bestimmen Sie die Gödelnummer  $c(w)$  der Wörter **steh**, **reste** und die Worte mit Gödelnummer 144, 1500.

AUFGABE 17. Gegeben sei das Alphabet  $\Sigma = \{e, l, \text{ste}, \text{te}\}$  mit Aufzählung in dieser Reihenfolge. Bestimmen Sie die Gödelnummer  $c(w)$  der Wörter **este**, **elle** und die Worte mit Gödelnummer 720, 12600.

AUFGABE 18. Gegeben sei das Alphabet  $\Sigma = \{e, n, \text{ern}, g, m\}$  mit Aufzählung in dieser Reihenfolge. Bestimmen Sie die Gödelnummer  $c(w)$  der Wörter **gern**, **engen** und die Worte mit Gödelnummer 120, 42000.





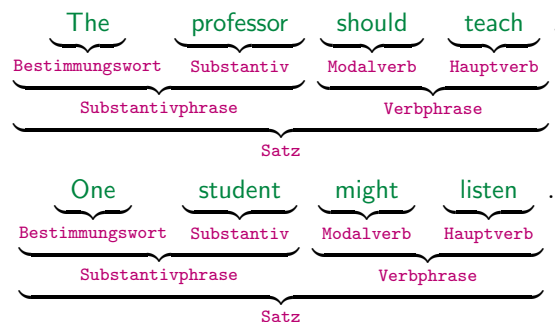
## KAPITEL 2

# Grammatiken

Bisher wurden formale Sprachen mit Hilfe von Mengen und logischen Prädikaten beschrieben. Eine besonders sinnvolle und mächtige Beschreibung von Sprachen sind Grammatiken, die auch eine Klassifikation von wichtigen Sprachklassen ermöglichen. Mit Hilfe von wenigen Regeln beschreiben diese rekursiv, welche Worte mit einer Sprache gebildet werden können.

### 2.1. Grammatiken

Besonders gut lässt sich das Prinzip einer Grammatik an einfachen englischen Sätzen illustrieren:



Die Konstruktion beginnt mit einem Startsymbol  $S = \text{Satz}$ . Sätze werden mit diesen Regeln  $R$  gebildet:

- Ein **Satz** besteht aus einer **Substantivphrase** und einer **Verbphrase**.
- Eine **Substantivphrase** besteht aus optionalem **Bestimmungswort** und einem **Substantiv**.
- Eine **Verbphrase** besteht aus optionalem **Modalverb** und einem **Hauptverb**.
- Ein **Bestimmungswort** ist **the** oder **one**.
- Ein **Substantiv** ist **student** oder **professor**.
- Ein **Modalverb** ist **should** oder **might**.
- Ein **Hauptverb** ist **listen** oder **teach**.

Darin wurden diese Variablen  $V$  und Symbole  $T$  verwendet:

$$V = \{\text{Satz, Substantivphrase, Verbphrase, Bestimmungswort, Substantiv, Modalverb, Hauptverb}\}$$

$$T = \{\text{the, one, student, professor, should, might, listen, teach}\}.$$

Die formale Definition einer Grammatik ist damit:

**DEFINITION 2.1.** Eine **Grammatik** ist ein Tupel  $G = (V, T, R, S)$ , wo

$V$  das Alphabet der **Variablen**,

$T$  das Alphabet der **Terminalen Symbole** mit  $V \cap T = \emptyset$ ,

$R = \{r_1, \dots, r_n\}$  die endliche Menge der **Regeln**  $r_k : (V \cup T)^* \setminus T^* \rightarrow (V \cup T)^*$ , und

$S \in V$  das **Startsymbol** ist.

Die Regeln von Grammatiken werden auch **Produktionen** genannt. Eine Grammatik definiert dann eine Sprache über Ableitungen:

DEFINITION 2.2. Sei  $G = (V, T, R, S)$  eine Grammatik. Eine **Ableitung** ist die Anwendung einer Regel  $r \in R$  der Form  $r : (V \cup T)^* T^* \rightarrow (V \cup T)^*$  mit  $a \mapsto b$  auf das Wort  $w_1 \in (V \cup T)^*$  zum Wort  $w_2 \in (V \cup T)^*$ , geschrieben  $w_1 \xrightarrow{r} w_2$ , wenn es  $x, y \in (V \cup T)^*$  gibt, so dass

$$\begin{array}{ccccc} w_1 & = & x & a & y \\ & & \Downarrow r & & \Downarrow r \\ w_2 & = & x & b & y \end{array}$$

ist. Eine **transitive Ableitung**  $w_1 \xRightarrow{*} w_n$  ist die Anwendung keiner oder beliebig vieler Regeln  $r \in R$ , um von  $w_1$  auf  $w_n$  zu schließen. Die **Sprache einer Grammatik**  $L(G)$  ist definiert als die Menge aller möglichen Wörter aus terminalen Symbolen, die durch die Regeln der Grammatik transitiv aus dem Startsymbol  $S$  abgeleitet werden können:

$$L(G) := \{w \in T^* \mid S \xRightarrow{*} w\}$$

BEISPIEL 2.3. Eine **Grammatik für boolesche Terme** ist  $G_{\text{Logik}} = (V, T, R, S)$  mit

$$\begin{aligned} V &= \{\text{Term}, \text{Literal}, \text{Variable}\}, \\ T &= \Sigma_{\text{Logik}} = \{\vee, \wedge, \neg, (, ), 0, 1, a, \dots, z\}, \\ R &= \{r_1, r_2, r_3, r_4\}, \text{ wo} \\ r_1 : \quad & \text{Term} \mapsto \text{Literal} \mid \text{Variable} \mid \neg \text{Term} \mid (\text{Term}), \\ r_2 : \quad & \text{Term} \mapsto \text{Term} \vee \text{Term} \mid \text{Term} \wedge \text{Term}, \\ r_3 : \quad & \text{Literal} \mapsto 0 \mid 1, \\ r_4 : \quad & \text{Variable} \mapsto a \mid \dots \mid z, \text{ und} \\ S &= \text{Term}. \end{aligned}$$

In diesem Beispiel wurde schon eine verkürzte Schreibweise zur Zusammenfassung von (meist ähnlichen) Regeln mit identischer linker Seite verwendet. Mit Hilfe einem senkrechten Strichs  $|$  werden Alternativen möglicher Produkte beschrieben. Die Regel

$$r_2 : \text{Term} \mapsto \text{Term} \vee \text{Term} \mid \text{Term} \wedge \text{Term}$$

besteht hier dann beispielsweise ausgeschrieben aus zwei Regeln

$$\begin{aligned} r_{2.1} : \quad & \text{Term} \mapsto \text{Term} \vee \text{Term} \text{ und} \\ r_{2.2} : \quad & \text{Term} \mapsto \text{Term} \wedge \text{Term}. \end{aligned}$$

Eine Ableitung des Terms  $S \xRightarrow{*} (a \wedge b) \vee c \in L(G_{\text{Logik}})$  kann dann beispielsweise so ablaufen:

$$\begin{array}{ccccccc} & & & & S = \text{Term} & & \\ & & & & \Downarrow & & \\ r_{2.1} & & & & \text{Term} & & \vee & \text{Term} \\ & & & & \Downarrow & & \Downarrow & \\ r_{1.4}, r_{1.2} & & & & (\text{Term}) & & \vee & \text{Variable} \\ & & & & \Downarrow & & \Downarrow & \\ r_{2.2}, r_4 & & & & (\text{Term} \wedge \text{Term}) & & \vee & c \\ & & & & \Downarrow & & \Downarrow & \\ r_{1.2}, r_{1.2} & & & & (\text{Variable} \wedge \text{Variable}) & & \vee & c \\ & & & & \Downarrow & & \Downarrow & \\ r_4, r_4 & & & & (a \wedge b) & & \vee & c \end{array}$$

**BEISPIEL 2.4.** Grammatiken zu den Sprachen aus Beispiel 1.11 auf dem Alphabet  $\Sigma = \{0, 1, 2\}$  :

$$(1) M_3 = \{0\} \cup \{1\} \times \{0, 1\}^* = \{0, 1, 10, 11, 100, 101, 110, 111, 1000, 1001, \dots\} = L(G):$$

$$G = (V, T, R, S)$$

$$V = \{\text{Start}, A\},$$

$$T = \{0, 1\},$$

$$R = \{r_1, r_2\}, \text{ wo}$$

$$r_1 : \text{Start} \mapsto 0 \mid 1 \mid 1A,$$

$$r_2 : A \mapsto 0 \mid 1 \mid 0A \mid 1A,$$

$$S = \text{Start}.$$

$$(2) M_2 = \{0^n 1^n \mid n \in \mathbb{N}\} = \{01, 0011, 000111, 00001111, 0000011111, \dots\} = L(G):$$

$$G = (V, T, R, S) \text{ mit}$$

$$V = \{S\},$$

$$T = \{0, 1\},$$

$$R = \{r_1\}, \text{ wo}$$

$$r_1 : S \mapsto 0S1 \mid 01,$$

$$S = S.$$

$$(3) M_1 = \{0^n 1^n 2^n \mid n \in \mathbb{N}\} = \{012, 001122, 000111222, 000011112222, \dots\} = L(G):$$

$$G = (V, T, R, S) \text{ mit}$$

$$V = \{S, B, C\},$$

$$T = \{0, 1, 2\},$$

$$R = \{r_1, r_2, r_3, r_4, r_5, r_6\}, \text{ wo}$$

$$r_1 : S \mapsto 0SBC \mid 0BC,$$

$$r_2 : CB \mapsto BC,$$

$$r_3 : 0B \mapsto 01,$$

$$r_4 : 1B \mapsto 11,$$

$$r_5 : 1C \mapsto 12,$$

$$r_6 : 2C \mapsto 22,$$

$$S = S.$$

$$(4) M_0 = \{w \in \Sigma^* \mid |w| = 2^n, n \in \mathbb{N}\} = \{0, 1, 2, 00, 01, \dots, 21, 22, 0000, 0001, \dots\} = L(G):$$

$$G = (V, T, R, S) \text{ mit}$$

$$V = \{\text{Start}, o, >, <, \#, *\},$$

$$T = \{0, 1, 2\},$$

$$R = \{r_1, r_2, r_3, r_4, r_5, r_6, r_7\}, \text{ wo}$$

$$r_1 : \text{Start} \mapsto \# < o \#,$$

$$r_2 : \# < \mapsto \# > \mid *,$$

$$r_3 : > o \mapsto oo > ,$$

$$r_4 : > \# \mapsto < \#,$$

$$r_5 : o < \mapsto < o ,$$

$$r_6 : * o \mapsto 0 * \mid 1 * \mid 2 * ,$$

$$r_7 : * \# \mapsto \varepsilon$$

$$S = \text{Start}.$$

## 2.2. Die Chomsky-Hierarchie

DEFINITION 2.5. Die Chomsky-Hierarchie unterteilt Grammatiken  $G = (V, T, R, S)$  formaler Sprachen in vier unterschiedliche Typen mit steigenden Anforderungen an die Regeln:

**Typ-0:** In einer **allgemeinen Chomsky-Grammatik** oder **Typ-0 Grammatik** sind alle Regeln

$$r_k : (V \cup T)^* \setminus T^* \rightarrow (V \cup T)^*$$

zugelassen.

**Typ-1:** In einer **kontextsensitiven Grammatik** oder **Typ-1 Grammatik** müssen die Regeln Prefix und Postfix vor und nach der Ersetzung erhalten, und die Länge des Wortes erhalten oder wachsen lassen, also

$$r_k : uAv \mapsto uvw$$

mit  $u, v \in (V \cup T)^*$ ,  $A \in V$  und  $w \in (V \cup T)^+$ . Einmalig ist die Regel  $S \mapsto \varepsilon$  erlaubt, dann darf aber  $S$  auf keiner rechten Seite einer anderen Regel auftreten.

**Typ-2:** In einer **kontextfreien Grammatik** oder **Typ-2 Grammatik** dürfen Regeln links nur aus einer Variablen bestehen, also

$$r_k : A \mapsto w$$

mit  $A \in V$  und  $w \in (V \cup T)^+$ . Einmalig ist die Regel  $S \mapsto \varepsilon$  erlaubt, dann darf aber  $S$  auf keiner rechten Seite einer anderen Regel auftreten.

**Typ-3:** In einer **regulären Grammatik** oder **Typ-3 Grammatik** dürfen Regeln links nur aus einer Variablen bestehen, und auf der rechten Seite aus einem terminalen Symbol und optional einer Variable, die bei allen Regeln nur links für **links-lineare Grammatiken** oder nur rechts für **rechts-lineare Grammatiken** stehen darf:

$$r_k : A \mapsto aB \text{ (rechts-linear)} \quad \text{oder} \quad A \mapsto Ba \text{ (links-linear)} \quad \text{oder} \quad A \mapsto a$$

mit  $A, B \in V$ ,  $a \in T$ . Einmalig ist die Regel  $S \mapsto \varepsilon$  erlaubt, dann darf aber  $S$  auf keiner rechten Seite einer anderen Regel auftreten.

**BEMERKUNG 2.6.** Die Regeln von Grammatiken mit höherem Typ erfüllen immer auch die Bedingungen von tieferem Typ. Eine Grammatik vom Chomsky Typ-2 ist beispielsweise auch immer eine Grammatik von Typ-1 und Typ-0. Daher wird bei der Frage nach dem Typ grundsätzlich nach dem höchsten erreichbaren Typ für eine Grammatik gefragt, da höhere Typen effizienter zu verarbeiten und auszuwerten sind.

DEFINITION 2.7. Eine formale Sprache  $L$  ist von einem bestimmten **Chomsky-Typ** und entsprechend kontextsensitiv, kontextfrei oder regulär, wenn es eine Grammatik  $G$  gibt, die die Sprache  $L = L(G)$  erzeugt.

**BEMERKUNG 2.8.** Entsprechend der Überlegungen aus Bemerkung 2.6 erfüllen damit die Grammatiken von Sprachen höheren Typs auch die Anforderungen niedrigerer Typen. Damit sind beispielsweise reguläre Sprachen auch kontextsensitiv und kontextfrei. Außerdem kann eine Sprache durch unterschiedliche Grammatiken erzeugt werden, also eine reguläre Sprache auch durch eine kontextsensitive Sprache. Es ist eine Herausforderung eine Grammatik von höchstmöglichen Typ zu einer Sprache zu finden, die dann den Typ der Sprache definiert, wie folgendes Beispiel darstellt:

BEISPIEL 2.9. In Beispiel 2.4 sind die Grammatiken der Sprachen  $M_3$ ,  $M_2$  und  $M_0$  genau von Typ-3, Typ-2 und Typ-0. Nur zur Sprache  $M_1$  gibt es ein Problem mit der Regel

$$r_2 : \text{CB} \mapsto \text{BC}$$

Diese entspricht nicht der Anforderung

$$r_k : uAv \mapsto uvw$$

mit  $u, v \in (V \cup T)^*$ ,  $A \in V$  und  $w \in (V \cup T)^+$  für eine Typ-1 Grammatik und ist damit eine Typ-0 Grammatik. Das liegt hier aber nur daran, dass hier die Vertauschung in einem Schritt dargestellt wurde. Mit zwei weiteren Variablen und vier Regeln für die Vertauschung kann für die Sprache auch eine Typ-1 Grammatik angegeben werden:

$G = (V, T, R, S)$  mit

$$V = \{\text{S}, \text{B}, \text{C}, \text{X}, \text{Y}\},$$

$$T = \{0, 1, 2\},$$

$$R = \{r_1, r_2, r_3, r_4, r_5, r_6\}, \text{ wo}$$

$$r_1 : \text{S} \mapsto 0\text{SBC} \mid 0\text{BC},$$

$$r_2 : \text{CB} \mapsto \text{CX},$$

$$r_3 : \text{CX} \mapsto \text{YX},$$

$$r_4 : \text{YX} \mapsto \text{YC},$$

$$r_5 : \text{YC} \mapsto \text{BC},$$

$$r_6 : 0\text{B} \mapsto 0\text{1},$$

$$r_7 : 1\text{B} \mapsto 1\text{1},$$

$$r_8 : 1\text{C} \mapsto 1\text{2},$$

$$r_9 : 2\text{C} \mapsto 2\text{2},$$

$$S = \text{S}.$$

### 2.3. Grammatiken allgemeiner (Typ-0) und kontextsensitiver (Typ-1) Sprachen

Die Sprachen von Typ-0 und Typ-1 haben in der Anwendung keine so große Bedeutung, ihre theoretischen Eigenschaften und die späteren Rechnermodelle zur Verarbeitung sind dagegen wichtig.

SATZ 2.10. Die Sprache einer allgemeinen, also Typ-0, Grammatik ist (rekursiv) aufzählbar.

BEWEIS. Ist die Sprache nicht leer und auch das Startsymbol kein terminales Symbol, so ist dazu eine Aufzählung zu konstruieren. Sei  $w \in L$  ein nach endlich vielen Ableitungen aus der Grammatik erzeugtes Wort der Sprache. Durch eine der  $r$  Regeln der Sprache kann die Sprache ein Wort oder ein Tupel aus Variablen und terminalen Symbolen verlängert werden. Sei  $m$  die maximale Verlängerung eines Wortes über alle Regeln der Sprache. Damit ist die Länge eines Wortes nach  $n$  Ableitungen bis zu  $1 + n \cdot m$ , also beschränkt. Nach  $n$  Ableitungen gibt es also höchstens  $(1 + n \cdot m) \cdot r$  Möglichkeiten eine der  $r$  Regeln ab einer Stelle des maximal  $1 + n \cdot m$  Positionen langen Tupels anzuwenden, wird beispielsweise die Regel  $\varrho$  auf die  $\mu$ -te Stelle angewendet, so kann dies in der Zahl  $\varrho + (r + 1) \cdot \mu$  eindeutig kodiert werden. Die Konstruktion der der Aufzählung  $f : \mathbb{N} \rightarrow L$  kann für  $f(n)$  so mit dem Prinzip der Gödelnummern aus der Primzahlzerlegung

$$n = \prod_{k=1}^s p_k^{\nu_k}$$

für die Folge der Primzahlen  $(p_k)$  bestimmt werden: Im  $k$ -ten Schritt wird bei  $\nu_k > 0$  die Regel  $\varrho \equiv \nu_k \pmod{r+1}$  auf das in vorherigen Schritten gebildete Wort an Stelle  $\mu = \frac{\nu_k - \varrho}{r+1}$  angewendet. Ist die Regel nicht anwendbar,  $\nu_k > r$ , ist eine Primpotenz vor der letzte Primzahl  $p_s$  schon 0, oder

die Regel  $\varrho_k$  mit der Folge der Ableitungen  $\varrho_1, \dots, \varrho_{k-1}$  nicht an der Stelle  $\mu_k$  anwendbar, so ist  $f(n) = w$ . Damit werden hintereinander mit steigenden Primzahlpotenzen alle möglichen Ableitungen mit fortlaufend weiteren Schritten aufgezählt.  $\square$

**BEMERKUNG 2.11.** Die Umkehrung ist ebenso korrekt: Wenn eine Sprache rekursiv aufzählbar ist, so kann sie durch eine Typ-0 Grammatik erzeugt werden. Dies kann mit der Beschreibung eines Programms bewiesen werden, da sich später die endlich vielen Schritte des äquivalenten Programms in endlich viele Regeln und Variablen in der Form einer Typ-0 Grammatik umgeschrieben werden kann.

Das bedeutet aber auch, dass eine Typ-0 Grammatik nicht entscheidbar oder rekursiv sein muss, da es aufzählbare (und damit Typ-0) Sprachen gibt, deren Komplement nicht aufzählbar sind. Ein Beispiel ist hier das **Halteproblem** aus Definition 1.36, dessen Sprache  $L_{\text{Halteproblem}}$  über dem Unicode leicht aufzählbar ist, wenn Programme mit Fehlern als sofort terminierende Programme zugelassen werden, und damit dann einer Typ-0 Sprache entsprechen, deren Komplement aber nicht entscheidbar ist.

Weiterhin deutet dies auch darauf hin, dass es viele formale Sprachen geben muss, die nicht dem Typ-0 angehören: Wenn alle Typ-0 Sprachen rekursiv aufzählbar sind, also mit einem Programm beschrieben werden können, so gibt es höchstens abzählbar unendliche viele Typ-0 Sprachen, da Programme geschrieben in einer formalen Sprache abzählbar sind. Nach Korollar 1.23 ist die Menge der Sprachen überabzählbar, also ist die Anzahl der Typ-0 Sprachen gering im Vergleich zu den formalen Sprachen.

**SATZ 2.12.** Die Sprache einer kontextsensitiven, also Typ-1, Grammatik ist entscheidbar

Def. „entscheidbar“  
s. S. 11-12; (Wortproblem)

**BEWEIS.** Im Gegensatz zu Typ-0 Grammatiken ist bei Typ-1 Grammatiken  $G = (V, T, R, S)$  die Länge von Produkten durch Anwendung von Regeln monoton wachsend. Gleichzeitig ist bei einem Wort  $w \in T^*$  der Länge  $n = |w|$  die Menge  $M$  der Produkte, die auf ein Wort der Länge  $n$  durch eine Regel abgebildet werden können

$$M = \{(V \cup T)^m \mid 0 < m \leq n\} \text{ durch } |M| = \sum_{m=1}^n (|V| + |T|)^m$$

beschränkt und endlich. Nach spätestens  $|M|$  Ableitungen (oder maximal  $|M| \cdot n$  Versuchen an jeder Position der möglichen Produkte) muss beginnend von  $m_1 = S$  entweder  $r(m_q) = w$  für ein  $m_q \in M$  mit  $q \leq |M|$  erreicht werden, dann gilt  $w \in L(G)$  oder es ist  $w \notin L(G)$ . Damit liegt die Entscheidung gesichert nach endlichen vielen Schritten vor, damit ist die Sprache entscheidbar.  $\square$

**BEMERKUNG 2.13.** Die Umkehrung ist hier nicht korrekt. Es kann eine Sprache konstruiert werden, die entscheidbar, jedoch nicht von Typ-1 ist.

**BEMERKUNG 2.14.** Kontextsensitive Sprachen sind als entscheidbare Sprachen eine Möglichkeit zur Definition von Programmiersprachen. Die reine Entscheidbarkeit sagt aber nichts darüber aus, was für einen Aufwand sie erfordert, und dieser ist deutlich höher als bei kontextfreien Grammatiken. Daher spielen letztere in der Anwendung eine weit größere Rolle, wenn auch sehr viele Programmiersprachen nicht vollständig kontextfrei sind, und daher genau genommen als kontextsensitiv klassifiziert werden müssten. Das ist praktisch aber kein echter Nachteil, da der Großteil der Sprache kontextfrei und damit effizient verarbeitet werden kann und die wenigen Ausnahmen im Compiler gesondert behandelt werden.

**SATZ 2.15.** Seien  $\mathcal{L}$  die Menge der formalen Sprachen,  $\mathcal{L}_k$  die Menge der Sprachen vom Chomsky-Typ  $k$ ,  $\mathcal{L}_{\text{aufzählbar}}$  die Menge der aufzählbaren formalen Sprachen und  $\mathcal{L}_{\text{entscheidbar}}$  die Menge der entscheidbaren formalen Sprachen, so gilt die **Sprachhierarchie**

$$\mathcal{L}_3 \subset \mathcal{L}_2 \subset \mathcal{L}_1 \subset \mathcal{L}_{\text{entscheidbar}} \subset \mathcal{L}_0 = \mathcal{L}_{\text{aufzählbar}} \subset \mathcal{L},$$

wobei jede Teilmengenbeziehung echt ist.

## 2.4. Grammatiken kontextfreier (Typ-2) Sprachen

**Kontextfreie Sprachen** sind die am häufigsten verwendete Sprachklasse für Programmiersprachen, da sie sowohl genug Möglichkeiten bieten komplexe Sachverhalte darzustellen, dabei aber auch noch effizient vom Rechner zu verarbeiten sind. Tatsächlich kann die Verarbeitung auch auf etwas erweiterten Sprachen noch effizient erfolgen, daher sind sehr viele Programmiersprachen nur zum größten Teil von Typ-2 mit teilweise nur kleinen eigentlich kontextsensitiven Erweiterungen, die aber algorithmisch noch gut zu verarbeiten sind und die gleichzeitig sehr die Ausdrucksfähigkeit der Sprache verbessern.

BEISPIEL 2.16. Grammatiken sind nicht eindeutig, da zum Beispiel diese beiden Grammatiken die identische Sprache, die Additionen und Multiplikationen mit einzelnen Zahlen beschreiben können:

$$G_1 = (V_1, T, R_1, S_1) \text{ mit}$$

$$V = \{\text{Term}\},$$

$$T = \{0, 1, \dots, 9, +, *\},$$

$$R = \{r_1, r_2, r_3\}, \text{ wo}$$

$$r_1 : \text{Term} \mapsto \text{Term} + \text{Term},$$

$$r_2 : \text{Term} \mapsto \text{Term} * \text{Term},$$

$$r_3 : \text{Term} \mapsto 0 \mid 1 \mid \dots \mid 9,$$

$$S = \text{Term}.$$

$$G_2 = (V_2, T, R_2, S_2) \text{ mit}$$

$$V = \{\text{Summe}, \text{Produkt}\},$$

$$T = \{0, 1, \dots, 9, +, *\},$$

$$R = \{r_1, r_2, r_3, r_4\}, \text{ wo}$$

$$r_1 : \text{Summe} \mapsto \text{Summe} + \text{Produkt},$$

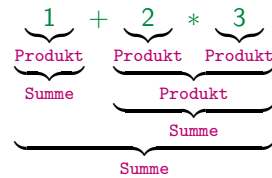
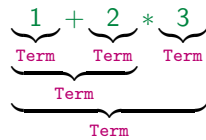
$$r_2 : \text{Summe} \mapsto \text{Produkt},$$

$$r_3 : \text{Produkt} \mapsto \text{Produkt} * \text{Produkt},$$

$$r_4 : \text{Produkt} \mapsto 0 \mid 1 \mid \dots \mid 9,$$

$$S = \text{Summe}.$$

Jedoch werden Grammatiken nicht nur zur Beschreibung von Sprachen verwendet, sondern der bei der Ableitung entstehende **Syntaxbaum** auch zur Analyse und Interpretation von Wörtern verwendet. Hier zeigen sich große Unterschiede in den beiden Grammatiken, wo der Term  $1 + 2 * 3 \in L(G_1) = L(G_2)$  sehr unterschiedlich analysiert werden kann:



An diesem Beispiel ist ersichtlich, dass die zweite Grammatik  $G_2$  die Rechenregel Punkt-vor-Strich schon in den Regeln abbildet. Wenn nun jeder Regel im Compiler oder Interpreter eine entsprechende Auswertung zugeordnet wird, so wird die zweite Grammatik immer zum korrekten Ergebnis führen, wobei die Auswertung mit Hilfe der ersten Grammatik nur bei zufällig passender Auswertungsreihenfolge ein richtiges Ergebnis liefern kann.

Darüber hinaus können auch bei einer sinnvollen Grammatik die Ableitungsbäume nicht eindeutig sein, was für die spätere Auswertung ein Problem darstellt.

DEFINITION 2.17. Eine Ableitung  $w_1 \Rightarrow_L w_2 \Rightarrow_L \dots \Rightarrow_L w_n$  wird als **Linksableitung** bezeichnet, wenn für alle  $i = 1, \dots, n - 1$  das Wort  $w_{i+1}$  durch Ersetzen der linken Variable in  $w_i$  entsteht. Entsprechend wird eine Ableitung als **Rechtsableitung** bezeichnet, wenn jeweils die rechte Variable verwendet wird.

Die Festlegung auf eine Ableitungsreihenfolge ist sinnvoll, da sonst grundsätzlich keine Eindeutigkeit eines Ableitungsbaums erreicht werden könnte, sobald mehrere Variablen in einem Wort vorhanden sind.

DEFINITION 2.18. Eine kontextfreie Grammatik  $G$  ist genau dann **mehrdeutig**, wenn es ein Wort  $w \in L(G)$  gibt, so dass  $G$  zwei unterschiedliche Linksableitungen zu  $w$  besitzt. Eine kontextfreie Sprache  $M$  ist **inhärent mehrdeutig**, wenn alle kontextfreien Grammatiken  $G$  mit  $M = L(G)$  mehrdeutig sind.

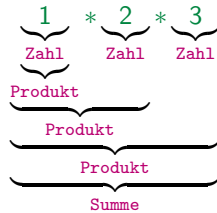
BEISPIEL 2.19. Die Grammatik  $G_2$  aus Beispiel 2.16 ist mehrdeutig, da das Wort  $1 * 2 * 3$  zwei unterschiedliche Linksableitungen besitzt:



Sobald von unten nach oben gelesen  $\text{Produkt} \xrightarrow{r_3} \text{Produkt} * \text{Produkt}$  abgeleitet wurde, ist es möglich in der Ableitung des Wortes mit Anwendung von  $r_3$  oder  $r_4$  auf die linke Variable  $\text{Produkt}$  fortzusetzen. Die Grammatik  $G_3$  ist hingegen nicht mehrdeutig, da in jedem Schritt der Linksableitung genau nur eine Regel zum Ziel führt:

$$\begin{aligned}
 G_3 &= (V_3, T, R_3, S_3) \text{ mit} \\
 V_3 &= \{\text{Summe}, \text{Produkt}, \text{Zahl}\}, \\
 T &= \{0, 1, \dots, 9, +, *\}, \\
 R_3 &= \{r_1, r_2, r_3, r_4, r_5\}, \text{ wo} \\
 r_1 : \quad &\text{Summe} \mapsto \text{Summe} + \text{Produkt}, \\
 r_2 : \quad &\text{Summe} \mapsto \text{Produkt}, \\
 r_3 : \quad &\text{Produkt} \mapsto \text{Produkt} * \text{Zahl}, \\
 r_4 : \quad &\text{Produkt} \mapsto \text{Zahl}, \\
 r_5 : \quad &\text{Zahl} \mapsto 0 \mid 1 \mid \dots \mid 9, \\
 S_3 &= \text{Summe}.
 \end{aligned}$$

Exemplarisch hat hier das Wort  $1 * 2 * 3$  eine eindeutige Linksableitung:



BEISPIEL 2.20. Leider ist die allgemeine Frage, ob eine Grammatik mehrdeutig ist oder nicht, bewiesen nicht entscheidbar, da es auf das zum Halteproblem einer Turingmaschine äquivalente Post'sche Korrespondenzproblem zurückgeführt werden kann. Trotzdem sind in vielen Fällen eindeutige Aussagen möglich: Eine **deterministische kontextfreie Grammatik** definiert sich daraus, dass sie durch einen effizienten und später noch behandelten deterministischen Kellerautomaten erkannt werden kann. Diese Grammatiken sind eine echte Teilmenge der nicht-mehrdeutigen kontextfreien Grammatiken, und je nach speziellem Automaten gibt es verschiedene Kriterien, wie eine erkennbare deterministische kontextfreie Grammatik nachgewiesen werden kann. Beispielsweise ist eine Grammatik eine deterministische  $LL(k)$ -Grammatik, wenn die Wahl der anzuwendenden Regel für die linke Variable durch Vorausschau auf  $k$  nächste terminale Zeichen entschieden werden kann. Dies schließt aber immer eine Linksrekursion wie in Regeln  $r_1$  und  $r_3$  von  $G_3$  aus Beispiel 2.19 aus, was hier aber leicht durch Vertauschung der resultierenden Variablen vermieden werden kann.

Aus den vorangegangenen Beispielen wird klar, dass eine nicht-mehrdeutige kontextfreie Grammatik einer kontextfreien Sprache nicht nur eine eindeutige Interpretation der Worte ermöglicht, sondern beispielsweise unter Verwendung von Vorausschau zu besonders effizienten Algorithmen zur Erkennung führen kann. Da aber nicht jede kontextfreie Sprache deterministisch oder wenigstens nicht-mehrdeutig



ist, verbleibt die Frage, wie das Wortproblem im allgemeinen Fall einer kontextfreien Sprache möglichst effizient zu beantworten ist, um die Eignung von kontextfreien Sprachen beispielsweise für Programmiersprachen beantworten zu können. Besonders problematisch sind dabei Grammatiken, die besonders viele oder sogar unendliche viele verschiedene Ableitungen für ein Wort in der Sprache zulassen. Ein einfacher Ringschluss in Form einer in einer kontextfreien Grammatik zulässigen Regel  $A \mapsto A$  wird eine naive erschöpfende Suche niemals enden lassen. Ist die Grammatik hingegen in einer Form, bei der jede Regelanwendung streng das Wort verlängert oder ein Symbol terminiert, so kann das Problem umgangen werden:

**DEFINITION 2.21.** Eine kontextfreie Grammatik  $G = (V, T, R, S)$  liegt in **Chomsky Normalform** vor, wenn alle Regeln in  $R$  in einer dieser drei Formen  $V \rightarrow V^2$ ,  $V \rightarrow T$  oder bei  $S = S$  maximal einmalig  $S \mapsto \varepsilon$  und dann mit  $V \rightarrow (V \setminus \{S\})^2$  statt  $V \rightarrow V^2$  vorliegen. Also werden Variablen entweder nur auf zwei Variablen oder ein terminales Symbol abgebildet, und nur das Startsymbol darf auf  $\varepsilon$  abbilden, wenn es dann auf keiner rechten Seite auftritt.

Die folgenden Lemmata zeigen, dass jede kontextfreie Grammatik auf eine äquivalente Grammatik in Chomsky Normalform umgeformt werden kann.

**LEMMA 2.22. (Terminalvariablen)** Zu jeder kontextfreien Grammatik  $G$  existiert eine kontextfreie Grammatik  $G_1$  mit  $L(G) = L(G_1)$ , bei der Terminale in den Produktionen nur einzeln auftreten, also die Regeln entweder  $V \rightarrow V^+$  oder  $V \rightarrow T$  lauten mit Ausnahme der optionalen  $S \mapsto \varepsilon$  Regel.

**BEWEIS.** In  $G_1$  werden für jedes Terminal  $x \in T$  eine Variable  $V_x$  und Regel  $V_x \mapsto x$  eingeführt, und jedes sonstige Auftreten des Terminals  $x$  auf rechten Seiten in Regeln von  $G$  durch entsprechende Regeln durch  $V_x$  ersetzt. Die Sprachen der Grammatiken  $G$  und  $G_1$  bleiben dabei identisch.  $\square$

**LEMMA 2.23. (Identitätselimination)** Zu jeder kontextfreien Grammatik  $G_1$  existiert eine kontextfreie Grammatik  $G_2$  mit  $L(G_1) = L(G_2)$ , die keine Regeln der Form  $V \rightarrow V$  enthält.

**BEWEIS.** Jede Regel der Form  $A \mapsto B$  in  $G_1$  wird nicht nach  $G_2$  übernommen, und zusätzlich alle Regeln mit  $B \mapsto \dots$  in  $G_1$  als  $A \mapsto \dots$  in  $G_2$  und  $B \mapsto \dots$  mit allem Auftreten von  $A$  durch  $B$  ersetzt neben allen weiteren nicht betroffenen Regeln übernommen. Wenn es weitere direkten Zuordnungen gibt, werden die weiteren Regeln auch transitiv hinzugefügt. Die Sprachen der Grammatiken  $G_1$  und  $G_2$  bleiben dabei identisch.  $\square$

**LEMMA 2.24. (Zweierregeln)** Zu jeder kontextfreien Grammatik  $G_2$  existiert eine kontextfreie Grammatik  $G_3$  mit  $L(G_2) = L(G_3)$ , bei der keine Regeln der Form  $V \rightarrow V^k$  mit  $k > 2$  auftreten.

**BEWEIS.** In  $G_3$  werden alle Regeln außer  $V \rightarrow V^k$  mit  $k > 2$  aus  $G_2$  übernommen. Für Regeln der Form  $V \rightarrow V^k$  veranschaulicht durch  $A \mapsto B_1 B_2 \dots B_k$  werden  $k-2$  neue Variablen  $A_i$  mit  $i = 1, \dots, k-2$  und  $k-1$  Regeln der Form  $A \mapsto B_1 A_1$ ,  $A_1 \mapsto B_2 A_2$ ,  $\dots$ ,  $A_{k-2} \mapsto B_{k-1} B_k$  hinzugefügt. Die Sprachen der Grammatiken  $G_2$  und  $G_3$  bleiben dabei identisch.  $\square$

**SATZ 2.25.** Zu jeder kontextfreien Grammatik  $G$  existiert eine äquivalente kontextfreie Grammatik  $G_3$  in Chomsky Normalform mit  $L(G) = L(G_3)$ .

**BEWEIS.** Anwendung der Lemmata 2.22, 2.23 und 2.24 in dieser Reihenfolge.  $\square$

**BEMERKUNG 2.26.** In anderer Literatur werden Typ-2 Grammatiken auch allgemeiner mit Regeln der Form  $V \rightarrow (T \cup V)^*$  statt hier  $V \rightarrow (T \cup V)^+$  mit optionalem  $S \mapsto \varepsilon$  ohne  $S$  auf der rechten Seite formuliert. Hier können sowohl  $\varepsilon$  auf der rechten Seite als auch die Startvariable auf der rechten Seite

auftreten. Dann wird eine äquivalente Grammatik ohne  $\varepsilon$  in der Sprache dadurch erzeugt, dass zunächst alle Variablen bestimmt werden, die transitiv auf  $\varepsilon$  abgebildet werden können. Die neue Grammatik übernimmt alle Regeln ohne  $\varepsilon$  auf der rechten Seite, und ergänzt alle Regeln mit den reduzierbaren Variablen auf der rechten Seite durch weitere Regeln ohne die reduzierbaren Variablen. Sollte die ursprüngliche Sprache das  $\varepsilon$  enthalten, so wird ein neues Startsymbol  $S'$  mit den Regeln  $S' \mapsto S \mid \varepsilon$  mit dem alten Startsymbol  $S$  eingefügt.

BEISPIEL 2.27. Gegeben sei die kontextfreie Grammatik

$$\begin{aligned} G &= (V, T, R, S) \text{ mit} \\ V &= \{E, F\}, \\ T &= \{(\,, \,), +, a\}, \\ R &= \{r_1, r_2\}, \text{ wo} \\ r_1 : E &\mapsto E+F \mid F, \\ r_2 : F &\mapsto (E) \mid a, \\ S &= E. \end{aligned}$$

Mit Anwendung von Lemma 2.22 ergibt sich mit Terminalvariablen die Grammatik

$$\begin{aligned} G_1 &= (V_1, T, R_1, S) \text{ mit} \\ V_1 &= \{E, F, V_(_), V_+), V_+, V_a\}, \\ T &= \{(\,, \,), +, a\}, \\ R_1 &= \{r_1, r_2, r_3, r_4, r_5, r_6\}, \text{ wo} \\ r_1 : E &\mapsto EV_+F \mid F, \\ r_2 : F &\mapsto V_+(EV) \mid V_a, \\ r_3 : V_(&\mapsto (, \\ r_4 : V_&\mapsto ), \\ r_5 : V_+ &\mapsto +, \\ r_6 : V_a &\mapsto a, \\ S &= E. \end{aligned}$$

Die Anwendung von Lemma 2.23 ergibt sich durch Identitätselemination wegen  $E \mapsto F$ ,  $F \mapsto V_a$  und transitiv  $E \mapsto V_a$  dann die Grammatik

$$\begin{aligned} G_2 &= (V_1, T, R_2, S) \text{ mit} \\ V_1 &= \{E, F, V_(_), V_+), V_+, V_a\}, \\ T &= \{(\,, \,), +, a\}, \\ R_2 &= \{r_1, r_2, r_3, r_4, r_5, r_6\}, \text{ wo} \\ r_1 : E &\mapsto EV_+F \mid EV_+V_a \mid FV_+F \mid FV_+V_a \mid V_aV_+F \mid V_aV_+V_a \mid V_+(EV) \mid V_+(FV) \mid V_+(V_aV) \mid a, \\ r_2 : F &\mapsto V_+(EV) \mid V_+(FV) \mid V_+(V_aV) \mid a, \\ r_3 : V_(&\mapsto (, \\ r_4 : V_&\mapsto ), \\ r_5 : V_+ &\mapsto +, \\ r_6 : V_a &\mapsto a, \\ S &= E. \end{aligned}$$

Schließlich überführt die Anwendung von Lemma 2.24 durch Einführung von Zweierregeln die Grammatik in die Chomsky Normalform:

$$\begin{aligned}
 G_3 &= (V_3, T, R_3, S) \text{ mit} \\
 V_3 &= \{E, E_1, E_2, E_3, E_4, F, F_1, V_(), V_+, V_a\}, \\
 T &= \{ (, ), +, a \}, \\
 R_3 &= \{r_1, r_2, r_3, r_4, r_5, r_6, r_7, r_8, r_9, r_{10}, r_{11}\}, \text{ wo} \\
 r_1 : E &\mapsto EE_1 \mid FE_2 \mid V_aE_3 \mid V_()E_4 \mid a, \\
 r_2 : E_1 &\mapsto V_+F \mid V_+V_a, \\
 r_3 : E_2 &\mapsto V_+F \mid V_+V_a, \\
 r_4 : E_3 &\mapsto V_+F \mid V_+V_a, \\
 r_5 : E_4 &\mapsto EV_() \mid FV_() \mid V_aV_(), \\
 r_6 : F &\mapsto V_()F_1 \mid a, \\
 r_7 : F_1 &\mapsto EV_() \mid FV_() \mid V_aV_(), \\
 r_8 : V_() &\mapsto (, \\
 r_9 : V_() &\mapsto ), \\
 r_{10} : V_+ &\mapsto +, \\
 r_{11} : V_a &\mapsto a, \\
 S &= E.
 \end{aligned}$$

Durch die direkte Übersetzung kann eine deutlich umfangreichere Grammatik entstehen, die hier sogar bei identischem Prefix sogar schon mit etwas weniger zusätzlichen Variablen dargestellt wurde, die hier offensichtlich durch Zusammenführung der Regeln  $r_2$ ,  $r_3$  und  $r_4$  sowie  $r_5$  und  $r_7$  und zugehörigen Variablen weiter reduziert werden kann.

Mit Hilfe des Chomsky Normalform kann das Wortproblem für allgemeine kontextfreie Sprachen auf Basis einer solchen Grammatik mit dem Algorithmus von Cocke, Younger und Kasami, kurz CYK, relativ effizient beantwortet werden.

DEFINITION 2.28. Für ein eine Grammatik  $G = (V, T, R, S)$  und einem Wort  $w \in T^*$  von Länge  $|w| = n$  mit  $w = x_1 \dots x_n$ ,  $x_i \in T$  sei das Teilwort  $w_{(i,j]}$  ab Stelle  $i$  bis Stelle  $j$  definiert als

$$w_{(i,j]} = x_{i+1}x_{i+2} \dots x_j$$

und die in der Grammatik das Teilworts  $w_{(i,j]}$  transitiv erzeugenden Variablen  $D_{(i,j]}$  durch

$$D_{(i,j]} = \{ X \in V \mid X \xRightarrow{*} w_{(i,j]} \}.$$

SATZ 2.29. (**CYK-Algorithmus**) Sei eine Grammatik  $G = (V, T, R, S)$  in Chomsky Normalform und ein Wort  $w \in T^*$  der Länge  $|w| = n$  gegeben. Für  $w \in T^*$  Es gilt  $w \in L(G)$  mit  $|w| = n$  genau dann, wenn ist  $S \in D_{(0,n]}$  und rekursiv können die Werte von  $D_{(i,i+k]}$  deterministisch aus zuvor bestimmten  $D_{(j,j+l]}$  für  $1 < l < k$  und  $D_{(i,i+1]}$  aus den Terminalregeln bestimmt werden.

BEWEIS. Es gilt  $D_{(i,i+1]} = \{ X \in V \mid (X \mapsto w_{(i,i+1]}) \in R \}$ , da dies die einzigen Regeln sind, die Terminale mit der Grammatik erzeugt werden können. Weiterhin ist für  $k > 1$  dann

$$D_{(i,i+k]} = \{ X \in V \mid \exists j \leq k, Y, Z : Y \in D_{(i,j]}, Z \in D_{(j,i+k]}, (X \mapsto YZ) \in R \}.$$

Ist dann  $S \in D_{(0,n]}$ , so existiert eine transitive Ableitung  $S \xRightarrow{*} w$  und damit ist  $w \in L(G) = \{w \in T^* \mid S \xRightarrow{*} w\}$  nach Definition.  $\square$

BEISPIEL 2.30. Gegeben sei die folgende Grammatik in Chomsky Normalform

$$G = (V, T, R, S) \text{ mit}$$

$$V = \{A, B, C, S\},$$

$$T = \{a, b\},$$

$$R = \{r_1, r_2, r_3, r_4\}, \text{ wo}$$

$$r_1 : S \mapsto AB \mid BC,$$

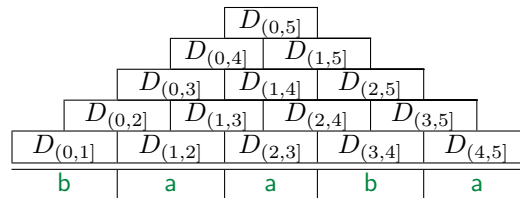
$$r_2 : A \mapsto BA \mid a,$$

$$r_3 : B \mapsto CC \mid b,$$

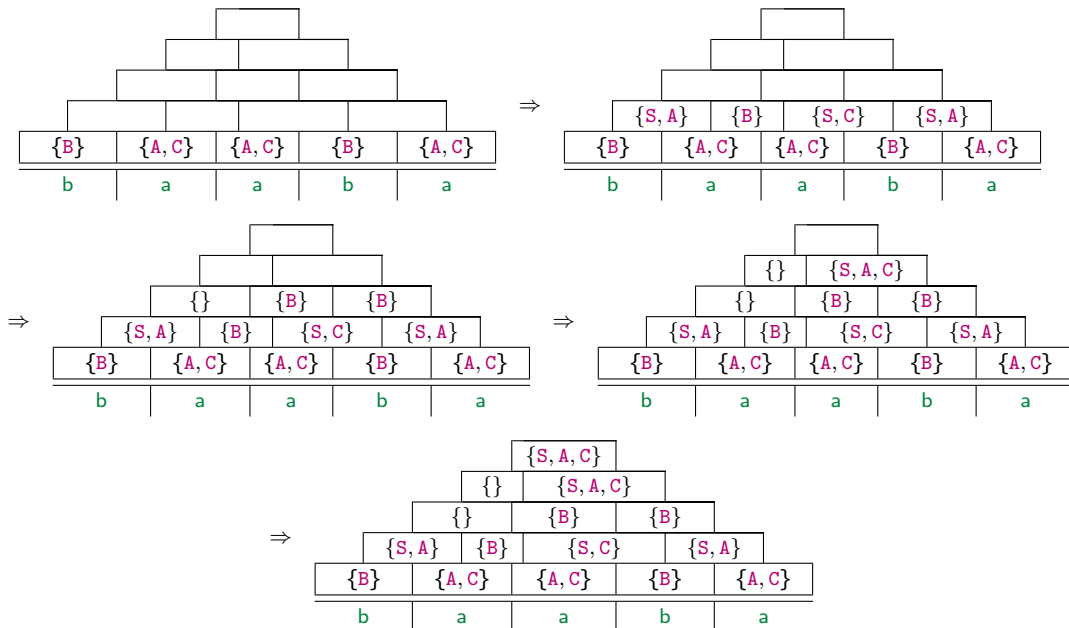
$$r_4 : C \mapsto AB \mid a,$$

$$S = S.$$

Die Frage ist nun, ob für das Wort  $w = baaba \in T^*$  dann  $w \in L(G)$  gilt, es also Wort der von  $G$  erzeugten Sprache ist. Dazu wird diese pyramidische Datenstruktur aufgebaut:



Zeilenweise ergibt sich dann von unten nach oben das folgende Ergebnis:



Da nun  $S \in D_{(0,5]}$  ist, liegt  $w \in L(G)$ , also in der Sprache. Da die Anzahl der Felder durch  $n^2$  abgeschätzt werden kann, und in jedem Feld abgeschätzt  $n$  Trennmöglichkeiten getestet werden, hat der Algorithmus die asymptotische Komplexität von  $\mathcal{O}(n^3)$ . Damit kann also mit das Wortproblem für kontextfreie Sprachen deterministisch und in polynomialer Zeit in Abhängigkeit von der Wortlänge beantwortet werden.

BEISPIEL 2.31. Es gibt verschiedene formale Beschreibungssprachen, in denen kontextfreie Sprachen formal beschrieben werden können, damit dafür ein Compiler oder Interpreter umgesetzt werden kann. Wichtig sind hier beispielsweise die **Backus-Naur-Form (BNF)** und die **Erweiterte Backus-Naur-Form (EBNF)**, die die BNF um praktische Elemente erweitert. In diesen können die Regeln kontextfreier

Grammatiken direkt umgesetzt werden. Die Grammatik  $G_2$  aus Beispiel 2.16 lautet in BNF (abgekürzt wegen Seitenbreite):

```
<summe> ::= <summe> "+" <produkt> | <produkt>
<produkt> ::= <produkt> "*" <produkt> | "0" | "1" | ... | "8" | "9"
```

In der Praxis waren die Darstellungen in der BNF recht mühselig, beispielsweise, wenn Zeichenbereiche wie die Zahlen 0-9 oder Buchstaben A-Z verwendet wurden und die Klammerung aller Variablen erscheint nicht wirklich erforderlich. Hier ermöglicht die EBNF eine deutlich kürzere Darstellung. Jede Grammatik in EBNF kann dabei auch in der BNF ausgedrückt werden, jedoch in der Regel mit deutlich mehr Zeichen. Leider gibt es viele unterschiedliche Darstellungen der EBNF, die sich je nach verwendeten Tools unterscheiden. Für das in den Übungen verwendete Online-Tool [Lark](#) ergibt sich für die Grammatik  $G_2$  aus Beispiel 2.16 hat beispielsweise diese Beschreibung in EBNF:

```
start:      summe
summe:      summe "+" produkt | produkt
produkt:    produkt "*" produkt | "0"|"1"|"2"|"3"|"4"|"5"|"6"|"7"|"8"|"9"
```

Fortlaufende Zeichenoptionen können hier unter anderem mit zwei Punkten beschrieben werden. Außerdem können Terme geklammert und mit einem Kleenschen Abschluss versehen werden. Die folgende Sprache ist damit vereinfacht und lässt auch Klammern und mehrere Ziffern in einer Zahl zu:

```
start:      summe
summe:      summe "+" produkt | produkt
produkt:    produkt "*" produkt | zahl
zahl:       "(" summe ")" | ( "0" .. "9" )+
```

Ein grundsätzliches Problem bei der Beschreibung von kontextfreien Grammatiken ist, dass bei Anwendbarkeit mehrerer Regeln keine Präferenz festgelegt wird. Damit kann es passieren, dass die Verwendung der Beschreibung in der Interpretation von Worten nicht eindeutig ist, und im Nachgang aufgelöst werden muss.

Ein Ansatz ist hier die Verwendung der zu (E)BNF sehr ähnlichen **Parsing Expression Grammar (PEG)**, die bei mehreren Optionen zur Auflösung immer die früheren Varianten wählt, die am Ende zu einem vollständigen Syntaxbaum führt. Hier sind die Beschreibungen nicht unmittelbar vom Ansehen unterscheidbar, doch die Bedeutung ist bei PEG eingeschränkt und Anwendung führt auf andere Algorithmen. Es ist anzunehmen, dass diese zusätzliche Einschränkung auch die darstellbaren Sprachen einschränkt, doch wurde bisher keine Sprache formuliert, die nicht mit einer PEG darstellbar ist.

**BEISPIEL 2.32.** Nur selten werden erfolgreich neue allgemeine Programmiersprachen entwickelt. Eine häufige Nutzung sind hingegen **domänenspezifische Sprachen (DSL)**, die schon lange ein deutlich vereinfachtes Nutzungsinterface für einen Anwendungsbereich zur Verfügung stellen und heute in **Low-Code-Entwicklungsplattformen** in vielen Wirtschaftsbereichen neu entdeckt werden.

Auf Basis von DSL werden Personen mit Domänen-Expertise oder spezifischen Anforderungen befähigt, Funktionen von Software zu verstehen, zu korrigieren und weiterzuentwickeln, ohne dass dazu das Erlernen einer allgemeinen Programmiersprache erforderlich ist. Dies umfasst in der Regel sowohl eine große Einschränkung auf wenige Funktionen und Sprachelemente, die aber umgekehrt verständlich sonst sehr komplexe Softwarekomponenten ansteuern können.

Es haben sich sehr viele spezielle formale Sprachen entwickelt, die in unterschiedlichen Domänen deutlich effizienter als allgemeine Programmiersprachen genutzt werden können: Prominente Beispiele für

**externe DSL** sind SQL, XML & HTML, Regular Expressions und VHDL. Externe DSL sind eigenständige formale Sprachen, die unabhängig von einer allgemeinen Programmiersprache stehen. Dagegen ist JSON (ursprünglich) eine **interne DSL**, da diese Beschreibungssprache gleichzeitig auch korrekten Javascript-Code darstellt und darin ihren Ursprung hat. Durch die langjährige Weiterentwicklung vieler DSL benötigen sie in vielen Aspekten zur Nutzung inzwischen einigen Lernaufwand.

Häufig werden interne DSL bei Interpretersprachen wie Python oder Lua mit vordefinierten Funktionen und Bibliotheken umgesetzt, bei compilierten Sprachen wie C oder C++ beispielsweise mit ihrem mächtigen Präprozessor zur Musterersetzung. Hier haben interne DSL unter anderem die Vorteile, dass keine gesonderte Entwicklung der Sprache erfolgen muss, und bei Bedarf die zu Grunde liegende Programmiersprache zusätzlich eingesetzt werden kann. Nachteile sind hier unter anderem, dass die Sprachentwicklung durch die Grenzen der Basissprache limitiert ist, die zulässige Nutzung aus Sicherheitsaspekten kaum eingeschränkt werden kann (beispielsweise zur Vermeidung von Remote Code Execution und speziell Injektionsangriffen) und die volle Entwicklungsumgebung zur Verfügung stehen muss. Externe DSL können hingegen der Domäne im Rahmen der möglichen Grammatiken frei angepasst, validiert, eventuell getestet und im Funktionsumfang beschränkt werden. Zusätzlich ist keine weitere Entwicklungsumgebung erforderlich, doch bieten sich eine Vielzahl von Entwicklungsumgebungen für DSL an, die zusätzlich zu den Parser-Generatoren auch Unterstützung für unterstützende Editoren, Syntax-Prüfungen, Tests, Debugger, Simulatoren und vielem mehr anbieten.

Während die Beschreibung über Grammatiken einen Fokus auf die textbasierte Darstellung vermittelt, können die Grammatiken ebenso Grundlage für graphische Entwicklungsumgebungen sein: Die Variablen definieren dann Komponenten, die nach den Regeln der Grammatik zusammengeführt werden können.

BEISPIEL 2.33. Abgesehen von der Nutzung von kontextfreien Sprachen für Programmiersprachen oder DSL werden diese auch zur Datenkompression verwendet. Der **Lempel-Ziv-Welch (LZW)-Algorithmus** wird unter anderem im **Graphics Interchange Format (GIF)** und **Portable Document Format (PDF)** verwendet, und erzeugt adaptiv eine kontextsensitive Grammatik, um wiederkehrende Worte effizient speichern zu können. Es werden fortlaufend neue Variablen mit zugehörigen Regeln erzeugt, und jeweils Repräsentationen der Variablen gespeichert. Beim Laden der komprimierten Daten wird die Grammatik synchron aufgebaut und durch Anwendung der Regeln auf die Variablen wieder der ursprüngliche Inhalt reproduziert. Die Kompression läuft schrittweise so ab:

- (1) Initialisierung der Grammatik mit Variablen und Regeln für jedes einzelne Zeichen.
- (2) Finde längste Wortentsprechung einer Variable mit nächsten Zeichen.
- (3) Speichere die Variable, und erzeuge neue Variable und Regel mit Wort und nächstem Zeichen.
- (4) Soweit noch Text vorhanden gehe zu Schritt 2.

Soll das Wort **endende** komprimiert werden, so wird zunächst die erste Grammatik angelegt:

$$G_o = (V_0, T_0, R_0, \#) \text{ mit } V = \{\#, \text{A}, \text{B}, \dots, \text{Z}\}, T = \{\text{a}, \text{b}, \dots, \text{z}\},$$

$$R = \{r_0, r_1, \dots, r_{26}\}, \text{ wo}$$

$$r_0 : \# \mapsto \text{A}\# \mid \dots \mid \text{Z}\# \mid \varepsilon,$$

$$r_1 : \text{A} \mapsto \text{a}, \dots, r_{26} : \text{Z} \mapsto \text{z}.$$

Das längste Teilwort, das durch eine Variable abgebildet werden kann ist **e** durch **E**. Es wird nun die neue Variable  $\alpha_1$  mit der Regel  $\alpha_1 \mapsto \text{en}$  eingefügt, da auf den ersten Buchstaben das **n** folgt. Die

nächste Grammatik  $G_1$  hat dann diese Form:

$$G_1 = (V_1, T_1, R_1, \#) \text{ mit } V = \{\#, A, B, \dots, Z, \alpha_1\}, T = \{a, b, \dots, z, \text{en}\},$$

$$R = \{r_0, r_1, \dots, r_{26}, r_{27}\}, \text{ wo}$$

$$\begin{aligned} r_0 : \quad \# &\mapsto A\# \mid \dots \mid Z\# \mid \varepsilon \mid \alpha_1, \\ r_1 : \quad A &\mapsto a, \dots, r_{26} : Z \mapsto z, \\ r_{27} : \alpha_1 &\mapsto \text{en}. \end{aligned}$$

Das nächste Zeichen **n** kann nur durch **N** gespeichert werden, damit wird eine neue Variable  $\alpha_2$  mit Regel  $\alpha_2 \mapsto \text{nd}$  zur Grammatik  $G_2$  hinzugefügt. Das nächste Zeichen ist **d**, das mit **D** gespeichert wird, die Grammatik  $G_3$  hat dann zusätzlich die Variable  $\alpha_3$  und  $\alpha_3 \mapsto \text{de}$ .

$$G_3 = (V_3, T_3, R_3, \#) \text{ mit } V = \{\#, A, B, \dots, Z, \alpha_1, \alpha_2, \alpha_3\}, T = \{a, b, \dots, z, \text{en}, \text{nd}, \text{de}\},$$

$$R = \{r_0, r_1, \dots, r_{26}, r_{27}, r_{28}, r_{29}\}, \text{ wo}$$

$$\begin{aligned} r_0 : \quad \# &\mapsto A\# \mid \dots \mid Z\# \mid \varepsilon \mid \alpha_1 \mid \alpha_2 \mid \alpha_3, \\ r_1 : \quad A &\mapsto a, \dots, r_{26} : Z \mapsto z, \\ r_{27} : \alpha_1 &\mapsto \text{en} \\ r_{28} : \alpha_2 &\mapsto \text{nd} \\ r_{29} : \alpha_3 &\mapsto \text{de} \end{aligned}$$

Als nächstes kann schon eine Zeichenfolge **en** mit Variable  $\alpha_1$  und es wird eine neue Variable  $\alpha_4$  mit Regel  $\alpha_4 \mapsto \text{end}$  angelegt. Insgesamt ergibt sich diese Kodierung:

$$\begin{array}{cccccc} e & n & d & e & n & d \\ E & N & D & \alpha_1 & \alpha_3 & \alpha_4 \end{array}$$

Statt 7 Zeichen werden hier nur 5 Zeichen gesendet, und abhängig von der Anzahl der Variablen wird die Anzahl der verwendeten Bits pro Zeichen bestimmt. Bei bis zu 32 Variablen werden jeweils 5 Bits pro Zeichen gespeichert. Statt sonst  $7 \cdot 5 = 35$  Bits würden jetzt nur  $5 \cdot 5 = 25$  Bits verwendet. Bei der Dekompression wird entsprechend parallel die kontextfreie Grammatik aufgebaut, damit die Variablen auf die terminalen Symbole abgebildet werden können.

## 2.5. Grammatiken regulärer (Typ-3) Sprachen

Zur Diskussion regulärer Sprachen hilft die Definition regulärer Ausdrücke:

**DEFINITION 2.34.** Ein **regulärer Ausdruck** ist eine effiziente Darstellung von Sprachen. Die Menge  $\mathcal{R}(\Sigma)$  umfasst alle regulären Ausdrücke über einem Alphabet  $\Sigma$ . Es sind  $\emptyset, \varepsilon \in \mathcal{R}(\Sigma)$  und  $\Sigma \subset \mathcal{R}(\Sigma)$ . Sind  $\alpha, \beta \in \mathcal{R}(\Sigma)$ , so sind auch  $\alpha\beta, (\alpha|\beta), (\alpha)^* \in \mathcal{R}(\Sigma)$ . Die Sprache  $L(\alpha)$  eines regulären Ausdrucks  $\alpha \in \mathcal{R}(\Sigma)$  sei definiert durch  $L(\emptyset) = \emptyset$ ,  $L(\varepsilon) = \{\varepsilon\}$  und  $L(a) = \{a\}$  für ein  $a \in \Sigma$ , sowie

- $L(\alpha\beta) = L(\alpha)L(\beta)$ ,
- $L(\alpha|\beta) = L(\alpha) \cup L(\beta)$ ,
- $L(\alpha^*) = (L(\alpha))^*$

für reguläre Ausdrücke  $\alpha, \beta \in \mathcal{R}(\Sigma)$ . Eine weitere Notation für  $L(\alpha)$  ist  $\langle \alpha \rangle$ , und wenn  $w \in \langle \alpha \rangle = L(\alpha)$ , so **matcht** oder **passt** das Wort  $w$  zum Ausdruck  $\alpha$ .

**BEMERKUNG 2.35.** Zur einfacheren Handhabung werden folgende Regeln zur syntaktischen **Klammerung regulärer Ausdrücke** eingeführt:

- Stern vor Punkt:  $\alpha\beta^* = \alpha(\beta^*)$
- Punkt vor Strich:  $\alpha|\beta\gamma = \alpha|(\beta\gamma)$
- Links vor Rechts:  $\alpha|\beta|\gamma = ((\alpha|\beta)|\gamma)$ ,  $\alpha\beta\gamma = (\alpha\beta)\gamma$

BEISPIEL 2.36. Der reguläre Ausdruck  $((01)0)$  ist syntaktisch gleichbedeutend mit  $010$ . Der reguläre Ausdruck  $((01)(00))$  ist aber syntaktisch **nicht** gleichbedeutend mit  $0100$ , da dies  $((01)0)0$  bedeutet. Die korrekte Vereinfachung lautet:

$$((01)(00)) = 01(00)$$

Dies entspricht dem Unterschied unterschiedlicher Grammatiken  $G_1, G_2$ , die aber die gleiche Sprache erzeugen können. Die Verwendung unterschiedlicher Syntax kann extreme Unterschiede in der Verwendung beispielsweise in Rechenzeit oder benötigtem Speicher ausmachen.

SATZ 2.37. *Das Problem, ob zwei syntaktisch unterschiedliche reguläre Ausdrücke  $\alpha, \beta \in \mathcal{R}(\Sigma)$  die gleiche Sprache  $L(\alpha) = L(\beta)$  erzeugen, ist PSPACE-vollständig.*

BEWEIS. [2]. Derzeit bekannte Algorithmen haben teilweise Rechenzeiten von  $\mathcal{O}(2^{(n^2)})$ , also sogar jenseits exponentiellem Verhalten, wobei  $n$  die Länge der regulären Ausdrücke ist.  $\square$

BEMERKUNG 2.38. Die in den theoretischen Betrachtungen verwendeten oben eingeführt regulären Ausdrücke sind zur Beschreibung von den betreffenden Sprachen ausreichend. Weitere in der Praxis praktische Operatoren sind nicht erforderlich, da sie durch die bisherigen Operatoren abgedeckt sind, und sonst alle Beweise nur komplizierter machen, wenn mehr Operationen berücksichtigt werden müssen. Beispielsweise ist:

$$\begin{aligned} "w \in L(\alpha^+)" &\Leftrightarrow w \in L(\alpha\alpha^*) \\ "w \in L(\wedge\alpha)" &\Leftrightarrow w \notin L(\alpha) \end{aligned}$$

BEISPIEL 2.39. Auf dem Alphabet  $\Sigma = \{a, b\}$  ist  $\alpha = a(ab)^*(a|bb) \in \mathcal{R}(\Sigma)$  ein regulärer Ausdruck, der diese Sprache beschreibt:

$$L(\alpha) = \{a a, a bb, a ab a, a ab bb, a abab a, a abab bb, a ababab a, a ababab bb, \dots\}$$

In der Darstellung sind die drei Abschnitte  $a$ ,  $(ab)^*$  und  $(a|bb)$  des Ausdrucks zur Veranschaulichung mit etwas Abstand dargestellt.

SATZ 2.40. *Reguläre Ausdrücke beschreiben reguläre Sprachen auf einem Alphabet.*

BEWEIS. Zu zeigen ist: Für jeden regulären Ausdruck  $\alpha$  existiert eine Chomsky-Grammatik  $G$  vom Typ 3, für die  $L(G) = L(\alpha)$  gilt.

Sei  $\alpha$  ein regulärer Ausdruck, ist  $\alpha = \emptyset$ ,  $\alpha = \varepsilon$  oder  $\alpha \in \Sigma$ , so direkt die einfache reguläre Grammatiken ohne Terminale, oder mit dem einzigen Terminal und gleichzeitigem Startsymbol  $\varepsilon$  oder  $a$  ohne Regel für das jeweilige  $a \in \Sigma$ , die das Geforderte erfüllen. Aus diesem Induktionsanfang aus Sprachen ohne oder mit einem Wort mit einem Zeichen, die damit eine Darstellung als Grammatik besitzen, wird die Aussage für alle weiteren regulären Ausdrücke bewiesen.

Seien  $\alpha$  und  $\beta$  reguläre Ausdrücke über einem Alphabet  $\Sigma$ , die jeweils rechtslineare Grammatiken  $G_\alpha = (V_\alpha, T_\alpha, R_\alpha, S_\alpha)$  und  $G_\beta = (V_\beta, T_\beta, R_\beta, S_\beta)$  besitzen.

Die Grammatik  $G_{\alpha\beta} = (V_{\alpha\beta}, T_{\alpha\beta}, R_{\alpha\beta}, S_{\alpha\beta})$  für den Ausdruck  $\alpha\beta$  hat dann die zusammengeführten Variablen  $V_{\alpha\beta} = V_\alpha \times \{\alpha\} \cup V_\alpha \times \{\beta\} \cup \{S_\beta\}$ , um Namenskollisionen zu vermeiden, die vereinigten Terminale  $T_{\alpha\beta} = T_\alpha \cup T_\beta$ , die Regeln  $R_{\alpha\beta} = \tilde{R}_\alpha \times \{\alpha\} \cup R_\beta \times \{\beta\}$ , wobei die Regeln jeweils um die veränderten Variablen korrigiert und die Regeln ohne Variablen bei  $R_\alpha$  auf der rechten Seite werden um  $S_\beta$  ersetzt. Die resultierende Grammatik ist dann auch rechtslinear. Der neue Start beginnt bei  $S_{\alpha\beta} = S_\alpha$ .



Die Grammatik  $G_{\alpha|\beta} = (V_{\alpha|\beta}, T_{\alpha|\beta}, R_{\alpha|\beta}, S_{\alpha|\beta})$  für den Ausdruck  $\alpha|\beta$  hat dann die zusammengeführten Variablen  $V_{\alpha|\beta} = V_\alpha \times \{\alpha\} \cup V_\beta \times \{\beta\} \cup \{\mathbf{S}_{\alpha|\beta}\}$ , um Namenskollisionen zu vermeiden, die vereinigten Terminale  $T_{\alpha|\beta} = T_\alpha \cup T_\beta$ , die Regeln  $R_{\alpha|\beta} = R_\alpha \times \{\alpha\} \cup R_\beta \times \{\beta\} \cup \{\mathbf{S}_{\alpha|\beta} \mapsto S_\alpha \mid S_\beta\}$  und  $S_{\alpha|\beta} = \mathbf{S}_{\alpha|\beta}$ .

Die Grammatik  $G_{\alpha^*}$  von  $(\alpha)^*$  ist  $G_{\alpha^*} = (V_\alpha \cup \{\mathbf{S}_\alpha\}, T_\alpha, \tilde{R}_\alpha \cup \{\mathbf{S}_\alpha \mapsto S_\alpha, \mathbf{S}_\alpha \mapsto \varepsilon\}, \mathbf{S}_\alpha)$ , wobei  $\tilde{R}_\alpha$  gerade  $R_\alpha$  bei allen Regeln mit Variablen in der rechten Seite entspricht, und bei Regeln ohne Variablen auf der rechten Seite um  $\mathbf{S}_\alpha$  optional ergänzt werden.

□

Später wird in Satz 3.34 gezeigt, dass auch zu jeder regulären Sprache ein äquivalenter regulärer Ausdruck gefunden werden kann.

BEISPIEL 2.41. Die Grammatik und Sprache zu  $M_3 = \{0\} \cup \{1\} \times \{0,1\}^*$  aus den Beispielen 2.4 und 1.10 über  $\Sigma = \{0,1,2\}$ ,

$$\begin{aligned} G &= (V, T, R, \mathbf{Start}), \quad V = \{\mathbf{Start}, \mathbf{A}\}, \quad T = \{0, 1\}, \\ R &= \{r_1, r_2\}, \quad \text{wo} \\ r_1 : \quad \mathbf{Start} &\mapsto 0 \mid 1 \mid \mathbf{1A}, \\ r_2 : \quad \mathbf{A} &\mapsto 0 \mid 1 \mid 0\mathbf{A} \mid \mathbf{1A}. \end{aligned}$$

ist durch regulären Ausdruck  $\alpha = 0 \mid 1(0 \mid 1)^*$  darstellbar. Die Darstellung ist aber nicht eindeutig, auch  $\beta = 0 \mid 1 \mid 1(0 \mid 1)^*$  ist ebenso ein zur Grammatik oder Sprachbeschreibung äquivalenter Ausdruck.

Mit regulären Ausdrücken lassen sich endliche formale Sprachen einordnen:

SATZ 2.42. *Alle endlichen formalen Sprachen sind reguläre Sprachen.*

BEWEIS. Eine endliche formale Sprache  $L$  besteht aus endlich vielen Worten  $L = \{w_1, \dots, w_n\}$ . Diese Sprache wird auch durch den regulären Ausdruck  $w_1 \mid \dots \mid w_n$  erzeugt und daher ist die Sprache regulär. □

BEISPIEL 2.43. Reguläre Ausdrücke sind nicht nur effiziente Darstellungen regulärer Sprachen, sondern sind auch in der Softwareentwicklung, Mustererkennung und Datenauswertung sehr wertvolle Hilfsmittel.

Zur Unterscheidung von den hier zentral behandelten formalen regulären Ausdrücken aus Definition 2.34 wird die in der praktischen Nutzung typische Darstellung hier als **Regular Expressions** bezeichnet. Die Regular Expressions basieren grundsätzlich auf dem ASCII oder Unicode-Alphabet und es gibt eine Vielzahl von Erweiterungen, Anpassungen und sogar Variablen, die in verschiedenen Anwendungen sehr hilfreich sein können. Es gibt viele unterschiedliche Syntaxvarianten für Regular Expressions wie für Java, JavaScript, PHP, Python, Swift, C#, Go, grep und viele mehr, und Tabelle 1 zeigt einen Teil der Syntax, den die meisten Varianten unterstützen.

Sollen beispielsweise in einer Unix-Shell alle Dateien gefunden werden, die auf .jpg, .JPG, .jpeg oder .JPEG enden, so könnte dies mit dieser Zeile erfolgen:

```
$ find . | egrep "\.(jpe?g|JPE?G)$"
```

Der Befehl `find .` listet alle Dateien im Verzeichnis und Unterverzeichnissen auf. Durch die Pipe `|` werden die Eingaben dann in das `grep`-Kommando weitergeleitet, das dann nur die Zeilen ausgibt, in denen es eine Entsprechung mit der regular expression gab. In diesem Beispiel wurde `egrep` verwendet, um die Klammerung und Fragezeichen einer erweiterten Regular Expression zu ermöglichen und sollte äquivalent zu `grep -E` sein. Ist keine Klammerung erforderlich, so kann einfach `grep` verwendet werden. Natürlich kann auch `grep -i` verwendet werden, damit die Groß- oder Kleinschreibung nicht beachtet wird und so die regular expression weiter vereinfacht werden kann:

| Abfolgen und Alternativen |                                |                        |                            |
|---------------------------|--------------------------------|------------------------|----------------------------|
|                           | XY                             | X nach Y               |                            |
|                           | X Y                            | X oder Y               |                            |
|                           | (X)                            | Klammerung und Abfrage |                            |
| Zeichen                   |                                | Positionen             |                            |
| x                         | Das Zeichen x                  | ^                      | Anfang der Zeile           |
| \\                        | Backslash \                    | \$                     | Ende der Zeile             |
| \.                        | Punkt .                        | \b                     | Wortgrenze                 |
| \t                        | Tabulator                      | \B                     | Nicht-Wortgrenze           |
| Mengen                    |                                | Anzahl                 |                            |
| .                         | Beliebiges Zeichen             | X?                     | einmal oder kein mal       |
| [abc]                     | Ein Zeichen aus Liste          | X*                     | kein mal oder beliebig oft |
| [^abc]                    | Ein Zeichen außer aus Liste    | X+                     | einmal oder beliebig oft   |
| [a - r]                   | Ein Zeichen aus Bereich        | X{n}                   | exakt n mal                |
| \d                        | Eine Ziffer [0 - 9]            | X{n,}                  | mindestens n mal           |
| \D                        | Keine Ziffer [^0 - 9]          | X{n,m}                 | von n bis m mal            |
| \w                        | Wortzeichen [a - zA - Z0 - 9_] | X*?                    | mit ? nicht gierig         |

TABELLE 1. (Extended) **Regular Expressions** - eine (unvollständige) Übersicht

```
$ find . | grep -Ei "\.jpe?g$"
```

Um in Java Strings zu durchsuchen, gibt es entsprechende Packages, die dies ermöglichen:

```
import java.util.regex.*;
public class TextSuche {
    public static void main(String[] args) {
        String text = "Hier_ein_kleiner_Text.";
        Pattern pattern = Pattern.compile("e.n"); // matches twice
        Matcher matcher = pattern.matcher(text);
        while(matcher.find())
            System.out.println(">" + matcher.start() + "-" + matcher.end() + " ' "
                               + text.substring(matcher.start(), matcher.end()) + "'");
    }
}
```

## 2.6. Aufgaben

AUFGABE 19. Bestimmen Sie die Sprache  $L(G)$  für  $G = (V, T, R, S)$ :

$$\begin{aligned} V &= \{A, B, C\}, \\ T &= \{e, r, s, w\}, \\ R &= \{r_1, r_2, r_3\}, \\ &\quad r_1 : A \mapsto Bw \mid wsC, \\ &\quad r_2 : B \mapsto Cr, \\ &\quad r_3 : C \mapsto e \mid s, \\ S &= A. \end{aligned}$$

AUFGABE 20. Bestimmen Sie die Sprache  $L(G)$  für  $G = (V, T, R, S)$ :

$$\begin{aligned} V &= \{A, B, C\}, \\ T &= \{i, k, r, t\}, \\ R &= \{r_1, r_2, r_3\}, \\ &\quad r_1 : A \mapsto Bk \mid r, \\ &\quad r_2 : B \mapsto tir, \\ &\quad r_3 : C \mapsto rtB \mid tA \mid tr, \\ S &= C. \end{aligned}$$

AUFGABE 21. Bestimmen Sie die Sprache  $L(G)$  für  $G = (V, T, R, S)$ :

$$\begin{aligned} V &= \{A, B, C\}, \\ T &= \{i, o, r, s\}, \\ R &= \{r_1, r_2, r_3\}, \\ &\quad r_1 : A \mapsto i \mid o \mid so, \\ &\quad r_2 : B \mapsto iC \mid rro \mid sA \mid so, \\ &\quad r_3 : C \mapsto Ar, \\ S &= B. \end{aligned}$$

AUFGABE 22. Bestimmen Sie die Sprache  $L(G)$  für  $G = (V, T, R, S)$ :

$$\begin{aligned} V &= \{A, B, C\}, \\ T &= \{i, r, t, w\}, \\ R &= \{r_1, r_2, r_3\}, \\ &\quad r_1 : A \mapsto Ct \mid ri, \\ &\quad r_2 : B \mapsto Air \mid riC, \\ &\quad r_3 : C \mapsto t \mid w, \\ S &= B. \end{aligned}$$

AUFGABE 23. Bestimmen Sie die Sprache  $L(G)$  für  $G = (V, T, R, S)$ :

$$\begin{aligned} V &= \{A, B, C\}, \\ T &= \{k, o, t\}, \\ R &= \{r_1, r_2, r_3, r_4\}, \\ &\quad r_1 : A \mapsto Bt \mid Co, \\ &\quad r_2 : B \mapsto Ct, \\ &\quad r_3 : C \mapsto k \mid o, \\ &\quad r_4 : Ctt \mapsto o \mid ok, \\ S &= A. \end{aligned}$$

AUFGABE 24. Bestimmen Sie die Sprache  $L(G)$  für  $G = (V, T, R, S)$ :

$$\begin{aligned}
 V &= \{\text{A, B, C}\}, \\
 T &= \{\text{e, r, u, w}\}, \\
 R &= \{r_1, r_2, r_3, r_4\}, \\
 &\quad r_1: \text{A} \mapsto \text{r} \mid \text{u} \mid \text{weCB} \mid \text{wuBe}, \\
 &\quad r_2: \text{B} \mapsto \text{r}, \\
 &\quad r_3: \text{C} \mapsto \text{uBBBe}, \\
 &\quad r_4: \text{eB} \mapsto \text{wr}, \\
 S &= \text{A}.
 \end{aligned}$$

AUFGABE 25. Wie wird das Wort **ewtiewet** aus der Grammatik  $G = (V, T, R, S)$  abgeleitet?

$$\begin{aligned}
 V &= \{\text{P, Q, R, S}\}, \\
 T &= \{\text{e, i, t, w}\}, \\
 R &= \{r_1, r_2, r_3, r_4, r_5\}, \\
 &\quad r_1: \text{P} \mapsto \text{i} \mid \text{wQ}, \\
 &\quad r_2: \text{Q} \mapsto \text{et} \mid \text{we} \mid \text{wit}, \\
 &\quad r_3: \text{R} \mapsto \text{Qwt} \mid \text{tieP}, \\
 &\quad r_4: \text{S} \mapsto \text{Pe} \mid \text{ewR} \mid \text{i} \mid \text{wQwe}, \\
 &\quad r_5: \text{wtieP} \mapsto \text{wtietie}, \\
 S &= \text{S}.
 \end{aligned}$$

AUFGABE 26. Geben Sie die Ableitung für das Wort **etrtrtse** aus der Grammatik  $G = (V, T, R, S)$  an.

$$\begin{aligned}
 V &= \{\text{X, Y, Z}\}, \\
 T &= \{\text{e, r, s, t}\}, \\
 R &= \{r_1, r_2, r_3\}, \\
 &\quad r_1: \text{X} \mapsto \text{rts}, \\
 &\quad r_2: \text{Y} \mapsto \text{etZ} \mid \text{reX}, \\
 &\quad r_3: \text{Z} \mapsto \text{rXe} \mid \text{srt} \mid \text{tse}, \\
 S &= \text{Y}.
 \end{aligned}$$

AUFGABE 27. Bestimmen Sie den Chomsky-Typ der Grammatik  $G = (V, T, R, S)$  und geben Sie eine Ableitung für das Wort **okoik** an.

$$\begin{aligned}
 V &= \{\text{X, Y, Z}\}, \\
 T &= \{\text{i, k, o, s}\}, \\
 R &= \{r_1, r_2, r_3, r_4, r_5\}, \\
 &\quad r_1: \text{X} \mapsto \text{io} \mid \text{isk} \mid \text{ok}, \\
 &\quad r_2: \text{Xo} \mapsto \text{ikso} \mid \text{ko} \mid \text{okio} \mid \text{oso}, \\
 &\quad r_3: \text{Y} \mapsto \text{Xoik} \mid \text{k} \mid \text{o} \mid \text{s}, \\
 &\quad r_4: \text{Z} \mapsto \text{oY}, \\
 &\quad r_5: \text{oXo} \mapsto \text{oko} \mid \text{osioo}, \\
 S &= \text{Z}.
 \end{aligned}$$

**AUFGABE 28.** Bestimmen Sie den Chomsky-Typ der Grammatik  $G = (V, T, R, S)$  und geben Sie eine Ableitung für das Wort **rw** an.

$$\begin{aligned}
 V &= \{\mathbf{X}, \mathbf{Y}, \mathbf{Z}\}, \\
 T &= \{\mathbf{o}, \mathbf{r}, \mathbf{t}, \mathbf{w}\}, \\
 R &= \{r_1, r_2, r_3, r_4, r_5, r_6\}, \\
 &\quad r_1 : \mathbf{X} \mapsto \mathbf{o} \mid \mathbf{r} \mid \mathbf{wY}, \\
 &\quad r_2 : \mathbf{Y} \mapsto \mathbf{oo} \mid \mathbf{tor} \mid \mathbf{wt}, \\
 &\quad r_3 : \mathbf{Yo} \mapsto \mathbf{owr} \mid \mathbf{tw}, \\
 &\quad r_4 : \mathbf{Z} \mapsto \mathbf{oYo} \mid \mathbf{ror} \mid \mathbf{rwt} \mid \mathbf{wX}, \\
 &\quad r_5 : \mathbf{oY} \mapsto \mathbf{o} \mid \mathbf{wtr}, \\
 &\quad r_6 : \mathbf{wwY} \mapsto \mathbf{orw} \mid \mathbf{rw}, \\
 S &= \mathbf{Z}.
 \end{aligned}$$

**AUFGABE 29.** Bestimmen Sie den Chomsky-Typ der Grammatik  $G = (V, T, R, S)$  und geben Sie eine Ableitung für das Wort **kuki** an.

$$\begin{aligned}
 V &= \{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}\}, \\
 T &= \{\mathbf{e}, \mathbf{i}, \mathbf{k}, \mathbf{u}\}, \\
 R &= \{r_1, r_2, r_3, r_4\}, \\
 &\quad r_1 : \mathbf{A} \mapsto \mathbf{iC} \mid \mathbf{kB} \mid \mathbf{uD}, \\
 &\quad r_2 : \mathbf{B} \mapsto \mathbf{e} \mid \mathbf{iC} \mid \mathbf{u} \mid \mathbf{uD}, \\
 &\quad r_3 : \mathbf{C} \mapsto \mathbf{i} \mid \mathbf{k} \mid \mathbf{u}, \\
 &\quad r_4 : \mathbf{D} \mapsto \mathbf{kC} \mid \mathbf{u}, \\
 S &= \mathbf{A}.
 \end{aligned}$$

**AUFGABE 30.** Bestimmen Sie den Chomsky-Typ der Grammatik  $G = (V, T, R, S)$  und geben Sie eine Ableitung für das Wort **iiskkski** an.

$$\begin{aligned}
 V &= \{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}\}, \\
 T &= \{\mathbf{i}, \mathbf{k}, \mathbf{s}, \mathbf{u}\}, \\
 R &= \{r_1, r_2, r_3, r_4\}, \\
 &\quad r_1 : \mathbf{A} \mapsto \mathbf{isBs}, \\
 &\quad r_2 : \mathbf{B} \mapsto \mathbf{ik} \mid \mathbf{kD} \mid \mathbf{u} \mid \mathbf{uiu}, \\
 &\quad r_3 : \mathbf{C} \mapsto \mathbf{iAki} \mid \mathbf{kB}, \\
 &\quad r_4 : \mathbf{D} \mapsto \mathbf{ik} \mid \mathbf{k} \mid \mathbf{s} \mid \mathbf{u}, \\
 S &= \mathbf{C}.
 \end{aligned}$$

**AUFGABE 31.** Bestimmen Sie den Chomsky-Typ von  $G = (V, T, R, S)$  und die Sprache  $L(G)$ :

$$\begin{aligned}
 V &= \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}, \\
 T &= \{\mathbf{r}, \mathbf{u}\}, \\
 R &= \{r_1, r_2, r_3, r_4, r_5\}, \\
 &\quad r_1 : \mathbf{A} \mapsto \mathbf{uB}, \\
 &\quad r_2 : \mathbf{B} \mapsto \mathbf{r}, \\
 &\quad r_3 : \mathbf{Bir} \mapsto \mathbf{ru} \mid \mathbf{u} \mid \mathbf{ur}, \\
 &\quad r_4 : \mathbf{C} \mapsto \mathbf{AiB} \mid \mathbf{r} \mid \mathbf{rB} \mid \mathbf{u}, \\
 &\quad r_5 : \mathbf{riB} \mapsto \mathbf{u}, \\
 S &= \mathbf{C}.
 \end{aligned}$$

AUFGABE 32. Bestimmen Sie den Chomsky-Typ von  $G = (V, T, R, S)$  und die Sprache  $L(G)$ :

$$\begin{aligned}
 V &= \{\mathbf{W}, \mathbf{X}, \mathbf{Y}, \mathbf{Z}\}, \\
 T &= \{\mathbf{e}, \mathbf{i}, \mathbf{t}, \mathbf{w}\}, \\
 R &= \{r_1, r_2, r_3, r_4\}, \\
 &\quad r_1 : \mathbf{W} \mapsto \mathbf{i}, \\
 &\quad r_2 : \mathbf{X} \mapsto \mathbf{Wi} \mid \mathbf{Zw}, \\
 &\quad r_3 : \mathbf{Y} \mapsto \mathbf{Xt}, \\
 &\quad r_4 : \mathbf{Z} \mapsto \mathbf{Wt} \mid \mathbf{e} \mid \mathbf{t} \mid \mathbf{w}, \\
 S &= \mathbf{Y}.
 \end{aligned}$$

AUFGABE 33. Bestimmen Sie den Chomsky-Typ von  $G = (V, T, R, S)$  und die Sprache  $L(G)$ :

$$\begin{aligned}
 V &= \{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D}\}, \\
 T &= \{\mathbf{i}, \mathbf{o}, \mathbf{s}, \mathbf{u}\}, \\
 R &= \{r_1, r_2, r_3, r_4\}, \\
 &\quad r_1 : \mathbf{A} \mapsto \mathbf{Co} \mid \mathbf{o}, \\
 &\quad r_2 : \mathbf{B} \mapsto \mathbf{iCu} \mid \mathbf{iDu} \mid \mathbf{uA}, \\
 &\quad r_3 : \mathbf{C} \mapsto \mathbf{is}, \\
 &\quad r_4 : \mathbf{D} \mapsto \mathbf{usoA}, \\
 S &= \mathbf{B}.
 \end{aligned}$$

AUFGABE 34. Bestimmen Sie den Chomsky-Typ von  $G = (V, T, R, S)$  und die Sprache  $L(G)$ :

$$\begin{aligned}
 V &= \{\mathbf{X}, \mathbf{Y}, \mathbf{Z}\}, \\
 T &= \{\mathbf{i}, \mathbf{k}, \mathbf{o}\}, \\
 R &= \{r_1, r_2, r_3, r_4\}, \\
 &\quad r_1 : \mathbf{X} \mapsto \mathbf{Zk} \mid \mathbf{i} \mid \mathbf{o}, \\
 &\quad r_2 : \mathbf{Y} \mapsto \mathbf{Xo}, \\
 &\quad r_3 : \mathbf{Z} \mapsto \mathbf{io}, \\
 &\quad r_4 : \mathbf{Zko} \mapsto \mathbf{kiko} \mid \mathbf{okko}, \\
 S &= \mathbf{Y}.
 \end{aligned}$$

AUFGABE 35. Bestimmen Sie den Chomsky-Typ von  $G = (V, T, R, S)$  und die Sprache  $L(G)$ :

$$\begin{aligned}
 V &= \{\mathbf{P}, \mathbf{Q}, \mathbf{R}, \mathbf{S}, \mathbf{T}\}, \\
 T &= \{\mathbf{e}, \mathbf{i}, \mathbf{o}, \mathbf{u}\}, \\
 R &= \{r_1, r_2, r_3, r_4, r_5\}, \\
 &\quad r_1 : \mathbf{P} \mapsto \mathbf{eQ} \mid \mathbf{oT}, \\
 &\quad r_2 : \mathbf{Q} \mapsto \mathbf{e} \mid \mathbf{i} \mid \mathbf{iS} \mid \mathbf{u}, \\
 &\quad r_3 : \mathbf{R} \mapsto \mathbf{iP}, \\
 &\quad r_4 : \mathbf{S} \mapsto \mathbf{i}, \\
 &\quad r_5 : \mathbf{T} \mapsto \mathbf{oQ}, \\
 S &= \mathbf{R}.
 \end{aligned}$$

**AUFGABE 36.** Bestimmen Sie den Chomsky-Typ von  $G = (V, T, R, S)$  und die Sprache  $L(G)$ :

$$\begin{aligned}
 V &= \{A, B, C, D\}, \\
 T &= \{e, r, s\}, \\
 R &= \{r_1, r_2, r_3, r_4, r_5\}, \\
 &\quad r_1 : A \mapsto Ds \mid re, \\
 &\quad r_2 : B \mapsto Ae, \\
 &\quad r_3 : C \mapsto eB, \\
 &\quad r_4 : D \mapsto er, \\
 &\quad r_5 : eD \mapsto ee \mid er \mid ere \mid esr, \\
 S &= C.
 \end{aligned}$$

**AUFGABE 37.** Ein Expertteam soll durch eine Sprache verschiedene Entscheidungsstrategien abbilden können. Die Entscheidungen werden auf der Basis von Variablen im Vergleich mit anderen Variablen oder Werten beschrieben, die mit „and“ oder „or“ verbunden werden können. Ist ein Bedingungsterm erfüllt, so wird eine Aktion ausgeführt, optional kann eine Aktion ausgeführt werden, falls die Bedingung nicht gilt. Eine Strategie kann aus mehreren Entscheidungen bestehen, Werte sind ganzzahlige Zahlen und Bezeichner für Werte oder Aktionen werden mit Wörtern aus Großbuchstaben beschrieben. Exemplarisch kann eine Strategie so aussehen:

```

if PERF>20 and PRICE<50 then BUY.
if PERF==80 then RESERVE.
if PERF<=10 or PRICE>COST or AGE>=90 then SELL else EVALUATE.

```

Entwerfen Sie eine Sprache vom Chomsky-Typ 2 (ohne Berücksichtigung von Whitespaces) auf dem Alphabet  $\Sigma_{\text{Unicode}}$  und eine EBNF-Beschreibung in <https://www.lark-parser.org/ide/>, die das Beispiel korrekt analysieren kann.

**AUFGABE 38.** Eine Netzwerkstruktur soll aus Switches, Geräten und den Verbindungen beschrieben werden. Die Switches und Geräte werden durch Bezeichner identifiziert. Bezeichner bestehen aus kleinen Buchstaben gefolgt von weiteren kleinen Buchstaben oder Ziffern. Switches können optional mit einer ganzzahligen Anzahl von Ports definiert werden. Geräte werden optional mit Bandbreiten mit Ziffern mit Suffixen T, G, M, K für entsprechende Bandbreitenanforderungen spezifiziert. Hier ein Beispiel für eine mögliche Spezifikation:

```

SWITCH blue1 PORTS 24.
SWITCH black22.
DEVICE pc19 BANDWIDTH 500M.
DEVICE laptop2.
CONNECT pc19 TO blue1.
CONNECT laptop2 TO black22.
CONNECT blue1 TO black22.

```

Entwerfen Sie eine Sprache vom Chomsky-Typ 2 (ohne Berücksichtigung von Whitespaces) auf dem Alphabet  $\Sigma_{\text{Unicode}}$  und eine EBNF-Beschreibung in <https://www.lark-parser.org/ide/>, die das Beispiel korrekt analysieren kann.

**AUFGABE 39.** Komprimieren Sie den folgenden Ausdruck mit dem LZW-Verfahren. Das initiale Alphabet sind alle Großbuchstaben mit gleichlautenden Variablen und alle neuen Variablen werden  $w_1, w_2, \dots$  benannt:

NACHRICHTENAGENTUREN

**AUFGABE 40.** Dekomprimieren Sie den folgenden Ausdruck, der mit dem LZW-Verfahren komprimiert wurde. Das initiale Alphabet waren alle Großbuchstaben mit gleichlautenden Variablen und alle neuen Variablen wurden  $w_1, w_2, \dots$  benannt:

SPITZEN $w_5w_3w_6$

**AUFGABE 41.** Bestimmen Sie einen möglichst kurzen regulären Ausdruck  $\alpha \in \mathcal{R}(T)$ , für den  $L(\alpha) = L(G)$  für die Grammatik  $G = (V, T, R, S)$  gilt:

$$\begin{aligned} V &= \{A, B, C\}, \\ T &= \{o, r, s\}, \\ R &= \{r_1, r_2, r_3\}, \\ &\quad r_1 : A \mapsto rC \mid sB, \\ &\quad r_2 : B \mapsto rC \mid oB, \\ &\quad r_3 : C \mapsto o \mid s, \\ S &= A. \end{aligned}$$

**AUFGABE 42.** Bestimmen Sie einen möglichst kurzen regulären Ausdruck  $\alpha \in \mathcal{R}(T)$ , für den  $L(\alpha) = L(G)$  für die Grammatik  $G = (V, T, R, S)$  gilt:

$$\begin{aligned} V &= \{A, B, C\}, \\ T &= \{e, i, r, s\}, \\ R &= \{r_1, r_2, r_3\}, \\ &\quad r_1 : A \mapsto Br \mid Cr \mid Ai, \\ &\quad r_2 : B \mapsto Ce \mid i, \\ &\quad r_3 : C \mapsto r \mid s, \\ S &= A. \end{aligned}$$

**AUFGABE 43.** Bestimmen Sie eine rechts-lineare Typ-3 Grammatik  $G = (V, T, R, S)$  für das Alphabet  $T = \{e, r, s, t\}$ , für die  $L(G) = L(\alpha)$  mit  $\alpha = r((s|t)^*|e)^*$  gilt.

**AUFGABE 44.** Bestimmen Sie eine links-lineare Typ-3 Grammatik  $G = (V, T, R, S)$  für das Alphabet  $T = \{a, b, k, n\}$ , für die  $L(G) = L(\alpha)$  mit  $\alpha = b^*an(k|a)^*$  gilt.

**AUFGABE 45.** Vereinfachen Sie den regulären Ausdruck  $\alpha = (a^*b^*a^*|aba)^*(a|ac^*|aba)$  zu einem äquivalenten kürzeren Ausdruck  $\beta$  mit  $L(\alpha) = L(\beta)$ .

**AUFGABE 46.** Vereinfachen Sie den regulären Ausdruck  $\alpha = (ab)^*a(ba)^*(abc|acc|abbc)$  zu einem äquivalenten kürzeren Ausdruck  $\beta$  mit  $L(\alpha) = L(\beta)$ .



## KAPITEL 3

### Rechnermodelle

Computer können auf der Basis von Vorschriften Daten verarbeiten. So wie es sehr viele unterschiedliche Geräte geben kann, die in diesem Sinne als Computer bezeichnet werden können, so gibt es auch sehr viele Methoden, wie Vorschriften die Verarbeitung beschreiben können. Die Automatentheorie und Turingmaschinen liefern ganz grundsätzliche Modelle, wie ein datenverarbeitendes Gerät und entsprechende Vorschriften beschrieben werden können, für die durch ihre unterschiedlichen Fähigkeiten sich sehr unterschiedliche Anwendungsszenarien ergeben. Sie können die heute gängigen Computer beschreiben und ermöglichen uns damit auch, die Komplexität von Algorithmen auf diesen Architekturen zu analysieren. Darüber hinaus gibt auch die Modelle wie für Quantencomputer, mit denen Probleme, die auf klassischen Rechnermodellen viel Aufwand benötigen, dann plötzlich deutlich schneller zu lösen sind.

Im Folgenden konzentriert sich die Betrachtung der Rechnermodelle auf **Akzeptoren**, also hauptsächlich auf die Frage der Akzeptanz von Sprachen. Mit den Betrachtungen des vorherigen Kapitels zeigt sich, dass damit sehr allgemein gearbeitet werden kann. Tatsächlich führt beispielsweise die Erweiterung von Akzeptoren zu **Transduktoren**, die außer der Akzeptanz auch Ausgaben erzeugen, zu einer erweiterten Theorie und weiteren Anwendungen beispielsweise für Steuerungen.

#### 3.1. Endliche Automaten

Eine erste grundsätzliche Annahme ist, dass Geräte sich in fest definierten Zuständen befinden können. Je nach Eingaben von Außen kann sich ein Zustand verändern. Zusammen mit der Annahme, dass die Anzahl von Zuständen endlich ist, es einen definierten Start und mögliche „positive“ Endpunkte geben kann, ergibt sich ein einfachstes Modell für eine Verarbeitungseinheit. Bezogen auf die Aufgabe die Wörter einer Sprache zu identifizieren, bedeutet das Erreichen eines Endpunkts, dass ein Wort der Sprache im Rahmen der Zustandsänderungen akzeptiert wurde.

Einfache Beispiele für solche Automationen mit definierten Zuständen sind beispielsweise Anforderungsampeln, die nach Knopfdruck und Signalen nach Zeitabschnitten zwischen verschiedenen Ampelphasen umschalten oder auch Steuerungen für Aufzüge. Auch die Geldannahme für einen Getränkeautomaten sollte genau definiert nur nach bestimmten passenden Einwurfskombinationen die Möglichkeit zum Getränkeauswahl ermöglichen.

**DEFINITION 3.1.** Ein **deterministischer endlicher Automat (DEA)** ist ein Tupel  $M = (Q, \Sigma, \delta, q_s, E)$ , wo  $Q$  die endliche Menge von Zuständen,  $\Sigma$  das Alphabet möglicher Eingaben,  $\delta : Q \times \Sigma \rightarrow Q$  die Übergangsfunktion,  $q_s \in E$  den Startzustand und  $E \subseteq Q$  die Menge der Endzustände beschreibt.

Da Wörter oft aus mehreren Symbolen bestehen, hilft die Definition der iterierten Übergangsfunktion:

**DEFINITION 3.2.** Die Funktion  $\delta^* : Q \times \Sigma^* \rightarrow Q$  sei die **iterierte Übergangsfunktion** von  $\delta$ , die induktiv so definiert ist:

$$\delta^*(q, w) = \begin{cases} q & , \text{ wenn } w = \varepsilon, \\ \delta^*(\delta(q, x), y) & , \text{ wenn } w = xy \text{ für } x \in \Sigma, y \in \Sigma^*. \end{cases}$$

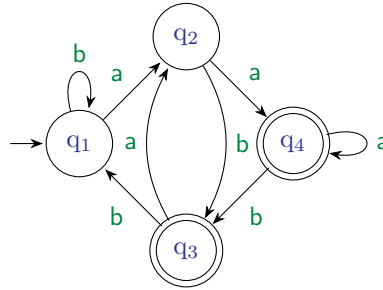


ABBILDUNG 3.1.1. Zustandsdiagramm des Automaten aus Beispiel 3.3

BEISPIEL 3.3. Gegeben sei ein deterministischer endlicher Automat  $M = (Q, \Sigma, \delta, q_s, E)$  mit  $Q = \{q_1, q_2, q_3, q_4\}$ ,  $\Sigma = \{a, b\}$ ,  $q_s = q_1$  und  $E = \{q_3, q_4\}$  mit

$$\begin{aligned} \delta(q_1, a) &= q_2, & \delta(q_2, a) &= q_4, & \delta(q_3, a) &= q_4, & \delta(q_4, a) &= q_4, \\ \delta(q_1, b) &= q_1, & \delta(q_2, b) &= q_3, & \delta(q_3, b) &= q_1, & \delta(q_4, b) &= q_3. \end{aligned}$$

Die Übergangsfunktion kann auch übersichtlicher mit einer Tabelle beschrieben werden:

| $\delta$ | $q_1$ | $q_2$ | $q_3$ | $q_4$ |
|----------|-------|-------|-------|-------|
| a        | $q_2$ | $q_4$ | $q_4$ | $q_4$ |
| b        | $q_1$ | $q_3$ | $q_1$ | $q_3$ |

Damit kann der Zustand des Automaten für beliebige Eingabewörter  $w \in \Sigma^*$  bestimmt werden:

$$\begin{aligned} \delta^*(q_s, abba) &= q_2, \text{ da } q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_3 \xrightarrow{b} q_1 \xrightarrow{a} q_2, \\ \delta^*(q_s, baab) &= q_3, \text{ da } q_1 \xrightarrow{b} q_1 \xrightarrow{a} q_2 \xrightarrow{a} q_4 \xrightarrow{b} q_3. \end{aligned}$$

Da  $\delta^*(q_s, abba) = q_2 \notin E$  und  $\delta^*(q_s, baab) = q_3 \in E$  wird **abba** nicht akzeptiert, hingegen **baab** ist ein akzeptiertes Wort.

**DEFINITION 3.4. Das Zustandsdiagramm** eines deterministischen endlichen Automaten  $M$  mit  $M = (Q, \Sigma, \delta, q_s, E)$  besteht aus einem gerichteten Graphen mit Knoten  $Q$  und mit Kanten von  $q_i$  nach  $q_k$  mit Bezeichnung  $x$ , wenn  $\delta(q_i, x) = q_k$ . Parallel verlaufende Kanten mit identischem Ursprung und Ziel können zu einer Kante mit allen Symbolen beschrifteten Kante zusammengefasst werden. Die Knoten in  $E$  werden durch doppelte Kreise dargestellt und es gibt eine Kante ohne Quelle nach  $q_s$ .

Das Zustandsdiagramm des Automaten aus Beispiel 3.3 ist in Abbildung 3.1.1 dargestellt. Die Abbildungen 3.1.2 und 3.1.3 zeigen, wie endliche Automaten mit **GraphViz** oder mit **TikZ/PGF** in der Form eines Zustandsdiagramms in Dokumenten oder Bildern visualisiert werden können.

**DEFINITION 3.5. Die Sprache des deterministischen endlichen Automaten**  $M = (Q, \Sigma, \delta, q_s, E)$ :

$$L(M) = \{w \in \Sigma^* \mid \delta^*(q_s, w) \in E\}.$$

BEISPIEL 3.6. Um die Sprache des Automaten  $M$  aus Beispiel 3.3 zu bestimmen, können zunächst die kürzesten Worte bestimmt werden, die akzeptiert werden. Von  $q_s = q_1$  müssen mindestens zwei Symbole gelesen werden:

$$\begin{aligned} \delta^*(q_s, aa) &= q_4 \in E, \text{ da } q_1 \xrightarrow{a} q_2 \xrightarrow{a} q_4, \\ \delta^*(q_s, ab) &= q_3 \in E, \text{ da } q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_3. \end{aligned}$$

# Datei beispiel.dot einfach als Bild in LyX einbinden,  
 # oder für LaTeX mit dot in svg oder pdf wandeln. Auch <https://t1p.de/pg3an> .  
 digraph finite\_state\_machine

```
{
    rankdir = LR;
    node [shape = doublecircle]; q3;
    node [shape = doublecircle]; q4;
    node [shape = point ]; S;

    node [shape = circle];
    S -> q1;
    q1 -> q2 [ label = "a" ]; q1 -> q1 [ label = "b" ];
    q2 -> q4 [ label = "a" ]; q2 -> q3 [ label = "b" ];
    q3 -> q2 [ label = "a" ]; q3 -> q1 [ label = "b" ];
    q4 -> q4 [ label = "a" ]; q4 -> q3 [ label = "b" ];
}
```

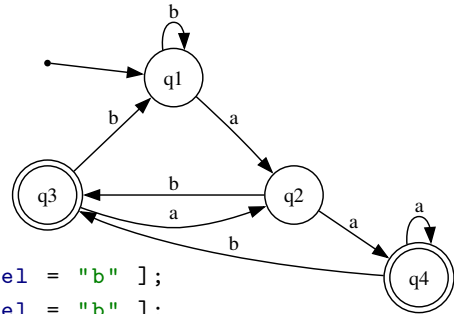


ABBILDUNG 3.1.2. Endliche Automaten automatisch mit GraphViz visualisieren

```
% Direkt in LaTeX oder als LaTeX-Inline in LyX eingeben
% Präambel: \usepackage{tikz} \usetikzlibrary{arrows, automata}
\begin{tikzpicture}[>=stealth, shorten >=1pt, auto, node distance=2cm,
    initial text={}, double distance=2pt]
\node[initial, state] (q1) {$q_1$};
\node[state] (q2) [above right of=q1] {$q_2$};
\node[state, accepting] (q3) [below right of=q1] {$q_3$};
\node[state, accepting] (q4) [below right of=q2] {$q_4$};

\path[->] (q1) edge node {$a$} (q2);
\path[->] (q1) edge [loop above] node {$b$} (q1);
\path[->] (q2) edge node {$a$} (q4);
\path[->] (q2) edge [bend left] node {$b$} (q3);
\path[->] (q3) edge [bend left] node {$a$} (q2);
\path[->] (q3) edge node {$b$} (q1);
\path[->] (q4) edge [loop right] node {$a$} (q4);
\path[->] (q4) edge node {$b$} (q3);
\end{tikzpicture}
```

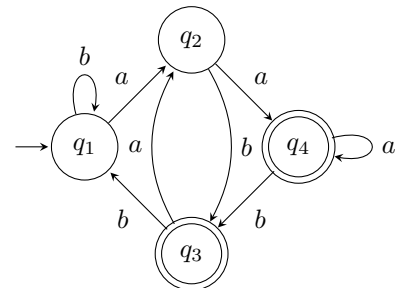


ABBILDUNG 3.1.3. Endliche Automaten mit TikZ/PGF mit LaTeX-Symbolen setzen

Mit drei Symbolen gibt es vier weitere Möglichkeiten:

$$\delta^*(q_s, \text{aaa}) = q_4 \in E, \text{ da } q_1 \xrightarrow{a} q_2 \xrightarrow{a} q_4 \xrightarrow{a} q_4,$$

$$\delta^*(q_s, \text{aab}) = q_4 \in E, \text{ da } q_1 \xrightarrow{a} q_2 \xrightarrow{a} q_4 \xrightarrow{b} q_3,$$

$$\delta^*(q_s, \text{baa}) = q_3 \in E, \text{ da } q_1 \xrightarrow{b} q_1 \xrightarrow{a} q_2 \xrightarrow{a} q_4,$$

$$\delta^*(q_s, \text{bab}) = q_3 \in E, \text{ da } q_1 \xrightarrow{b} q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_3.$$

Es scheint so zu sein, dass ein Wort  $w = (w_1, \dots, w_{n-1}, w_n) \in L(M)$ , genau dann wenn  $w_{n-1} = a$  ist. Diese Vermutung kann auch bewiesen werden: Hat einerseits  $w$  als vorletztes Symbol ein  $a$ , so endet  $w$  auf  $aa$  oder auf  $ab$ :

- (1) Endet  $w$  auf  $aa$ , so gilt  $\delta^*(q, aa) = q_4 \in E$  für alle  $q \in Q$ .
- (2) Endet  $w$  auf  $ab$ , so gilt  $\delta^*(q, ab) = q_3 \in E$  für alle  $q \in Q$ .

Endet hingegen  $w$  auf  $ba$  oder  $bb$ , so gilt:

- (1) Endet  $w$  auf  $ba$ , so gilt  $\delta^*(q, ab) = q_2 \notin E$  für alle  $q \in Q$ .
- (2) Endet  $w$  auf  $bb$ , so gilt  $\delta^*(q, bb) = q_1 \notin E$  für alle  $q \in Q$ .

Da sowohl der Automat vor den letzten beiden Buchstaben in einem Zustand  $q \in Q$  sein muss und auch alle Möglichkeiten für die letzten beiden Symbole aufgeführt sind, ist damit bewiesen, dass

$$L(M) = \{xaz \mid x \in \Sigma^*, z \in \Sigma\}.$$

Ein regulärer Ausdruck mit  $L(\alpha) = L(M)$  lautet entsprechend  $\alpha = (a|b)^*a(a|b)$ .

**DEFINITION 3.7.** Die **Gleichheit von Sprachen von Automaten, Grammatiken** oder regulären Ausdrücken wird kann als Relation geschrieben werden: Sind beispielsweise  $\alpha, \beta \in \mathcal{R}(\Sigma)$ , so ist

$$\alpha \equiv \beta \iff L(\alpha) = L(\beta).$$

Mit dem Lemma von Arden kann ein allgemeines Verfahren angegeben werden, wie Automaten in reguläre Ausdrücke übersetzt werden können:

**LEMMA 3.8. (Lemma von Arden)** Sind  $\alpha, \beta, \gamma \in \mathcal{R}(\Sigma)$  und  $\varepsilon \notin L(\beta)$ , so gilt  $\alpha \equiv \beta\alpha \mid \gamma \Rightarrow \alpha \equiv \beta^*\gamma$ .

**BEWEIS.** Das Lemma von Arden ist sogar allgemeiner auf formalen Sprachen gültig. Seien  $L, B, C \subseteq \Sigma^*$  mit  $\varepsilon \notin B$ , so gilt  $L = BL \cup C \Rightarrow L = B^*C$ . Diese Aussage umfasst die zu beweisende Behauptung und wird hier bewiesen: Sei nun  $L = BL \cup C$  und zu zeigen ist  $L = B^*C$ .

- (1) Teilmengenrichtung:  $L \supseteq B^*C$ . Sei  $w \in B^*C$  und zu zeigen ist, dass dann  $w \in L$ .

Aus der Prämisse folgt, dass  $w = w_n = b_n b_{n-1} \dots b_1 c$  mit Wörtern  $b_k \in B$  mit  $n = 0$  möglich und  $c \in C$ . Da nach Voraussetzung  $L = BL \cup C$  ist beginnend mit  $k = n$

- (a) entweder  $w_k = b_k b_{k-1} \dots b_1 c \in L$ , und das gilt mindestens falls  $k = 0$ , da  $w_0 = c \in C \subseteq L$ , oder
- (b)  $w = b_k \underbrace{b_{k-1} \dots b_1 c}_{=w_{k-1}} = b_k w_{k-1} \in BL$  mit  $b_k \in B$  bekannt und es wird gezeigt, dass

$w_{k-1} \in L$  durch Rekursion, und damit ergibt ist  $w \in BL \subseteq L$ .

In beiden Fällen gilt  $w \in L$ , damit ist die Aussage  $B^*C \subseteq L$  gezeigt.

- (2) Teilmengenrichtung:  $L \subseteq B^*C$ . Sei  $w \in L$ , zu zeigen ist  $w \in B^*C$ . Beweis durch vollständige Induktion über alle Worte  $w$  mit  $n = |w|$ .

$n = 0$ : Für alle  $w \in L$  mit  $|w| = 0$  gilt  $w = \varepsilon$ . Wenn  $w \notin L$ , so gibt es (ex falso) kein Gegenbeispiel zu Aussage und ist damit wahr. Wenn  $w \in L$ , so ist wegen  $L = BL \cup C$  und  $\varepsilon \notin B$  dann auch  $w \notin BL$ . Also ist  $w \in C$ . Damit ist  $w \in B^*C$ .

$n - 1 \rightarrow n$ : Sei die Aussage  $w \in B^*C$  für alle  $w \in L$  mit  $|w| < n$  erfüllt. Zu zeigen ist, dass dann  $w \in B^*C$ . Da  $w \in L = BL \cup C$  ist entweder  $w \in C$ , und dann ist automatisch  $w \in B^*C$ , da  $C \subseteq B^*C$ , oder es gibt eine Zerlegung  $w = bw_k$  mit  $b \in B$  und  $b \neq \varepsilon$ , da  $\varepsilon \notin B$ , und  $w_k \in L$  mit  $|w_k| = k = n - |b| < n$ . Laut Induktionsvoraussetzung folgt dann aber aus  $w_k \in L$ , dass  $w_k \in B^*C$  ist, und somit  $bw_k \in BB^*C \subseteq B^*C$  ist.

Nach Induktion gilt damit  $w \in B^*C$ , damit ist die Aussage  $L \subseteq B^*C$  gezeigt.

Aus beiden Beweisrichtungen folgt daher, dass aus  $L = BL \cup C$  mit  $\varepsilon \notin B$  direkt  $L = B^*C$  folgt.

□

**SATZ 3.9. (Konstruktion äquivalenter regulärer Ausdrücke aus DEA)** Sei  $M = (Q, \Sigma, \delta, q_s, E)$  ein deterministischer endlicher Automat. Dann existiert ein deterministisch berechenbarer regulärer Ausdruck  $\alpha \in \mathcal{R}(\Sigma)$  mit  $L(\alpha) = L(M)$ .

**BEWEIS.** Die endliche Anzahl von Zuständen  $Q = \{q_1, \dots, q_n\}$  seien so aufgezählt, dass  $q_s = q_1$ , und ebenso das endlich Alphabet beliebig aufgezählt  $\Sigma = \{a_1, \dots, a_m\}$ . Auf Basis der Übergänge  $\delta : Q \times \Sigma \rightarrow Q$  gibt es zu jedem Zustand  $q_k \in Q$  einen zugehörigen ablesbaren regulären Ausdruck  $\alpha_{q_k} = \alpha_{q_k}^{(n+1)}$  der Form

$$\alpha_{q_k} = \alpha_{q_k}^{(n+1)} \equiv a_1 \alpha_{\delta(q_k, a_1)} \mid \dots \mid a_m \alpha_{\delta(q_k, a_m)} \mid \underbrace{\varepsilon}_{\text{wenn } q_k \in E}.$$

Diese Ausdrücke besitzen nun bis zu  $n$  unbekannte reguläre Ausdrücke auf der rechten Seite. Beginnend von  $k = n$  werden nun Ausdrücke  $\alpha_{q_l}^{(k)}$ ,  $l < k$ , erzeugt, die nur noch  $k-1$  unbekannte reguläre Ausdrücke  $\alpha_{q_l}$  mit  $l < k$  auf der rechten Seite besitzt, bis zum Schluss  $\alpha_{q_1}^{(1)}$  den regulären Ausdruck des Automaten darstellt. Beginnend wird das Lemma 3.8 von Arden bei Bedarf auf den Ausdruck  $\alpha_{q_k}^{(k+1)}$  angewendet, um  $\alpha_{q_k}$  zu eliminieren. Das ergibt in  $\alpha_{q_k}^{(k)}$  und dies wird in alle  $\alpha_{q_l}$ ,  $l < k$ , eingesetzt, in denen dann maximal noch Terme  $\alpha_{q_l}$  mit  $l < k$  auftreten können. Dann wird  $k$  um 1 reduziert. Terme der Form  $\alpha = \beta\alpha$  sind unbeschränkt und fallen wegen keiner weiteren Lösung daraus weg.  $\square$

**BEISPIEL 3.10.** Die Reihenfolge der Auflösung und womöglich ersichtliche Vereinfachungen haben einen sehr deutlichen Einfluss auf den resultierenden regulären Ausdruck. Zum Automaten aus dem Beispiel 3.3 wurde durch verschiedene Überlegungen in Beispiel 3.6 der sehr kurze Ausdruck  $\alpha = (a|b)^*a(a|b)$  bestimmt. Das systematische Vorgehen nach Beweis von Satz 3.9 ergibt in der Regel keinen sehr kurzen Ausdruck. In diesem Beispiel wird das Verfahren in der gegebenen Reihenfolge  $\{\alpha_{q_1}, \alpha_{q_2}, \alpha_{q_3}, \alpha_{q_4}\}$  durchgeführt. Aus dem Schaubild oder Tableau können diese nicht aufgelösten regulären Ausdrücke bestimmt werden:

$$\begin{aligned}\alpha_{q_1} &= \alpha_{q_1}^{(5)} \equiv a\alpha_{q_2} \mid b\alpha_{q_1} \\ \alpha_{q_2} &= \alpha_{q_2}^{(5)} \equiv a\alpha_{q_4} \mid b\alpha_{q_3} \\ \alpha_{q_3} &= \alpha_{q_3}^{(5)} \equiv a\alpha_{q_2} \mid b\alpha_{q_1} \mid \varepsilon \\ \alpha_{q_4} &= \alpha_{q_4}^{(5)} \equiv a\alpha_{q_4} \mid b\alpha_{q_3} \mid \varepsilon\end{aligned}$$

Zunächst ist nach Lemma  $\alpha_{q_4}^{(4)} = a^*b\alpha_{q_3} \mid a^*$ . Eingesetzt in die weiteren Terme ergibt sich:

$$\begin{aligned}\alpha_{q_1}^{(4)} &\equiv a\alpha_{q_2} \mid b\alpha_{q_1} \\ \alpha_{q_2}^{(4)} &\equiv aa^*b\alpha_{q_3} \mid b\alpha_{q_3} \mid aa^* \equiv (aa^*b|b)\alpha_{q_3} \mid aa^* \equiv a^*b\alpha_{q_3} \mid aa^* \\ \alpha_{q_3}^{(4)} &\equiv a\alpha_{q_2} \mid b\alpha_{q_1} \mid \varepsilon\end{aligned}$$

Der Ausdruck  $\alpha_{q_3}^{(4)} \equiv a\alpha_{q_2} \mid b\alpha_{q_1} \mid \varepsilon = \alpha_{q_3}^{(3)}$  hat keine Rekursion und es ergibt sich:

$$\begin{aligned}\alpha_{q_1}^{(3)} &\equiv a\alpha_{q_2} \mid b\alpha_{q_1} \\ \alpha_{q_2}^{(3)} &\equiv a^*ba\alpha_{q_2} \mid a^*bb\alpha_{q_1} \mid a^*b \mid aa^*\end{aligned}$$

Mit dem Lemma ergibt sich  $\alpha_{q_2}^{(2)} \equiv (a^*ba)^*(a^*bb\alpha_{q_1} \mid a^*b \mid aa^*)$  und damit:

$$\alpha_{q_1}^{(2)} \equiv a(a^*ba)^*(a^*bb\alpha_{q_1} \mid a^*b \mid aa^*) \mid b\alpha_{q_1} \equiv (a(a^*ba)^*a^*bb|b)\alpha_{q_1} \mid a(a^*ba)^*a^*b \mid a(a^*ba)^*aa^*$$

Mit einer letzten Anwendung des Lemmas ergibt sich das Resultat:

$$\alpha_{q_1}^{(1)} = (a(a^*ba)^*a^*bb|b)^*(a(a^*ba)^*a^*b \mid a(a^*ba)^*aa^*)$$

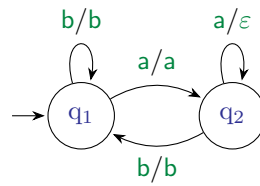


ABBILDUNG 3.1.4. Ein Mealy-Automat zu Beispiel 3.13, der direkt wiederholte **a** löscht

In Anwendungen werden deterministische Automaten weniger nur passiv als **Akzeptor** zur Erkennung von Sprachen sondern auch aktiv für Steuerungen mit Ausgaben eingesetzt. Beispielsweise soll ein Getränkeautomat je nach Einwurf verschiedener Münzen und Auswahl entweder nach weiteren Münzen verlangen, die gewünschte Ware ausgeben, oder das bisher eingeworfene Geld zurückgeben.

DEFINITION 3.11. Ein **Transduktor** ist ein Automat, der in jedem Schritt ein oder mehrere Zeichen eines Ausgabealphabetes ausgibt.

Speziell werden für deterministische endliche Automaten zwei Grundtypen verwendet. Der Mealy-Automat bestimmt die Ausgabe aus Zustand und Eingabezeichen, der Moore-Automat bestimmt die Ausgabe nur aus dem aktuellen Zustand.

DEFINITION 3.12. Ein **Mealy-Automat** ist ein Tupel

$$M = (Q, \Sigma, \delta, q_s, \Gamma, \omega),$$

wo  $Q$  die endliche Menge von Zuständen,  $\Sigma$  das Alphabet möglicher Eingaben,  $\delta : Q \times \Sigma \rightarrow Q$  die Übergangsfunktion,  $q_s$  der Startzustand,  $\Gamma$  das Alphabet möglicher Ausgaben und  $\omega : Q \times \Sigma \rightarrow \Gamma^*$  die Ausgabefunktion in Abhängigkeit von Zustand und Eingabe. Soll die Transduktion enden können, so ist eine entsprechende Definition mit Akzeptanzzuständen möglich.

BEISPIEL 3.13. Für eine Beschreibung eines Mealy-Automaten bietet sich sowohl eine tabellarische Darstellung der Übergangs- und Ausgabefunktionen an, als auch die Darstellung durch ein Zustandsdiagramm. Soll beispielsweise ein Transduktor alle direkt wiederholten **a** aus Wörtern in  $\{a, b\}^*$  streichen, so könnte dies mit einem Mealy-Automaten  $M = (Q, \Sigma, \delta, q_s, \Gamma, \omega)$  mit  $Q = \{q_1, q_2\}$ ,  $\Sigma = \Gamma = \{a, b\}$ ,  $q_s = q_1$  und diesen Übergangs- und Ausgabefunktionen realisiert werden.

| $\delta / \omega$ | $q_1$   | $q_2$             |
|-------------------|---------|-------------------|
| <b>a</b>          | $q_2/a$ | $q_2/\varepsilon$ |
| <b>b</b>          | $q_1/b$ | $q_1/b$           |

Die gemeinsame Darstellung von  $\delta$  und  $\omega$  in einem Schaubild bietet sich an, da die beiden erforderlichen Tabellen die identische Struktur haben. Die Abbildung 3.1.4 zeigt das entsprechende Zustandsdiagramm des Mealy-Automaten.

DEFINITION 3.14. Ein **Moore-Automat** ist ein Tupel

$$M = (Q, \Sigma, \delta, q_s, \Gamma, \omega),$$

wo  $Q$  die endliche Menge von Zuständen,  $\Sigma$  das Alphabet möglicher Eingaben,  $\delta : Q \times \Sigma \rightarrow Q$  die Übergangsfunktion,  $q_s$  der Startzustand,  $\Gamma$  das Alphabet möglicher Ausgaben und  $\omega : Q \rightarrow \Gamma^*$  die Ausgabefunktion in Abhängigkeit von Zustand. Soll die Transduktion enden können, so ist eine entsprechende Definition mit Akzeptanzzuständen möglich.

BEISPIEL 3.15. Für eine Beschreibung eines Moore-Automaten bietet sich ebenso eine tabellarische Darstellung der Übergangs- und Ausgabefunktionen an, als auch die Darstellung durch ein Zustandsdiagramm. Soll beispielsweise ein Transduktor alle direkt wiederholten **a** aus Wörtern in  $\{a, b\}^*$  wie

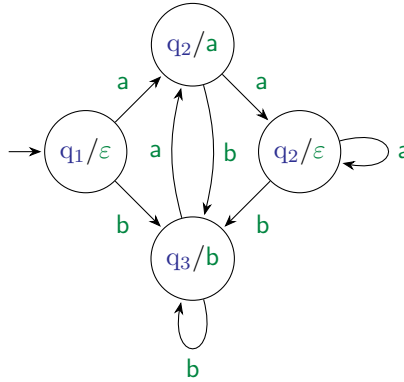


ABBILDUNG 3.1.5. Ein Moore-Automat zu Beispiel 3.15, der direkt wiederholte **a** löscht

in Beispiel 3.13 streichen, so könnte dies mit einem Moore-Automaten  $M = (Q, \Sigma, \delta, q_s, \Gamma, \omega)$  mit  $Q = \{q_1, q_2, q_3, q_4\}$ ,  $\Sigma = \Gamma = \{a, b\}$ ,  $q_s = q_1$  und diesen Übergangs- und Ausgabefunktionen realisiert werden.

| $\delta / \omega$ | $q_1/\varepsilon$ | $q_2/a$ | $q_3/b$ | $q_4/\varepsilon$ |
|-------------------|-------------------|---------|---------|-------------------|
| <b>a</b>          | $q_2$             | $q_4$   | $q_2$   | $q_4$             |
| <b>b</b>          | $q_3$             | $q_3$   | $q_3$   | $q_3$             |

Die gemeinsame Darstellung von  $\delta$  und  $\omega$  in einem Schaubild bietet sich an, da die beiden erforderlichen Tabellen die identische Struktur haben. Die Abbildung 3.1.5 zeigt das entsprechende Zustandsdiagramm des Mealy-Automaten.

**SATZ 3.16.** *Mealy- und Moore-Automaten sind gleich mächtig: Jeder Moore-Automat ist direkt als Mealy-Automat darstellbar und für jeden Mealy-Automaten gibt es einen Moore-Automaten, der auf gegebene Eingaben identisch zum Mealy-Automaten die Ausgaben produziert.*

**BEWEIS.** Liegt ein Moore-Automat vor, so ist die Ausgabefunktion  $\varepsilon : Q \rightarrow \Gamma^*$  direkt als  $\tilde{\omega} : Q \times \Sigma \rightarrow \Gamma^*$  interpretierbar, in dem  $\tilde{\omega}(q, a) = \omega(q)$  unabhängig von  $a \in \Sigma$  gesetzt wird. Liegt ein Mealy-Automat  $(Q, \Sigma, \delta, q_s, \Gamma, \omega)$  vor, so wird der Moore-Automat als  $(\tilde{Q}, \Sigma, \tilde{\delta}, \tilde{q}_s, \Gamma, \tilde{\omega})$  definiert mit

$$\begin{aligned}\tilde{Q} &= (Q \times \Gamma^*) \cup (s_0, \varepsilon), \\ \tilde{\delta}((q, x), a) &= \begin{cases} (\delta(q_s), \omega(q_s, a)), & \text{wenn } (q, x) = (s_0, \varepsilon), \\ (\delta(q), \omega(q, a)), & \text{sonst,} \end{cases} \\ \tilde{q}_s &= (s_0, \varepsilon), \\ \tilde{\omega}((q, x)) &= x.\end{aligned}$$

□

**BEISPIEL 3.17.** Da die beiden vorgestellten Transduktoren-Typen äquivalent sind, sollte je nach Anwendung die passende Beschreibung gewählt werden. Beispielsweise können mit Transduktoren Strategien und Agenten sehr exakt und effizient beschrieben werden: Auf Grund der beschränkten Anzahl von möglichen Zuständen, können Auswirkungen von Strategien deutlich einfacher als bei allgemeinen Programmen dokumentiert und auch bewiesen werden und dazu ist die Ausführung in jedem Schritt in algorithmisch meistens sehr erstreblichem konstantem Aufwand, also  $\mathcal{O}(1)$ -Zeit und Speicher, möglich. Gleichzeitig kann die begrenzte Anzahl als elementar limitierender Faktor und auch die sehr kleinteilige Beschreibung mit Einzelzeichen für die Übergänge gegen die Nutzung dieses Verfahrens sprechen. Soll beispielsweise eine Strategie für das Spiel Schere-Stein-Papier oder „Rock-Paper-Scissors“ (im englischen sind die drei Züge mit den Anfangsbuchstaben **R**, **P**, **S** eindeutig) beschrieben werden, so ist dies

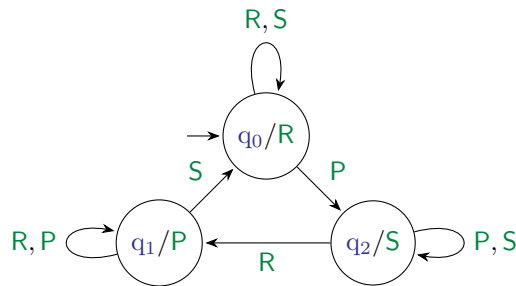


ABBILDUNG 3.1.6. Zustandsdiagramm des Moore-Automaten zur Rock-Paper-Scissors Strategie aus Beispiel 3.17

mit einem Moore-Automaten  $M$  möglich:

|                                    |                   |         |         |         |
|------------------------------------|-------------------|---------|---------|---------|
| $M = (Q, \Sigma, \delta, q_s, E),$ | $\delta / \omega$ | $q_0/R$ | $q_1/P$ | $q_2/S$ |
| $Q = \{q_0, q_1, q_2\},$           | $R$               | $q_0$   | $q_1$   | $q_1$   |
| $\Sigma = \{R, P, S\},$            | $P$               | $q_2$   | $q_1$   | $q_2$   |
| $q_s = q_0,$                       | $S$               | $q_0$   | $q_0$   | $q_2$   |

Diese Strategie, dessen Zustandsdiagramm in Abbildung dargestellt ist, beginnt immer mit der Wahl  $R$  für den ersten Zug und wechselt dann grundsätzlich nur den Zustand, wenn die Wahl vom Gegner zum Verlust führt. Im ersten Zustand  $q_0$  wird nur dann auf  $q_2$  mit Wahl  $S$  gewechselt, wenn vom Gegner  $P$  angezeigt wurde. In der Simulation für die Übung wird dieser Automat mit diesem Vektor aus Startzustand, Übergangsvektor, Ausgabevektor und Bezeichnung kodiert, wobei die Symbole  $R, P, S$  in den Arrays mit 0, 1, 2 identifiziert werden:

$[ \underbrace{0}_{q_s=q_0}, [ \underbrace{[0, 2, 0]}_{(\delta(q_0, R), \delta(q_0, P), \delta(q_0, S))}, [ \underbrace{[1, 1, 0]}_{(\delta(q_1, R), \delta(q_1, P), \delta(q_1, S))}, [ \underbrace{[1, 2, 2]}_{(\delta(q_2, R), \delta(q_2, P), \delta(q_2, S))} ], [ \underbrace{0}_{\omega(q_0)}, \underbrace{1}_{\omega(q_1)}, \underbrace{2}_{\omega(q_2)} ], "switcher" ]$   
Bezeichnung

Als Verallgemeinerung deterministischer akzeptierender Automaten können nicht-deterministische Automaten kürzer und ausdrucksstärker sein. Die Konzepte der nicht-deterministischen Automaten können dabei auch direkt auf Transduktoren übertragen werden.

->

**DEFINITION 3.18.** Ein **nichtdeterministischer endlicher Automat (NEA)** ist ein Tupel

$$M = (Q, \Sigma, \delta, q_s, E),$$

wo  $Q$  die endliche Menge von Zuständen,  $\Sigma$  das Alphabet möglicher Eingaben,  $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$  die Übergangsrelation,  $q_s$  der Startzustand und  $E \subseteq Q$  die Menge der Endzustände beschreibt.

Die Verallgemeinerung liegt also darin, dass statt einer deterministischen Übergangsfunktion jetzt mit einer Relation gleichzeitig mehrere neue Zustände angegeben werden können. Da die Menge der neuen Zustände auch leer sein kann, wird der Automat nichtdeterministisch genannt, da im Verlauf eines Pfades noch nicht klar ist, ob daraus am Ende eine Zustandsmenge resultiert. Da jede Übergangsfunktion  $\delta : Q \times \Sigma \rightarrow \Sigma$  auch eine Übergangsrelation  $\tilde{\delta} : Q \times \Sigma \rightarrow \mathcal{P}(\Sigma)$  mit  $\tilde{\delta}(q, w) = \{\delta(q, w)\}$  definiert, ist jeder deterministische endliche Automat auch ein nichtdeterministischer endlicher Automat, bei dem aber dann jeder Zustand bei Akzeptanz nach Symbolen genau determiniert ist.

Eine Darstellung eines nichtdeterministischen endlichen Automaten mit einem **Zustandsdiagramm** überträgt sich direkt, wobei von einem Knoten mehrere Kanten mit gleichem Symbol zu unterschiedlichen Knoten geben kann, oder entsprechend Kanten mit identischem Ursprung zu unterschiedlichen Zielen, deren Bezeichnungen einen echten Schnitt besitzen.

Entsprechend zur iterierten Übergangsfunktion definiert sich eine iterierte Übergangsrelation:



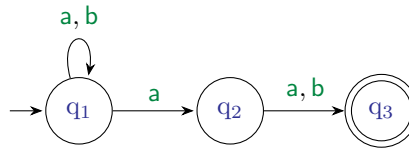


ABBILDUNG 3.1.7. Zustandsdiagramm des NEA aus Beispiel 3.21

-> **DEFINITION 3.19.** Die Funktion  $\delta^* : \mathcal{P}(Q) \times \Sigma^* \rightarrow \mathcal{P}(Q)$  sei dann **die iterierte Übergangsrelation** von  $\delta$ , die induktiv definiert ist:

$$\delta^*(R, w) = \begin{cases} R & , \text{ wenn } w = \varepsilon, \\ \bigcup_{q \in R} \delta^*(\delta(q, x), y) & , \text{ wenn } w = xy \text{ für } x \in \Sigma, y \in \Sigma^*. \end{cases}$$

Außerdem sei  $\delta^*(q, w) := \delta^*({q}, w)$  für ein  $q \in Q$  gemeint.

Daraus folgt die entsprechende Definition für die Sprache eines nichtdeterministischen Automaten:

-> **DEFINITION 3.20.** Die **Sprache des nichtdeterministischen endlichen Automaten**  $M$  mit  $M = (Q, \Sigma, \delta, q_s, E)$  lautet:  $L(M) = \{w \in \Sigma^* \mid \delta^*(q_s, w) \cap E \neq \emptyset\}$ .

BEISPIEL 3.21. Die Sprache  $L(M)$  des deterministischen endlichen Automaten  $M$  aus Beispiel 3.3 kann auch mit einem kleineren nichtdeterministischen Automaten  $N = (Q, \Sigma, \delta, q_s, E)$  mit  $Q = \{q_1, q_2, q_3\}$ ,  $\Sigma = \{a, b\}$  und

$$\begin{aligned} \delta(q_1, a) &= \{q_1, q_2\}, & \delta(q_2, a) &= \{q_3\}, & \delta(q_3, a) &= \emptyset, \\ \delta(q_1, b) &= \{q_1\}, & \delta(q_2, b) &= \{q_3\}, & \delta(q_3, b) &= \emptyset, \end{aligned}$$

oder tabellarisch

| $\delta$ | $q_1$          | $q_2$     | $q_3$       |
|----------|----------------|-----------|-------------|
| a        | $\{q_1, q_2\}$ | $\{q_3\}$ | $\emptyset$ |
| b        | $\{q_1\}$      | $\{q_3\}$ | $\emptyset$ |

mit  $q_s = q_1$  und  $E = \{q_3\}$  erzeugt werden, also  $L(M) = L(N)$ , wie gleich zu sehen ist. Beispielsweise wird das Wort  $w = baab$  durch  $\delta^*$  so abgebildet

$$\delta^*(q_1, baab) = \{q_1, q_3\}, \text{ da } \{q_1\} \xrightarrow{b} \{q_1\} \xrightarrow{a} \{q_1, q_2\} \xrightarrow{a} \{q_1, q_2, q_3\} \xrightarrow{b} \{q_1, q_3\}$$

und wegen  $\delta^*(q_1, baab) \cap E = \{q_3\} \neq \emptyset$  ist damit  $baab \in L(N)$ . Das Zustandsdiagramm von  $N$  ist in Abbildung 3.1.7 dargestellt. Mit dieser Darstellung ist unmittelbar ersichtlich, dass  $L(N) = \{xaz \mid x \in \Sigma^*, z \in \Sigma\}$  ist, und damit gilt tatsächlich  $L(N) = L(M)$ , wie behauptet. Im Zustandsdiagramm werden Kanten mit vielen Symbolen oft auch mit Mengen abgekürzt, so könnte hier  $a, b$  durch  $\Sigma$  ersetzt werden.

Jeder deterministische endliche Automat ist unmittelbar auch ein nichtdeterministischer endlicher Automat, so wie jede Funktion auch eine Relation ist. Umgekehrt ist ein nichtdeterministischer Automat kein deterministischer Automat sobald für nur eine Kombination von Zustand und Symbol nicht genau ein neuer Zustand entsteht. In diesen Fällen ist es möglich, einen deterministischen Automaten zu konstruieren, der die gleiche Sprache erzeugt:

-> **SATZ 3.22.** Für jeden nichtdeterministischen endlichen Automaten gibt es einen deterministischen endlichen Automaten, der die gleiche Sprache erzeugt.

BEWEIS. Sei  $N = (Q, \Sigma, \delta, q_s, E)$  ein nichtdeterministischer Automat. Sei  $\tilde{Q} = \mathcal{P}(Q)$  und  $\tilde{E} = \{q \in \tilde{Q} \mid q \cap E \neq \emptyset\}$ . Dann ist  $M = (\tilde{Q}, \Sigma, \tilde{\delta}, \{q_s\}, \tilde{E})$  mit  $\tilde{\delta}(q, w) = \delta^*(q, w)$  ein deterministischer endlicher Automat, der den Automaten  $N$  simuliert und daher  $L(M) = L(N)$  gilt:

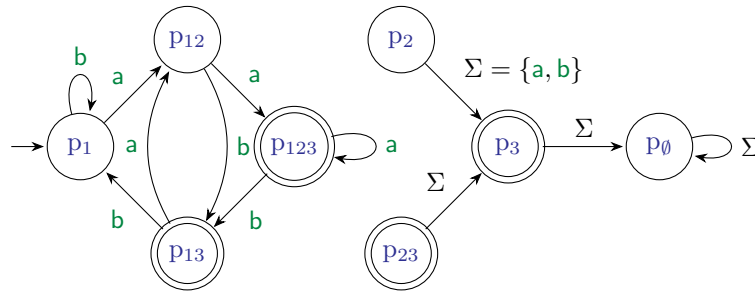


ABBILDUNG 3.1.8. Zustandsdiagramm des simulierenden DEA aus Beispiel 3.23

- (1) Sei  $w \in L(N)$ . Dann ist  $\delta^*(q_s, w) \cap E \neq \emptyset$ . Dann ist  $\tilde{q} = \delta^*(q_s, w) = \tilde{\delta}^*(q_s, w)$ . Da  $\tilde{q} \cap E \neq \emptyset$  ist  $\tilde{q} \in \tilde{E}$  nach Definition und damit  $w \in L(M)$ .
- (2) Sei  $w \in L(M)$ . Dann ist  $\tilde{q} = \tilde{\delta}^*(q_s, w) \in \tilde{E}$  und damit  $\tilde{q} \cap E \neq \emptyset$ . Da nun  $\delta^*(q_s, w) = \tilde{\delta}^*(q_s, w) = \tilde{q}$  ist  $\delta^*(q_s, w) \cap E \neq \emptyset$  und damit  $w \in L(N)$ .

□

### Potenzmengenkonstruktion!

BEISPIEL 3.23. Der simulierende deterministische endliche Automat  $M = (\tilde{Q}, \Sigma, \tilde{\delta}, \tilde{q}_s, \tilde{E})$  für den nichtdeterministischen endlichen Automaten  $N = (Q, \Sigma, \delta, q_s, E)$  aus Beispiel 3.21 hat die Zustände  $\mathcal{P}(Q) = \{\emptyset, \{q_1\}, \{q_2\}, \{q_3\}, \{q_1, q_2\}, \{q_1, q_3\}, \{q_2, q_3\}, Q\}$ . Zur verkürzten Schreibweise werden diese Zustände jetzt  $\tilde{Q} = \{p_0, p_1, p_2, p_3, p_{12}, p_{13}, p_{23}, p_{123}\}$  geschrieben. Dann ist  $\tilde{\delta}$  tabellarisch definiert durch

| $\tilde{\delta}$ | $p_0$ | $p_1$    | $p_2$ | $p_3$ | $p_{12}$  | $p_{13}$ | $p_{23}$ | $p_{123}$ |
|------------------|-------|----------|-------|-------|-----------|----------|----------|-----------|
| a                | $p_0$ | $p_{12}$ | $p_3$ | $p_0$ | $p_{123}$ | $p_{12}$ | $p_3$    | $p_{123}$ |
| b                | $p_0$ | $p_1$    | $p_3$ | $p_0$ | $p_{13}$  | $p_1$    | $p_3$    | $p_{13}$  |

mit  $\tilde{q}_s = p_1$  und  $\tilde{E} = \{p_3, p_{13}, p_{23}, p_{123}\}$ . Das Zustandsdiagramm von  $M$  ist in Abbildung 3.1.8 dargestellt. Die Konstruktion hat hier mit anderen Bezeichnungen sogar die vollständige Struktur des ursprünglichen deterministischen endlichen Automaten aus Beispiel 3.3 rekonstruiert zuzüglich von nicht erreichbaren Zuständen  $\{p_2, p_3, p_{23}, p_0\}$ .

Es ist zu erwarten, dass so ein simulierender deterministischer endlicher Automat keineswegs optimal ist, sondern sogar sehr wahrscheinlich Zustände enthält, die nicht erreichbar sind, da durch die Bildung der Potenzmenge alle möglichen Zustandsmengen erzeugt werden, auch wenn sie vom simulierten nichtdeterministischen endlichen Automaten nie auftreten können.

In vielen Anwendungen ist eine Beschreibung mit optionalen Eingaben sinnvoll:

BEISPIEL 3.24. Wenn Dezimalzahlen mit Dezimalpunkt der Form  $+4.$ ,  $-.234$  oder  $3.1415$  optional ein Vorzeichen haben und entweder  $r$  vor dem Punkt oder nach dem Punkt mindestens eine Ziffer besitzen müssen, so könnte ein erkennender Automat die Form wie in Abbildung 3.1.9 besitzen. Dieser Automat ist aber genau genommen kein NEA, da dort keine  $\varepsilon$ -Übergänge erlaubt sind. Das motiviert die Definition von  $\varepsilon$ -NEA.

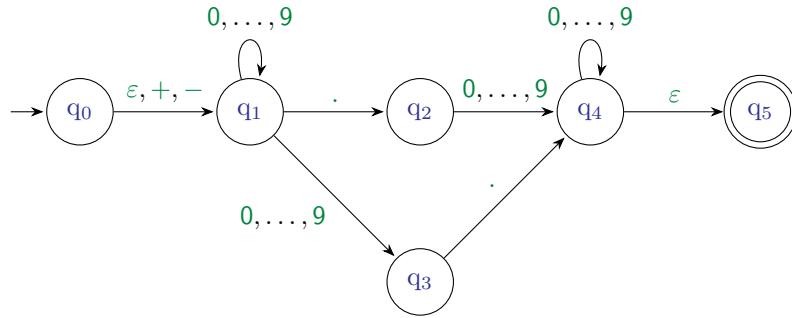
**DEFINITION 3.25.** Ein **nicht-deterministischer endlicher Epsilon-Automat ( $\varepsilon$ -NEA)** ist ein Tupel

$$M = (Q, \Sigma, \delta_\varepsilon, q_s, E),$$

→

wo  $Q$  die endliche Menge von Zuständen,  $\Sigma$  das Alphabet möglicher Eingaben,  $\delta_\varepsilon : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q)$  die Übergangsrelation,  $q_s$  der Startzustand und  $E \subseteq Q$  die Menge der Endzustände beschreibt.

Entsprechend zur iterierten Übergangsrelation von NEA definiert sich eine iterierte Übergangsrelation für  $\varepsilon$ -NEA mit Unterstützung der  $\varepsilon$ -Hülle, die die  $\varepsilon$ -Übergänge berücksichtigt:

ABBILDUNG 3.1.9. Ein  $\varepsilon$ -NEA für Dezimalzahlen wie  $+4.$ ,  $-.234$  oder  $3.1415$ 

DEFINITION 3.26. Für einen  $\varepsilon$ -NEA  $M = (Q, \Sigma, \delta_\varepsilon, q_s, E)$  ist die  $\varepsilon$ -**Hülle**  $H_\varepsilon : \mathcal{P}(Q) \rightarrow \mathcal{P}(Q)$  einer Menge von Knoten  $Q$  rekursiv definiert durch die  $\varepsilon$ -Verbindungen der Knoten zuzüglich aller Knoten in den  $\varepsilon$ -Hüllen der  $\varepsilon$ -Verbindungen:

$$H_\varepsilon(P) = \bigcup_{p \in P} \{q \in Q \mid q = p \vee q \in \delta_\varepsilon(p, \varepsilon) \vee (q \in H_\varepsilon(\{r\}) \wedge r \in \delta_\varepsilon(p, \varepsilon))\}$$

Die Funktion  $\delta_\varepsilon^* : \mathcal{P}(Q) \times \Sigma^* \rightarrow \mathcal{P}(Q)$  sei dann die **iterierte  $\varepsilon$ -Übergangsrelation** von  $\delta$ , die induktiv definiert ist:

$$\delta_\varepsilon^*(R, w) = \begin{cases} H_\varepsilon(R) & , \text{ wenn } w = \varepsilon, \\ \bigcup_{q \in H_\varepsilon(R)} \delta_\varepsilon^*(H_\varepsilon(\delta_\varepsilon(q, x)), y) & , \text{ wenn } w = xy \text{ für } x \in \Sigma, y \in \Sigma^*. \end{cases}$$

Außerdem sei  $\delta_\varepsilon^*(q, w) := \delta_\varepsilon^*({q}, w)$  für ein  $q \in Q$  gemeint.

Daraus folgt die entsprechende Definition für die Sprache eines nichtdeterministischen  $\varepsilon$ -Automaten:

DEFINITION 3.27. Die **Sprache des nichtdeterministischen endlichen Epsilon-Automaten**  $M$  mit  $M = (Q, \Sigma, \delta_\varepsilon, q_s, E)$  lautet:  $L(M) = \{w \in \Sigma^* \mid \delta_\varepsilon^*(q_s, w) \cap E \neq \emptyset\}$ .

Offensichtlich ist jeder DEA, ein DEA ergänzt mit  $\varepsilon$ -Übergängen und jeder NEA auch ein  $\varepsilon$ -NEA, und damit kann das Modell als allgemeinste Darstellung eines endlichen Automaten gesehen werden. Er ist aber trotzdem nicht mächtiger als ein deterministischer endlicher Automat:

SATZ 3.28. Für jeden nichtdeterministischen endlichen Automaten gibt es einen deterministischen endlichen Automaten, der die gleiche Sprache erzeugt.

BEWEIS. Die Konstruktion zum Beweis von Satz 3.22 ist hier direkt anwendbar, nur mit der Verwendung der iterierten  $\varepsilon$ -Übergangsrelation  $\delta_\varepsilon^*$  des  $\varepsilon$ -NEA statt der iterierten Übergangsrelation  $\delta^*$  eines NEA.  $\square$

BEISPIEL 3.29. Für den  $\varepsilon$ -NEA aus Beispiel 3.24 können diese  $\varepsilon$ -Hüllen bestimmt werden:

| $q$                | $q_0$          | $q_1$     | $q_2$     | $q_3$     | $q_4$          | $q_5$     |
|--------------------|----------------|-----------|-----------|-----------|----------------|-----------|
| $H_\varepsilon(q)$ | $\{q_0, q_1\}$ | $\{q_1\}$ | $\{q_2\}$ | $\{q_3\}$ | $\{q_4, q_5\}$ | $\{q_5\}$ |

Daraus kann dann dieser simulierende DEA ohne nicht erreichbare Zustände bestimmt werden:

| $\tilde{\delta}_\varepsilon$ | $p_\emptyset$ | $p_0$    | $p_1$         | $p_2$         | $p_{13}$      | $p_{45}$      | $p_{245}$     |
|------------------------------|---------------|----------|---------------|---------------|---------------|---------------|---------------|
| $+$                          | $p_\emptyset$ | $p_1$    | $p_\emptyset$ | $p_\emptyset$ | $p_\emptyset$ | $p_\emptyset$ | $p_\emptyset$ |
| $-$                          | $p_\emptyset$ | $p_1$    | $p_\emptyset$ | $p_\emptyset$ | $p_\emptyset$ | $p_\emptyset$ | $p_\emptyset$ |
| $0, \dots, 9$                | $p_\emptyset$ | $p_{13}$ | $p_{13}$      | $p_{45}$      | $p_{13}$      | $p_{45}$      | $p_{45}$      |
| $.$                          | $p_\emptyset$ | $p_2$    | $p_2$         | $p_\emptyset$ | $p_{245}$     | $p_\emptyset$ | $p_\emptyset$ |

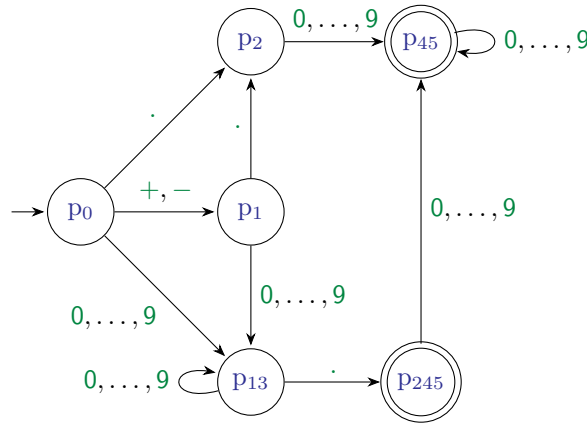


ABBILDUNG 3.1.10. Partielles Zustandsdiagramm (d.h. ohne  $p_0$ ) eines DEA, der den  $\varepsilon$ -NEA aus Abbildung 3.1.9 simuliert.

Das **partielle Zustandsdiagramm**, also das Zustandsdiagramm, wo alle fehlenden Übergänge zu einem nicht terminierenden Zustand wie  $p_0$  ausgeblendet wurden, zu diesem simulierenden DEA ist in Abbildung 3.1.10 dargestellt.

Die in der Praxis durchaus relevante Erweiterung von endlichen Automaten mit  $\varepsilon$ -Übergängen kompliziert durch die  $\varepsilon$ -Hüllen leider die Darstellung ohne dabei theoretisch auf neue Konzepte zu führen. Daher werden diese im Folgenden nicht immer explizit zusätzlich aufgeführt.

Damit konnten wir soweit nachweisen, dass die Darstellungen von Geräten durch deterministische und nichtdeterministische endliche Automaten zwar unterschiedlich ausdrucksstark sind, jedoch bezogen auf die erzeugten Sprachen genau äquivalent sind. Die Beobachtung in Beispiel 3.6, dass die Sprache aus dem Beispiel 3.3 gerade durch eine reguläre Grammatik und damit einen regulären Ausdruck beschrieben werden kann, ist sogar allgemein gültig:

**SATZ 3.30.** Für jeden deterministischen endlichen Automaten  $M = (Q, \Sigma, \delta, q_s, E)$  gibt es eine reguläre Grammatik  $G = (V, \Sigma, R, S)$ , die die gleiche Sprache erzeugt, also  $L(M) = L(G)$  gilt.

**BEWEIS.** Eine zu einer regulären Grammatik äquivalente Grammatik, die die Sprache des endlichen Automaten erzeugt, kann direkt konstruiert werden. Sei  $Q = \{q_1, \dots, q_n\}$ , dann gibt es für jeden Zustand  $q_i$  des Automaten eine entsprechende Variable  $X_i$ :  $V = \{X_1, \dots, X_n\}$ . Aus dem Startzustand  $q_s = q_1$  ergibt sich dann die Startvariable  $S = X_1$ . Sei  $\Sigma = \{a_1, \dots, a_m\}$  das Alphabet. Dann werden für alle Zustände  $q_i$  und alle Symbole  $r_k$  mit  $\delta(q_i, r_k) = q_l$  die Regeln  $r_{i,k} : X_i \mapsto a_k X_l$  sowie für alle akzeptablen Zustände  $q_i \in E$  die Regeln  $r_{i,0} : X_i \mapsto \varepsilon$  hinzugefügt.

- (1) Sei nun  $w \in L(M)$ . Zu zeigen ist, dass dann auch  $w \in L(G)$ : Wenn  $w = (w_1, \dots, w_n) \in L(M)$ , so ist  $\delta^*(q_s, w) \in E$  und es gibt eine Folge von Zuständen  $q_{b_i}$  mit  $b_0 = s$  und  $q_{b_n} \in E$ , für die  $\delta(q_{b_{i-1}}, w_i) = q_{b_i}$  für  $i = 1, \dots, n$  ist. Beginnend vom Startzustand  $q_s$  und Startsymbol  $S$  ist dann induktiv jedes  $w_i = a_k$  für ein  $k$  und der Zustand der Maschine  $q_{b_{i-1}}$ . Gleichzeitig liegt in der Grammatik das Wort  $w_1 w_2 \dots w_{i-1} X_{a_{i-1}}$  vor, und die Regel  $r_{i,k}$  erzeugt sowohl genau das  $a_k = w_i$ , das in diesem Schritt von der Maschine akzeptiert wird im Übergang zu  $\delta(q_{b_{i-1}}, w_i) = q_{b_i}$ , als auch die Variable  $X_{b_i}$ . Dies läuft bis zum akzeptierten Zustand  $q_{b_n}$ , für den es als akzeptierten Zustand eine Regel  $r_{b_n,0}$  gibt, mit dem die verbleibende Variable  $X_{b_n}$  eliminiert wird, und genau das Wort  $w$  wurde erzeugt, also ist  $w \in L(G)$ .

- (2) Sei nun  $w \in L(G)$ . Zu zeigen ist, dass dann auch  $w \in L(M)$ : Wenn  $w = (w_1, \dots, w_n) \in L(G)$ , so gibt es nach Definition der Grammatik eine Folge von Regeln  $r_{b_i, c_i}$ , wobei im jeden Schritt  $w_i = a_{c_i}$  ist. Dabei ist  $X_{b_1}$  das Startsymbol, und daher  $q_{b_1}$  auch ein Startzustand nach Definition der Grammatik. Da die Regeln  $r_{b_i, c_i}$  in der Anwendungsreihenfolge nach Definition gerade den Übergängen des Automaten entsprechen, durchläuft der Automat mit Akzeptanz von  $a_{c_i}$  die Zustände  $q_{c_i}$  für  $i = 1, \dots, n$ . Und endet in einem akzeptierten Zustand, weil die Grammatik nur in entsprechenden Variablen enden kann. Damit ist  $w \in L(M)$ .

□

BEISPIEL 3.31. Die nach Satz 3.30 konstruierte Grammatik  $G = (V, \Sigma, R, S)$ , die die gleiche Sprache wie  $M = (Q, \Sigma, \delta, q_s, E)$  aus Beispiel 3.3 erzeugt, lautet:

$$\begin{aligned} V &= \{X_1, X_2, X_3, X_4\}, \\ T &= \{a, b\}, \\ R &= \{r_1, r_2, r_3, r_4\}, \\ &\quad r_1 : X_1 \mapsto aX_2 \mid bX_1, \\ &\quad r_2 : X_2 \mapsto aX_4 \mid bX_3, \\ &\quad r_3 : X_3 \mapsto aX_2 \mid bX_1 \mid \varepsilon, \\ &\quad r_4 : X_4 \mapsto aX_4 \mid bX_3 \mid \varepsilon, \\ S &= X_1. \end{aligned}$$

Hier sind die Regeln  $r_{3,0}$  in  $r_{3,3}$  und  $r_{4,0}$  in  $r_{4,3}$  zu finden. Diese Grammatik ist wegen der Regeln  $r_{3,0}$  und  $r_{4,3}$  genau genommen keine Typ-3 Grammatik, da dort nur ein Produkt  $\varepsilon$  in der Form  $S \mapsto \varepsilon$  auftreten darf.

Das kann in einem nachträglichen Schritt korrigiert werden, in dem alle Ziele der betreffenden Variablen mit solchen Regeln, hier  $X_3$  und  $X_4$ , jeweils um Regeln ohne Variable ergänzt werden (oder gegebenenfalls einmalig  $S \rightarrow \varepsilon$  hinzuzufügen), um das Ergebnis zu erzielen:

$$\begin{aligned} V &= \{X_1, X_2, X_3, X_4\}, \\ T &= \{a, b\}, \\ R &= \{r_1, r_2, r_3, r_4\}, \\ &\quad r_1 : X_1 \mapsto aX_2 \mid bX_1, \\ &\quad r_2 : X_2 \mapsto aX_4 \mid bX_3 \mid a \mid b, \\ &\quad r_3 : X_3 \mapsto aX_2 \mid bX_1, \\ &\quad r_4 : X_4 \mapsto aX_4 \mid bX_3 \mid a \mid b, \\ S &= X_1. \end{aligned}$$

SATZ 3.32. Für jede rechts-lineare reguläre oder Typ-3 Grammatik  $G$  gibt es einen nichtdeterministischen endlichen Automaten  $M$ , so dass  $L(G) = L(M)$ .

BEWEIS. Für jede Variable  $A \in V$  in  $G = (V, \Sigma, R, S)$  gibt es einen Zustand  $q_A \in Q$  in  $M = (Q, \Sigma, \delta, q_s, E)$ . Zusätzlich gibt es für jedes Symbol  $x \in \Sigma$  einen akzeptierenden Zustand  $q_x \in Q$  und  $q_x \in E$ . Der Startzustand ist  $q_s = q_S$  und falls es eine Regel  $S \mapsto \varepsilon$  gibt, so ist auch der Startzustand in der Menge der akzeptierten Zustände  $q_s \in E$ . Für jede Regel  $r \in R$  mit  $A \mapsto aB$  ist  $q_B \in \delta(q_A, a)$  und für jede Regel  $A \mapsto a$  ist  $q_a \in \delta(q_A, a)$ . Damit ist jede Regelanwendung in einer Ableitung eines Wortes in  $w \in L(G)$  ein Zustandsübergang in  $M$ , durch das entsprechende Symbol des Wortes. Umgekehrt ist jeder Übergang in  $M$  bei Akzeptieren eines Wortes  $w \in L(M)$  die Anwendung einer Regel in  $G$  und Teil der Ableitung des Wortes  $w$  in  $G$ . Damit gilt  $L(G) = L(M)$ . □

BEISPIEL 3.33. Für die Typ-3 Grammatik aus Beispiel 2.41 zur Sprache  $M_3 = \{0\} \cup \{1\} \times \{0, 1\}^*$  hat die Zustände  $Q = \{q_s, q_A, q_0, q_1\}$ , die Endzustände  $E = \{q_0, q_1\}$  und diese Übergangsrelation:

| $\delta$ | $q_s$          | $q_A$          | $q_0$       | $q_1$       |
|----------|----------------|----------------|-------------|-------------|
| 0        | $\{q_0\}$      | $\{q_0, q_A\}$ | $\emptyset$ | $\emptyset$ |
| 1        | $\{q_1, q_A\}$ | $\{q_1, q_A\}$ | $\emptyset$ | $\emptyset$ |

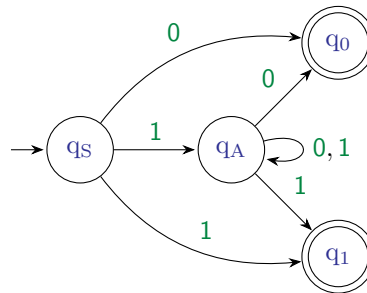


ABBILDUNG 3.1.11. Zustandsdiagramm des NEA aus einer Grammatik aus Beispiel 3.33

Das Zustandsdiagramm von  $M$  ist in Abbildung 3.1.11 dargestellt.

Aus den bisherigen Ergebnissen kann nun zusammenfassend festgestellt werden:

SATZ 3.34. Sei  $\Sigma$  ein Alphabet und  $L \subseteq \Sigma^*$  eine Sprache über  $\Sigma$ . Dann sind äquivalent:

- (1)  $L$  ist regulär.
- (2) Es gibt einen nichtdeterministischen endlichen Automaten  $M$  mit  $L = L(M)$ .
- (3) Es gibt einen deterministischen endlichen Automaten  $M$  mit  $L = L(M)$ .
- (4) Es gibt einen regulären Ausdruck  $\alpha \in \mathcal{R}(\Sigma)$  mit  $L = L(\alpha)$

BEWEIS. Die Aussagen können mit einem Ringschluss gezeigt werden:

- (1)  $\Rightarrow$  (2) Satz 3.32
- (2)  $\Rightarrow$  (3) Satz 3.22
- (3)  $\Rightarrow$  (4) Satz 3.9
- (4)  $\Rightarrow$  (1) Satz 2.40

Zusätzlich gilt direkt (3)  $\Rightarrow$  (1) mit Satz 3.30 und NEA schließen nach Satz 3.28 auch  $\varepsilon$ -NEA ein.  $\square$

Das ist ein besonderes Ergebnis, weil es nicht nur die Umrechnung verschiedener Darstellungen ermöglicht, sondern auch mehrere praktische Methoden beschreibt, wie eine reguläre Sprache erkannt werden kann. Das Ergebnis ist gleichzeitig auch eine Beschreibung dafür, welche Komplexität reguläre Sprachen aus Sicht eines Ausführungsmodells haben, und umgekehrt was das Rechnermodell in der Form von endlichen Automaten genau leisten kann. Diese Äquivalenz drückt weiterhin aus, dass je nach Problemstellung eine der verschiedenen Darstellungen benutzt werden können, um eine Lösbarkeit zu bestimmen oder eine Lösung herzuleiten, und das über sehr verschiedene Domänen der rein formalen Beschreibung über Grammatiken oder regulären Ausdrücken hin zu recht hardware-nahen und umsetzungs-nahen Beschreibungen als endliche Automaten.

Ein Beispiel, wie die Äquivalenz für theoretische Resultate genutzt werden kann, ist der folgende Satz:



**SATZ 3.35. (Pumping-Lemma für reguläre Sprachen)** Zu jeder regulären Sprache  $L$  gibt es eine Zahl  $l$ , so dass sich alle Wörter  $x \in L$  mit  $|x| \geq l$  in  $x = uvw$  zerlegen lassen mit

- (1)  $1 \leq |v| < l$  und
- (2)  $uv^i w \in L$  für alle  $i \geq 0$ .

Falls so eine Zahl  $l$  existiert, so wird die kleinste mögliche Zahl  $l$  die **Pumping-Zahl** von  $L$  genannt.

BEWEIS. Sei  $M = (Q, \Sigma, \delta, q_s, E)$  ein deterministischer endlicher Automat zur Sprache  $L$ . Wähle  $l = |Q| + 1$  höher als die Anzahl der Zustände des Automaten. Da nun  $x \in L$  durch den Automaten  $M$  akzeptiert wird und dabei verschiedene Zustände des Automaten durchläuft sowie  $|x| \geq l$  ist, muss nach spätestens  $l - 1$  Zeichen von  $x$  ein Zustand  $q$  erneut besucht werden. Sei  $0 \leq j$  die Anzahl der Zeichen, bis dieser Zustand  $q$  zum ersten Mal erreicht wird, sowie  $k > j$  die Anzahl der Zeichen, wenn der Zustand  $q$  erneut erreicht wird. Ist  $x = x_1 \dots x_n$ , so sei  $u = x_1 \dots x_j$  oder  $u = \varepsilon$  falls  $j = 0$ ,  $v = x_{j+1}, \dots, x_k$  und  $w = x_{k+1} \dots x_n$  oder  $w = \varepsilon$  falls  $k = n$ . Dann ist  $|v| \geq 1$  sowie  $|v| < l$  und

$$\delta^*(q_s, u) = \delta^*(q_s, uv) = \delta^*(\underbrace{\delta^*(q_s, u)}_{=q}, v) = q.$$

Damit ist  $\delta^*(q_s, uv) = \delta^*(q, v) = \delta^*(\delta^*(q, v), w) = \delta^*(q_s, uvw) \in E$  und auch induktiv für  $i \geq 1$

$$\delta^*(q_s, uv^i w) = \delta^*(q, v^i w) = \delta^*(\delta^*(q, v), v^{i-1} w) = \delta^*(q, v^{i-1} w) = \dots = \delta^*(q, w) \in E.$$

Also ist auch  $uv^i w \in L$  für alle  $i \geq 0$ . Die Pumping-Zahl hängt damit von der Anzahl der Zustände im kleinsten deterministischen Automaten ab, der die Sprache erkennt.  $\square$

Das Pumping-Lemma ist eine erste Möglichkeit, wie ausgeschlossen werden kann, dass eine gegebene Sprache keine reguläre Grammatik besitzt oder gleichbedeutend von keinem endlichen Automaten akzeptiert werden kann.



**SATZ 3.36.** Die Sprache  $M_2 = \{0^n 1^n \mid n \in \mathbb{N}\}$  ist keine Typ-3 Sprache.

BEWEIS. Angenommen  $M_2$  ist reguläre Sprache, so gibt es laut Pumping-Lemma ein  $l \in \mathbb{N}$ , so dass jedes Wort  $x \in L$  mit  $|x| \geq l$  eine Zerlegung  $x = uvw$  mit den Eigenschaften des Pumping-Lemmas gibt. Ist  $l$  gerade so wähle  $n = \frac{l}{2} + 1$  sonst sei  $n = \frac{l+1}{2}$ . Dann ist  $x = 0^n 1^n$  von der Länge  $2n > l$ . Jetzt gibt es drei Möglichkeiten für die Wahl von  $v$  in der Zerlegung  $x = uvw$ . Entweder  $v = 0^m$  für ein  $1 < m \leq n$  oder  $v = 0^{m_0} 1^{m_1}$  mit  $1 \leq m_0, m_1 \leq n$ , oder  $v = 1^m$  für ein  $1 < m \leq n$ :

- (1) Ist  $v = 0^m$ , so ist  $uv^2w = 0^{n+m} 1^n \notin M_2$ , da  $n + m \neq n$ . Also ist das nicht möglich.
- (2) Ist  $v = 0^{m_0} 1^{m_1}$ , so ist  $uv^2w = 0^n 1^{m_1} 0^{m_0} 1^n \notin M_2$ , da alle 0 und 1 in dieser Reihenfolge auftreten müssen. Also kann auch das nicht sein.
- (3) Ist  $v = 1^m$ , so ist  $uv^2w = 0^n 1^{n+m} \notin M_2$ , da  $n \neq n + m$ . Also ist dies ebenso nicht möglich.

Da alle Möglichkeiten für  $v$  auf einen Widerspruch führen, kann die Annahme, dass  $M_2$  eine reguläre Sprache ist, nicht korrekt sein. Sie ist also keine Typ-3 Sprache.  $\square$

Die Pumping-Zahl im Lemma deutet aber auch auf die interessante Fragestellung hin, wie ein gegebener deterministischer endlicher Automat minimiert werden kann, also ob und wie ein äquivalenter endlicher Automat gefunden werden kann, der exakt die gleiche Sprache akzeptiert.

---

**Algorithmus 1** Minimalisierung eines deterministischen endlichen Automaten  $M = (Q, \Sigma, \delta, q_s, E)$

---

- 1: Entferne nicht erreichbare Zustände aus  $Q$
  - 2: Initialisiere  $D' := \{ \{q, e\} \mid q \in Q \setminus E, e \in E \}$
  - 3: **repeat**
  - 4:      $D := D'$
  - 5:      $D' := D' \cup \{ \{q_1, q_2\} \mid \exists x \in \Sigma : \{ \delta(q_1, x), \delta(q_2, x) \} \in D' \}$
  - 6: **until**  $D == D'$
  - 7:  $q_1 \sim q_2 \Leftrightarrow \{q_1, q_2\} \notin D$ ,  $M_{\min} := (Q/\sim, \Sigma, \delta/\sim, [q_s]_{\sim}, E/\sim)$
-

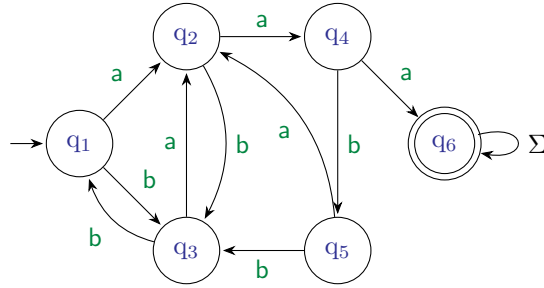


ABBILDUNG 3.1.12. Zustandsdiagramm des zu minimierenden DEA aus Beispiel 3.37

Der Algorithmus basiert auf dem Prinzip, dass Zustände gefunden werden, die zueinander äquivalent sind. Dies wird mit Hilfe einer Äquivalenzrelation ausgedrückt. Eine Äquivalenzrelation  $\sim$  auf einer Menge  $M = \{q_1, \dots, q_n\}$  partitioniert diese Menge in Äquivalenzklassen, der Faktormenge  $M/\sim = \{[q_1]_\sim, \dots\}$ , mit weniger Elementen, wenn mindestens zwei unterschiedliche Elemente zueinander äquivalent sind. Wenn die Äquivalenzrelation kompatibel zu Funktionen  $f : M \rightarrow M, x \mapsto f(x)$  ist, so kann die Funktion auch auf der Faktormenge verwendet werden:  $f/\sim : M/\sim \rightarrow M/\sim, [x]_\sim \mapsto f([x]_\sim)$ . Der Algorithmus basiert darauf, dass am Ende zunächst nur alle akzeptierten zu nichtakzeptierten Zustände unterschiedlich sein müssen. Von dort wird rückwärts geschlossen, welche zwei Zustände mit einem beliebigen Zeichen auf unterschiedliche Zustände gelangen können, denn dann sind diese beiden Zustände auch nicht äquivalent. Sind am Ende alle paarweise nicht äquivalenten Zustände gefunden, so verbleiben die paarweise äquivalenten Zustände, die dann in Äquivalenzklassen zusammengeführt werden können.

BEISPIEL 3.37. Gegeben sei der deterministische endliche Automat  $M = (Q, \Sigma, \delta, q_s, E)$  mit  $\Sigma = \{a, b\}$ ,  $q_s = q_1$  und  $E = \{q_6\}$  mit dieser tabellarisch definierten Übergangsfunktion:

| $\delta$ | $q_1$ | $q_2$ | $q_3$ | $q_4$ | $q_5$ | $q_6$ |
|----------|-------|-------|-------|-------|-------|-------|
| a        | $q_2$ | $q_4$ | $q_2$ | $q_6$ | $q_2$ | $q_6$ |
| b        | $q_3$ | $q_3$ | $q_1$ | $q_5$ | $q_3$ | $q_6$ |

Abbildung 3.1.12 zeigt das Zustandsdiagramm dieses noch nicht minimierten Automaten. Zu Beginn ist zu sehen, dass es keine unerreichbaren Zustände gibt. Diese müssten vorab entfernt werden. Bei der Durchführung des Minimierungsverfahrens werden dann jeweils alle Paare von Zuständen identifiziert, die unterschiedlich in der Eigenschaft sind, ob das Wort dort enden kann oder nicht. Dies sind hier alle Paare von Zuständen von  $q_6$  und allen anderen, da nur dieser Zustand ein Endzustand ist. Es ist also

$$D_1 = \{ \{q_1, q_6\}, \{q_2, q_6\}, \{q_3, q_6\}, \{q_4, q_6\}, \{q_5, q_6\} \}.$$

Die Nummerierung  $D_1$  ist hier nur zur Übersicht eingeführt. Da die Paare als Mengen aufgezeichnet werden, spielt die Reihenfolge keine Rolle, denn beispielsweise ist  $\{q_1, q_6\} = \{q_6, q_1\}$ . Auch wenn diese Schreibweise im Beweis sehr kompakt ist, hilft in der Durchführung per Hand eine Dreieckstabelle, wo alle Einträge mit einem Kreuz markiert werden:

| $q_2$ |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|
| $q_3$ |       |       |       |       |       |
| $q_4$ |       |       |       |       |       |
| $q_5$ |       |       |       |       |       |
| $q_6$ | ×     | ×     | ×     | ×     | ×     |
|       | $q_1$ | $q_2$ | $q_3$ | $q_4$ | $q_5$ |

$$D_1 = \{ \{q_1, q_6\}, \{q_2, q_6\}, \{q_3, q_6\}, \{q_4, q_6\}, \{q_5, q_6\} \}.$$



Für alle verbleibenden Paare, wie zuerst  $\{q_1, q_2\}$  wird geprüft, was paarweise bei Anwendung von **a** oder **b** der nächste Zustand ist:  $\{\delta(q_1, a), \delta(q_2, a)\} = \{q_2, q_4\}$  und  $\{\delta(q_1, b), \delta(q_2, b)\} = \{q_3, q_3\}$ . Beide Mengen (das zweite Ergebnis ist ja kein Paar mehr) sind noch nicht in  $D_1$  und haben entsprechend kein Kreuz in der Tabelle. Eine erste Ergänzung erhalten wir bei  $\{\delta(q_1, a), \delta(q_4, a)\} = \{q_2, q_6\} \in D_1$ . Damit erhält  $\{q_1, q_4\}$  ein Kreuz, wie auch  $\{q_2, q_4\}$ ,  $\{q_3, q_4\}$  und  $\{q_4, q_5\}$ . Damit haben wir nach dem ersten Durchgang diese Tabelle:

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| q2 |    |    |    |    |    |
| q3 |    |    |    |    |    |
| q4 | ×  | ×  | ×  |    |    |
| q5 |    |    |    | ×  |    |
| q6 | ×  | ×  | ×  | ×  | ×  |
|    | q1 | q2 | q3 | q4 | q5 |

$$D_2 = \{ \{q_1, q_4\}, \{q_1, q_6\}, \{q_2, q_4\}, \{q_2, q_6\}, \{q_3, q_4\}, \{q_3, q_6\}, \{q_4, q_5\}, \{q_4, q_6\}, \{q_5, q_6\} \}.$$

In der nächsten Runde müssen wieder alle noch offenen Paare geprüft werden. Für das Paar  $\{q_1, q_2\}$  ist jetzt  $\{\delta(q_1, a), \delta(q_2, a)\} = \{q_2, q_4\} \in D_2$  und wird in dieser Runde markiert. Dies trifft auch auf die Paare  $\{q_2, q_3\}$  und  $\{q_2, q_5\}$  zu, aber keine weiteren, damit ergibt sich nach dieser Runde:

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| q2 | ×  |    |    |    |    |
| q3 |    | ×  |    |    |    |
| q4 | ×  | ×  | ×  |    |    |
| q5 |    | ×  |    | ×  |    |
| q6 | ×  | ×  | ×  | ×  | ×  |
|    | q1 | q2 | q3 | q4 | q5 |

$$D_3 = \{ \{q_1, q_2\}, \{q_1, q_4\}, \{q_1, q_6\}, \{q_2, q_3\}, \{q_2, q_4\}, \{q_2, q_5\}, \{q_2, q_6\}, \{q_3, q_4\}, \{q_3, q_6\}, \{q_4, q_5\}, \{q_4, q_6\}, \{q_5, q_6\} \}.$$

Für die verbleibenden Paare ergibt sich in der nächsten Runde keine Veränderung, wie bei  $\{q_1, q_3\}$ :

$$\{\delta(q_1, a), \delta(q_3, a)\} = \{q_2, q_2\} \notin D_3$$

$$\{\delta(q_1, b), \delta(q_3, b)\} = \{q_3, q_1\} \notin D_3$$

Damit endet die Schleife und es gibt diese Paare, die nicht in  $D$  beziehungsweise hier  $D_3$  auftauchen:

$$\{q_1, q_3\} \notin D_3 \Rightarrow q_1 \sim q_3$$

$$\{q_1, q_5\} \notin D_3 \Rightarrow q_1 \sim q_5$$

$$\{q_3, q_5\} \notin D_3 \Rightarrow q_3 \sim q_5$$

Damit können alle drei Zustände in einen Zustand  $[q_1]_{\sim} = \{q_1, q_3, q_5\}$  vereinigt werden und es ergibt sich der minimierte deterministische endliche Automat

$$M_{\min} = (Q/\sim, \Sigma, \delta/\sim, [q_s]_{\sim}, E/\sim)$$

$$Q/\sim = \{[q_1]_{\sim}, [q_2]_{\sim}, [q_4]_{\sim}, [q_6]_{\sim}\}$$

$$[q_s]_{\sim} = [q_1]_{\sim}$$

$$E/\sim = [q_6]_{\sim}$$

mit der Übergangsfunktion  $\delta/\sim$

|               |                |                |                |                |
|---------------|----------------|----------------|----------------|----------------|
| $\delta/\sim$ | $[q_1]_{\sim}$ | $[q_2]_{\sim}$ | $[q_4]_{\sim}$ | $[q_6]_{\sim}$ |
| a             | $[q_2]_{\sim}$ | $[q_4]_{\sim}$ | $[q_6]_{\sim}$ | $[q_6]_{\sim}$ |
| b             | $[q_1]_{\sim}$ | $[q_1]_{\sim}$ | $[q_1]_{\sim}$ | $[q_6]_{\sim}$ |

und das Zustandsdiagramm in Abbildung 3.1.13. Der Automat akzeptiert alle Wörter, die **aaa** enthalten.

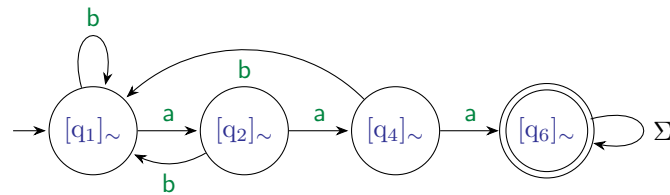


ABBILDUNG 3.1.13. Zustandsdiagramm des minimierten DEA aus Beispiel 3.37

BEISPIEL 3.38. Neben den Anwendungen in der Softwareentwicklung zur Erkennung von Mustern und Finden von Textbestandteilen sind endliche Automaten auch besonders in sicherheitskritischen Anwendungen, Benutzungsschnittstellen als auch sehr energiesparenden Geräten oder hochfrequenten Protokollen von sehr hoher Bedeutung:

Für sicherheitskritische Aufgaben ist es wichtig, dass ein System keine unerwarteten Nebenwirkungen hat. Ist eine Aufgabe so einfach, dass sie in übersichtlicher Art mit einem endlichen Automaten oder einer **Zustandsmaschine** gelöst werden kann, so hat dieses einfache Ausführungsmodell die großen Vorteile, dass im Gegensatz zu einem herkömmlichen Programm alle auftretenden Zustände im Voraus bekannt und überprüft werden können. Außerdem bietet sich die Darstellung des Zustandsdiagramms zur Dokumentation sehr an. Der kritische Start vieler Rechnersysteme wird über solche Zustandsdarstellungen beschrieben.

Für Benutzungsschnittstellen ergibt sich der unmittelbare Vorteil, dass im Fall der Beschreibung durch einen deterministischen endlichen Automaten immer alle Eingaben und Reaktionen berücksichtigt werden können und fehlende Beschreibungen unmittelbar auffallen. So bieten sich endliche Automaten auch alleine zur Dokumentation von Teilsystemen an.

Aktuell werden endliche Automaten sowohl zahlreich, und naheliegenderweise, in Industriesteuerungen aber auch in sehr energiebeschränkten Anwendungen im Bereich des **Internet-of-Things (IoT)** eingesetzt, wo bisher nur speziell hergestellte Chips oder programmierbare Logiken in der Form von **Field Programmable Gate Array (FPGA)** in Frage kamen. Beispielsweise wurde im Januar 2021 der sehr preiswerte und energiesparsame RP2040 Prozessor veröffentlicht, der neben einem herkömmlichen Zweikernprozessor auch 8 **Programmable Input/Output (PIO)** Zustandsmaschinen beinhaltet, die jetzt eine leichter programmierbare Alternative zu den bisherigen Technologien darstellt. Die Zustandsmaschinen sind natürlich nicht nur Akzeptoren sondern Transduktoren, die während eines Zustandübergangs Seiteneffekte auslösen.

Die Zustandsmaschinen teilen sich bis zu 32 mögliche Zustände, wo jede der Maschine mit einem „Program Counter“ und zugehörigen Ein- und Ausgängen sich den eigenen Zustand speichert. In einem Zustand wird in einem Takt einer von neun möglichen Befehlen ausgeführt, die neben der eigentlichen Aktion auch Pins konfigurieren und bis zu 31 zusätzliche Taktzyklen warten kann. Da die Zustandsmaschinen von weiteren Komponenten wie FIFO-Schlangen unterstützt werden, sind mit der geringen Anzahl von Zuständen sehr komplexe Aufgaben lösbar.

Die [Entwicklungsdokumentation der Zustandsmaschinen des RP2040](#) beschreibt Anwendungen in einem extrem großen Geschwindigkeitsspektrum von 1-10Hz für Taster oder LED, über das I2C-Protokoll (bis zu 100kHz), serielle Schnittstellen (10kHz-3MHz), USB (12MHz-480MHz), VGA (20-300MHz), Ethernet (10MHz-4GHz) bis hin zu HDMI/DVI (250MHz-200GHz). Die PIO des stromsparenden RP2040 arbeitet mit einem Takt von bis zu 125MHz, so sind HDMI/DVI eigentlich gerade nicht mehr zu erreichen, aber auch eine [direkte HDMI-Ausgabe mit dem RP2040](#) wurden mit minimaler zusätzlicher Elektronik (zum Anschluss der Buchsen) erreicht.

### 3.2. Kellerautomaten

Da im Satz 3.34 die Fähigkeiten von endlichen Automaten vollständig beschrieben wurden, ist klar, dass allgemeinere Sprachenklassen nicht durch dieses Rechnermodell erkennbar sind. Zur Erkennung der wichtigen Klasse von kontextfreien Sprachen ist ein mächtigeres Rechnermodell erforderlich. Dies ist auch an der Beispielsprache für den Typ-2

$$M_2 = \{0^n 1^n \mid n \in \mathbb{N}\} = \{01, 0011, 000111, 00001111, 0000011111, \dots\}$$

erkennbar, da ein erkennender Automat sich die Anzahl der führenden 0-Symbole merken muss und damit unendlich viele Zustände einnehmen kann. Genau dies kann ein endlicher Automat durch die Beschränkung der Zustände nicht leisten. Dabei ist diese Herausforderung überhaupt nicht nur theoretischer Natur, sondern schon arithmetische Ausdrücke können eine beliebige Anzahl von Klammern enthalten, wie diese Illustration zum Multiply-and-Square-Verfahren zeigt:

$$x = a^{17} = \underbrace{a \cdot \dots \cdot a}_{16 \text{ Multiplikationen}} = a \cdot a^{16} = a \cdot \underbrace{\left( \left( (a^2)^2 \right)^2 \right)^2}_{5 \text{ Multiplikationen}}$$

In jeder Klammerebene wird mindestens ein neuer Zustand erzeugt, der bei der Erkennung eines arithmetischen Ausdrucks erforderlich ist. Ein Konzept, das genau diese Eigenschaft abbilden kann und den endlichen Automaten ergänzen kann, ist der Stack, LIFO-, Stapel- oder Kellerspeicher. Dabei bleiben die zu beschreibenden Zustände des Automaten beschränkt und die nicht vorhersehbare Tiefe der Verschachtelung wird mit dem Stapel verarbeitet. Damit lässt sich ein Kellerautomat definieren, der kontextfreie Sprachen erkennen kann. Im Gegensatz zu den äquivalenten deterministischen und nichtdeterministischen endlichen Automaten besteht ein Unterschied zwischen der Mächtigkeit von deterministischen und nichtdeterministischen Kellerautomaten. Da die deterministischen Kellerautomaten nicht alle kontextfreien Sprachen erkennen können, werden hauptsächlich die nichtdeterministischen Kellerautomaten betrachtet:

----->

**DEFINITION 3.39.** Ein **nichtdeterministischer Kellerautomat (NKA)** wird durch das sieben-stellige Tupel  $K = (Q, \Sigma, \Gamma, \delta, q_s, \perp, E)$  beschrieben, wo  $Q$  die endliche Menge von Zuständen,  $\Sigma$  das Alphabet möglicher Eingaben,  $\Gamma$  das Alphabet des Kellerspeichers mit  $\Sigma \cap \Gamma = \emptyset$ ,  $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$  die Übergangsrelation,  $q_s \in Q$  den Startzustand,  $\perp \in \Gamma$  den Stackbegrenzer und  $E \subseteq Q$  die Menge der Endzustände beschreibt.

Eine **Konfiguration eines Kellerautomaten** ist ein Tripel  $(q, A, \Phi) \in Q \times \Sigma^* \times \Gamma^*$  und die initiale Konfiguration lautet  $(q_s, w, \perp)$  für ein  $w \in \Sigma^*$ . Liegt eine Konfiguration  $(q, xA, \gamma\Phi)$  für  $q \in Q$ ,  $x \in \Sigma \cup \{\varepsilon\}$ ,  $A \in \Sigma^*$ ,  $\gamma \in \Gamma$ ,  $\Phi \in \Gamma^*$  und ist  $(p, \Psi) \in \delta(q, x, \gamma)$  mit  $p \in Q$  und  $\Psi \in \Gamma^*$ , so sei die  $(p, A, \Psi\Phi)$  eine mögliche folgende Konfiguration, geschrieben  $(q, xA, \gamma\Phi) \xrightarrow{\delta} (p, A, \Psi\Phi)$ , wobei  $\Psi = \varepsilon$  möglich ist.

Die **Sprache eines nichtdeterministischen Kellerautomaten** ist

$$L(K) = \{w \in \Sigma^* \mid \exists \left( (q_s, w, \perp) \xrightarrow{\delta} \dots \xrightarrow{\delta} (q, \varepsilon, \gamma) \right) \text{ für ein } q \in E \text{ und beliebiges } \gamma \in \Gamma^* \}.$$

**BEMERKUNG 3.40.** Diese Definition ist eine Fortführung der Definitionen 3.1, 3.18 und 3.25 der endlichen Automaten, bei denen ein Wort akzeptiert wird, wenn ein Zustand aus  $E$  erreicht wird. Alternativ und äquivalent können Kellerautomaten auch ohne die Menge der Endzustände  $E$  definiert werden, wo ein Wort akzeptiert wird, wenn der Kellerspeicher zum letzten Zeichen leer ist. Das verkleinert die Definition, besonders, wenn in manchen Quellen zusätzlich der Stackbegrenzer fest auf das Symbol  $\perp$  definiert wird.

DEFINITION 3.41. Die Sprache eines **leerenden nichtdeterministischen Kellerautomaten** mit Tupel  $K = (Q, \Sigma, \Gamma, \delta, q_s, \perp)$  ist

$$L(K) = \{w \in \Sigma^* \mid \exists \left( (q_s, w, \perp) \xrightarrow{\delta} \dots \xrightarrow{\delta} (q, \varepsilon, \varepsilon) \right) \text{ für beliebiges } q \in Q\}.$$

SATZ 3.42. Die Kellerautomaten aus den Definitionen 3.39 und 3.41 sind gleich mächtig: Für jeden NKA  $K_1 = (Q_1, \Sigma, \Gamma, \delta_1, q_{s,1}, \perp, E)$  gibt es einen leerenden NKA  $K_2 = (Q_2, \Sigma, \Gamma, \delta_2, q_{s,2}, \perp)$  mit  $L(K_1) = L(K_2)$  und umgekehrt.

BEWEIS. Ist ein  $K_1$  gegeben, so ergibt sich ein äquivalenter  $K_2$  daraus, dass ein zusätzlicher Zustand  $q_e \notin Q_1$  eingeführt wird  $Q_2 = Q_1 \cup \{q_e\}$  und  $\delta_2$  ergibt sich aus  $\delta_1$  mit zusätzlichen  $\varepsilon$ -Übergängen von allen Zuständen in  $E$  und  $q_e$  zu  $q_e$ , wo alle möglichen Stacksymbole  $\Gamma$  eliminiert werden.

Ist ein  $K_2$  gegeben, so ergibt sich ein  $K_1$  durch Einführung von zwei neuen Zuständen  $q_s$  und  $q_e$  und einem neuen Stackbegrenzung-Symbol  $\top$ , das vom neuen  $q_s = q_s$  im ersten Schritt in  $\delta_1$  durch  $\varepsilon$ -Übergang auf den Stack geschrieben wird, und von allen Zuständen durch  $\varepsilon$ -Übergang von allen Zuständen in  $Q_2$  bei dem Symbol  $\top$  in den neuen Endzustand  $q_e$  übergeht. Ansonsten übernimmt  $\delta_1$  alle weiteren Übergänge von  $\delta_2$ .  $\square$

BEISPIEL 3.43. Die Sprache  $M_2 = \{0^n 1^n \mid n \in \mathbb{N}\}$  wird durch diesen Kellerautomaten  $K$  erkannt:

$$\begin{aligned} K &= (Q, \Sigma, \Gamma, \delta, q_s, \perp, E), \\ Q &= \{q_1, q_2, q_3\} \\ \Sigma &= \{0, 1\}, \\ \Gamma &= \{A\}, \\ q_s &= q_1, \\ E &= \{q_3\}, \end{aligned}$$

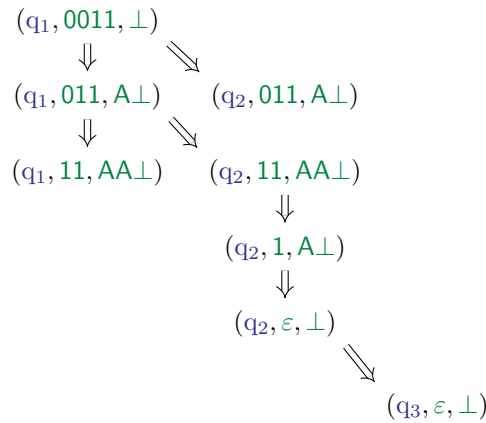
und dieser Übergangsrelation für das nächste Zeichen  $x$  im Wort und  $\gamma$  auf dem Kellerspeicher:

| $x$           | $\gamma$ | $q_1$                              | $q_2$                    | $q_3$       |
|---------------|----------|------------------------------------|--------------------------|-------------|
| 0             | $\perp$  | $\{(q_1, A\perp), (q_2, A\perp)\}$ | $\emptyset$              | $\emptyset$ |
| 0             | A        | $\{(q_1, AA), (q_2, AA)\}$         | $\emptyset$              | $\emptyset$ |
| 1             | $\perp$  | $\emptyset$                        | $\emptyset$              | $\emptyset$ |
| 1             | A        | $\emptyset$                        | $\{(q_2, \varepsilon)\}$ | $\emptyset$ |
| $\varepsilon$ | $\perp$  | $\emptyset$                        | $\{(q_3, \perp)\}$       | $\emptyset$ |
| $\varepsilon$ | A        | $\emptyset$                        | $\emptyset$              | $\emptyset$ |

Ein  $\varepsilon$ -Übergang ist ein spontaner Übergang, der ohne Akzeptanz eines Zeichens sich vollziehen kann: Hier tritt dieser am Ende auf, um vom Stackbegrenzer in den akzeptierten Zustand zu gelangen. Die Übergänge für ein Wort  $w = 0011$  aus der Konfiguration  $(q_1, 0011, \perp)$  zu diesem NKA sind in Abbildung 3.2.1 dargestellt.

Dieser Kellerautomat ist nur ein Beispiel für diese Sprache, da hier auch eine deterministische Variante formuliert werden kann, hier auch reduziert um nicht akzeptierte Kombinationen:

| $x$           | $\gamma$ | $q_1$                    | $q_2$                    | $q_3$       |
|---------------|----------|--------------------------|--------------------------|-------------|
| 0             | $\perp$  | $\{(q_1, A\perp)\}$      | $\emptyset$              | $\emptyset$ |
| 0             | A        | $\{(q_1, AA)\}$          | $\emptyset$              | $\emptyset$ |
| 1             | A        | $\{(q_2, \varepsilon)\}$ | $\{(q_2, \varepsilon)\}$ | $\emptyset$ |
| $\varepsilon$ | $\perp$  | $\emptyset$              | $\{(q_3, \perp)\}$       | $\emptyset$ |

ABBILDUNG 3.2.1. Übergänge des ersten NKA aus Beispiel 3.43 für  $w = 0011$ 

Ein äquivalenter leerender Kellerautomat kommt sogar mit nur zwei Zuständen aus:

| $x$           | $\gamma$ | $q_1$                    | $q_2$                    |
|---------------|----------|--------------------------|--------------------------|
| 0             | ⊥        | $\{(q_1, A\perp)\}$      | $\emptyset$              |
| 0             | A        | $\{(q_1, AA)\}$          | $\emptyset$              |
| 1             | A        | $\{(q_2, \varepsilon)\}$ | $\{(q_2, \varepsilon)\}$ |
| $\varepsilon$ | ⊥        | $\emptyset$              | $\{(q_2, \varepsilon)\}$ |

Letzterer ist keine Übersetzung nach Satz 3.42 sondern eine so optimierte Version, die statt dem letzten Wechsel in den akzeptierten Zustand einfach den Stapel leert.

**SATZ 3.44.** Für jede kontextfreie Grammatik  $G = (V, T, R, S)$  kann ein nichtdeterministischer Kellerautomat  $K = (Q, \Sigma, \Gamma, \delta, q_s, \perp, E)$  angegeben werden, so dass  $L(G) = L(K)$  gilt.

**BEWEIS.** Hier als Beweisskizze mit der Konstruktion: Der NKA kann direkt konstruiert werden: Zunächst ist  $Q = \{q_s, q, q_e\}$  mit  $q_s = q_s$  und  $E = \{q_e\}$ ,  $\Sigma = T$  und  $\Gamma = V \cup \hat{T}$ , wobei  $\hat{T}$  eine Bijektion zu  $T$  darstellt- hier einfach in Großbuchstaben- um  $\Sigma \cap \Gamma = \emptyset$  sicher zu stellen. Weiterhin ist  $\delta(q_s, \varepsilon, \perp) = \{(q, S\perp)\}$ , um den Stapel initial mit der Startvariablen zu belegen. Für jede Regel  $r : A \mapsto w$  mit  $A \in V$  und  $w \in (V \cup T)^*$  ist  $(q, \hat{w}) \in \delta(q, \varepsilon, A)$ , wobei  $\hat{w} \in (V \cup \hat{T})^*$  gerade  $w$  mit den ersetzten Terminalen ist. Weiterhin ist  $\delta(q, q, \hat{x}) = \{(q, \varepsilon)\}$ , um die Stapelrepräsentanten von Symbolen genau diese Symbole akzeptieren. Abschließend ist  $\delta(q, \varepsilon, \perp) = (q_e, \perp)$ , um die Erkennung zu beenden, wenn kein Zeichen mehr auf dem Stapel ist. Diese Konstruktion erzeugt genau nur die Linksableitungen der Wörtern der Sprache der Grammatik.  $\square$

**BEISPIEL 3.45.** Für ein Wort  $x = x_1 \dots x_n \in \Sigma^*$  über einem Alphabet  $\Sigma$  sei  $x^R = x_n \dots x_1$  das umgedrehte Wort. Eine Grammatik der Typ-2 **Sprache der geraden Palindrome**

$$L = \{xx^R \mid x \in \Sigma^*\}$$

über dem Alphabet  $\Sigma = \{a, b\}$  lautet:

$$\begin{aligned}
 V &= \{S, X\}, \\
 T &= \{a, b\}, \\
 R &= \{r_1, r_2\}, \\
 &\quad r_1 : S \mapsto \varepsilon \mid X, \\
 &\quad r_2 : X \mapsto aXa \mid bXb \mid aa \mid bb, \\
 S &= S.
 \end{aligned}$$

Ein entsprechend nach Satz 3.44 konstruierter NKA  $K = (Q, \Sigma, \Gamma, \delta, q_s, \perp, E)$  lautet dann:

$$Q = \{q_s, q, q_e\},$$

$$\Sigma = \{a, b\},$$

$$\Gamma = \{S, X, A, B\},$$

$$q_s = q_s,$$

$$E = \{q_e\}$$

mit  $\delta$  definiert durch diese Tabelle, wo die Symbole, die Variablen waren, weiter wie Variablen geschrieben werden:

| $x$           | $\gamma$ | $q_s$             | $q$                                        | $q_e$       |
|---------------|----------|-------------------|--------------------------------------------|-------------|
| $\varepsilon$ | $\perp$  | $\{(q, S\perp)\}$ | $\emptyset$                                | $\emptyset$ |
| $\varepsilon$ | $S$      | $\emptyset$       | $\{(q, \varepsilon), (q, X)\}$             | $\emptyset$ |
| $\varepsilon$ | $X$      | $\emptyset$       | $\{(q, AXA), (q, BXB), (q, AA), (q, BB)\}$ | $\emptyset$ |
| $a$           | $A$      | $\emptyset$       | $\{(q, \varepsilon)\}$                     | $\emptyset$ |
| $b$           | $B$      | $\emptyset$       | $\{(q, \varepsilon)\}$                     | $\emptyset$ |
| $\varepsilon$ | $\perp$  | $\emptyset$       | $\{(q_e, \perp)\}$                         | $\emptyset$ |

Es ist ersichtlich, dass die Zustände für die Erkennung der Sprache keine große Rolle spielen. Tatsächlich gibt es hier auch eine äquivalente Darstellung als **zustandslose Kellermaschine** gibt, die in jedem Schritt ein Zeichen oder  $\varepsilon$  sowie ein Zeichen vom Stack liest, und dann eine beliebige Anzahl von Zeichen auf den Stack schiebt.

**SATZ 3.46.** Für jeden nichtdeterministischen Kellerautomaten  $K = (Q, \Sigma, \Gamma, \delta, q_s, \perp, E)$  gibt es eine Grammatik  $G = (V, T, R, S)$ , so dass  $L(K) = L(G)$  gilt.

**BEWEIS.** Nur als Beweisskizze mit der Konstruktion: Die Grammatik kann direkt aus einem leerenden nichtdeterministischen Kellerautomaten konstruiert werden. Da jeder NKA einen äquivalenten leerenden NKA besitzt, ist das für den Beweis hinreichend. Dabei werden alle abzuarbeitenden Symbole auf dem Stapel zu Variablen der Grammatik. Da aber Variablen einer Grammatik jederzeit abgebildet können, hingegen der Kellerautomat dies nur in bestimmten Zuständen erlaubt, werden alle Symbole des Stapels zusätzlich unten mit Index von welchem Zustand und oben zu welchem Zustand ergänzt. Dadurch können pro Symbol des Stapels  $n^2 = |Q|^2$  Variablen, insgesamt also bis zu  $|\Gamma \cup \{\perp\}| \cdot |Q|^2$  Variablen zuzüglich einer Startvariablen, entstehen. Es ist also

$$V = \{S\} \cup \{X_p^q \mid X \in \Gamma \cup \{\perp\}, p, q \in Q\}$$

und es werden die folgenden Produktionen für die Startvariable definiert für die Abbildung vom Startzustand definiert:

$$S \mapsto \perp_{q_s}^p \text{ für alle } p \in Q$$

Damit kann die Variable von Startzustand zum simulierten Wechsel in einen beliebigen Zustand verwendet werden, was eventuell dann weitere Regeln ermöglichen. Die weiteren Übergänge dazu sind in  $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$  definiert und für jeden Eintrag

$$(p, \Psi) \in (q, a, X)$$

mit  $p, q \in Q$  und  $\Psi = (R(1), \dots, R(k)) \in \Gamma^*$ ,  $k \geq 0$ , sowie  $a \in \Sigma \cup \{\varepsilon\}$  und  $X \in \Gamma$  werden Produktionen folgendermaßen definiert: Ist  $k = 0$ , wird hier also nichts auf den Stapel geschrieben, so entstehen diese Produktionen:

$$X_p^q \mapsto a$$

Diese Produktion kann auf der rechten Seite ein  $\varepsilon$  sein, aber wie schon zuvor kann diese Grammatik dann in einem zweiten Schritt korrigiert werden. Ist hingegen  $k > 0$ , so seien  $u_1, \dots, u_k \in Q^k$  alle Vektorkombinationen von Zuständen. Dann entstehen diese  $|Q|^k$  Produktionen für jeden Vektor, optional mit Zeichen  $a$  voran, wenn es kein  $\varepsilon$  war:

$$x_q^{u_k} \mapsto aR(1)_p^{u_1} R(2)_{u_1}^{u_2} \dots R(k)_{u_{k-1}}^{u_k}$$

Dadurch entstehen alle möglichen Abfolgen von Zuständen, die beim sequentiellen Bearbeiten einseitig von links, wie bei einem Stapel, erfolgen können. Welche dabei überhaupt zu einem Ergebnis führen können, ergibt sich dann durch die weiteren Regeln, die ja Vorgaben über den Quell-Zustand beim Übergang machen.

Es lässt sich nun mit vollständiger Induktion sowohl zeigen, dass alle Akzeptanzzustände des NKA über entsprechende Produktionen der Grammatik erreicht werden können, und ebenso mit Induktion, dass alle Produktionen, die zu einem Wort aus Terminalen führen, über Zustandsänderungen des NKA abgebildet werden können.  $\square$

BEISPIEL 3.47. Der leerende NKA aus Beispiel 3.43 wurde durch diese Übergänge beschrieben:

| $x$           | $\gamma$ | $q_1$                    | $q_2$                    |
|---------------|----------|--------------------------|--------------------------|
| 0             | $\perp$  | $\{(q_1, A\perp)\}$      | $\emptyset$              |
| 0             | A        | $\{(q_1, AA)\}$          | $\emptyset$              |
| 1             | A        | $\{(q_2, \varepsilon)\}$ | $\{(q_2, \varepsilon)\}$ |
| $\varepsilon$ | $\perp$  | $\emptyset$              | $\{(q_2, \varepsilon)\}$ |

Daraus entstehen die folgenden Produktionen, wo in den Indizes  $q_1$  und  $q_2$  durch 1 und 2 abgekürzt werden:

|                                                              |  | Abbildungen vom Startzustand                             |
|--------------------------------------------------------------|--|----------------------------------------------------------|
| $S \mapsto \perp_1^1 \mid \perp_1^2$                         |  |                                                          |
| $\perp_1^1 \mapsto 0 A_1^1 \perp_1^1 \mid 0 A_1^2 \perp_1^1$ |  | $(q_1, A\perp) \in \delta(q_1, 0, \perp)$                |
| $\perp_1^2 \mapsto 0 A_1^1 \perp_1^2 \mid 0 A_1^2 \perp_1^2$ |  |                                                          |
| $A_1^1 \mapsto 0 A_1^1 A_1^1 \mid 0 A_1^2 A_1^1$             |  | $(q_1, AA) \in \delta(q_1, 0, A)$                        |
| $A_1^2 \mapsto 0 A_1^1 A_1^2 \mid 0 A_1^2 A_1^2$             |  |                                                          |
| $A_1^2 \mapsto 1$                                            |  | $(q_2, \varepsilon) \in \delta(q_1, 1, A)$               |
| $A_2^2 \mapsto 1$                                            |  | $(q_2, \varepsilon) \in \delta(q_2, 1, A)$               |
| $\perp_2^2 \mapsto \varepsilon$                              |  | $(q_2, \varepsilon) \in \delta(q_2, \varepsilon, \perp)$ |

Die vielen Produktionen werden natürlich deutlich weniger, wenn alle die Produktionen wegfallen, die links nicht mehr auftreten:

$$\begin{aligned}
 S &\mapsto \perp_1^2 \\
 \perp_1^2 &\mapsto 0 A_1^2 \perp_1^2 \\
 A_1^2 &\mapsto 0 A_1^2 A_2^2 \\
 A_1^2 &\mapsto 1 \\
 A_2^2 &\mapsto 1 \\
 \perp_2^2 &\mapsto \varepsilon
 \end{aligned}$$

Nach der Elimination von  $\varepsilon$  auf der rechten Seite und Zusammenfassung gleicher Quellen oder direkter Abbildungen verbleibt, zur Illustration auch mit anderen Variablen:

$$\begin{aligned}
 S &\mapsto 0 A_1^2 & S &\mapsto 0 X \\
 A_1^2 &\mapsto 0 A_1^2 A_2^2 \mid 1 & X &\mapsto 0 X Y \mid 1 \\
 A_2^2 &\mapsto 1 & Y &\mapsto 1
 \end{aligned}$$

Die Ableitung von 000111 erfolgt mit dieser Grammatik dann so:

$$S \Rightarrow 0X \Rightarrow 00XY \Rightarrow 000XYY \Rightarrow 0001YY \Rightarrow 00011Y \Rightarrow 000111$$

**SATZ 3.48.** *Die Menge der von kontextfreien Grammatik erzeugbaren Sprachen ist gleich der Menge von nichtdeterministischen Kellerautomaten akzeptierbaren Sprachen.*

**BEWEIS.** Dies fasst die Ergebnisse der Sätze 3.44 und 3.46 zusammen: Zu jeder kontextfreien Grammatik kann ein nichtdeterministischer Kellerautomat konstruiert werden, und umgekehrt für jeden nichtdeterministischen Kellerautomaten eine kontextfreie Grammatik abgeleitet werden kann.  $\square$

**BEISPIEL 3.49.** Für das Beispiel 3.45 der geraden Palindrome wurde ein NKA aus der Grammatik hergeleitet. Der dort abgeleitete Automat erzeugt zunächst mögliche Muster bevor überhaupt ein tatsächliches Matching mit dem zu untersuchendem Wort erfolgt. Bei ungeschickter Wahl der Reihenfolge könnte der Automat sequentiell ewig rechnen und zu keinem Ergebnis kommen. Mit zwei Zuständen ist es auch etwas direkter möglich mit dem Vorteil, dass es sequentiell terminieren würde: Erst wird die erste Hälfte für jedes Zeichen ein entsprechendes Stapelzeichen in den Stack gespeichert und dann für die zweiten Hälfte mit jeweils dem nächsten Stackzeichen verglichen. Für ein Alphabet  $|\Sigma| > 1$  (sonst wäre sie sogar regulär) ist diese Sprache aber von einem deterministischen Kellerautomaten nicht zu erkennen: Vergleichen wir die Worte  $w_1 = 1001$  und  $w_2 = 100001$ , so kann ein deterministischer Kellerautomat nach dem zweiten Zeichen nicht wissen, ob die erste Hälfte schon beendet ist. Dazu bräuchte es ein explizites Trennungszeichen, das einem deterministischen Kellerautomaten das Umschalten zwischen den Zuständen zum Speichern und Überprüfen mit beispielsweise  $T \in \Sigma$  ermöglicht:

$$\tilde{L} = \{xTx^R \mid x \in (\Sigma \setminus \{T\})^*\}$$

Für nichtdeterministische Kellerautomaten sind beide Sprachen erkennbar. Diesen Unterschied in der Erkennbarkeit ermöglicht auch eine weitere Klassifikation von formalen Sprachen. Die Erkennbarkeit durch einen deterministischen Kellerautomaten definiert eine Sprachklasse von **determiniert kontextfreien Sprachen**, die echt zwischen den regulären Sprachen und den kontextfreien Sprachen liegt und durch die Verwendung eines deterministischen Automaten linear ausgewertet werden kann.

Offensichtlich ist es zur Analyse einer kontextfreien Sprache vorteilhaft, wenn diese mit einem deterministischen Automaten erkannt werden kann. Aber auch sonst ist die Situation in der Regel nicht so problematisch, da auch bei einem nichtdeterministischen Kellerautomaten zumindest ohne  $\epsilon$ -Übergänge in jedem Schritt ein Zeichen reduziert wird und damit die Anzahl der verbleibenden Schritte abschätzbar endlich ist. Es gibt aber sinnvolle Erweiterungen, die den Umfang der effizient erkennbaren Sprachen deutlich erweitert.

Zwar charakterisiert nun die Menge der durch nichtdeterministische Kellerautomaten akzeptierbare Sprachen die Menge der kontextfreien Sprachen weiter, doch erneut entsteht daraus nicht direkt ein Ausschlusskriterium, dass eine gegebene Sprache nicht kontextfrei sein kann, wenn einfach nur noch keine Grammatik oder Kellerautomat gefunden wurde. Entsprechend zum Pumpinglemma für reguläre Sprachen gibt es Pumpinglemma auch für kontextfreie Sprachen, das für eine gegebene Sprache helfen kann, die Konstruierbarkeit auszuschließen. Dazu wird wie bei der Größe des akzeptierenden Automaten bei regulären Sprachen eine Beschränkung der Wortlänge bei kontextfreien Sprachen benötigt.

**LEMMA 3.50.** *Sei  $G = (V, T, R, S)$  eine kontextfreie Grammatik für eine beliebige Typ2-Sprache  $L = L(G)$  in Chomsky-Normalform nach Definition 2.21, die nach Satz 2.25 für jede kontextfreie Sprache existiert. Dann besitzt jedes Wort  $w \in L$ , einen endlichen binären Ableitungsbaum aus  $G$ . Sei  $n$  zusätzlich die Tiefe des Ableitungsbaums. Dann hat  $w$  weiterhin eine durch die Tiefe beschränkte Länge  $|w| \leq 2^{n-1}$ .*

**BEWEIS.** Ist  $w = \epsilon$ , so ist die Aussage für  $|w| = 0$  trivial erfüllt. Für alle weitere Wortlängen folgt die Aussage induktiv:



Ist  $n = 1$ , so besteht der Ableitungsbaum aus genau einer Ableitung der Form  $\mathbf{X} \mapsto \mathbf{x}$ , und das Wort besteht aus genau einem Terminal. Die Aussage  $|w| = 1 \geq 2^{1-1} = 1$  ist erfüllt und der Ableitungsbaum ist ein (degenerierter) Binärbaum mit maximal zwei Ästen an jedem Knoten.

Seien die Aussagen für alle Wörter mit Ableitungsbäumen der Tiefen  $k = 1, \dots, n-1$  erfüllt. Ein Wort mit Ableitungsbaum der Tiefe  $k = n$  besteht nach erster Ableitung der Form  $\mathbf{X} \mapsto \mathbf{X}_1 \mathbf{X}_2$  aus maximal zwei binären Teilbäumen der Längen  $n-1$  und ist damit auch ein Binärbaum. Nach Induktionsvoraussetzung gilt für die Länge der aus den Teilbäumen entstehenden Teilworte  $w_1, w_2$  die Eigenschaft  $|w_1|, |w_2| \leq 2^{n-2}$  und damit ist  $|w| = |w_1 w_2| = |w_1| + |w_2| \leq 2^{n-2} + 2^{n-2} = 2 \cdot 2^{n-2} = 2^{n-1}$ .  $\square$

**SATZ 3.51. (Pumping-Lemma für kontextfreie Sprachen)** *Zu jeder kontextfreien Sprache  $L$  gibt es eine Zahl  $l$ , so dass sich alle Worte  $x \in L$  mit  $|x| \geq l$  in  $x = uvwyz$  zerlegen lassen mit*

- (1)  $|vy| > 0$ ,
- (2)  $|vwy| < l$  und
- (3)  $uv^iwy^iz \in L$  für alle  $i \geq 0$ .

**BEWEIS.** Sei  $G = (V, T, R, S)$  eine Grammatik zur Sprache  $L$  in Chomsky-Normalform nach Satz 2.25 und  $m = |V|$  die Anzahl der Variablen der Grammatik. Dann sei  $l = 2^m$  und sei  $x \in L$  beliebig mit  $|x| \geq l$ .

Nach Lemma 3.50 gibt es zur Erzeugung von  $x$  einen Binärbaum, in dem mindestens ein Ast die Länge  $m+1$  besitzt, also  $m+1$  Ableitungen besitzt. Seien die Variablen der linken Seiten der letzten  $m+1$  Ableitungen durch  $A_0, \dots, A_m$  durchnummeriert. Da es nur  $m$  Variablen gibt, muss es dann  $i < j$  geben, so dass  $A = A_i = A_j \in V$  gilt  $A \Rightarrow \mathbf{VAY}$  mit  $\mathbf{V}, \mathbf{Y} \in (V \cup T)^*$  und sowohl  $\mathbf{VY} \neq \epsilon$  und damit  $|vy| > 0$  mit  $\mathbf{V} \Rightarrow v$  und  $\mathbf{Y} \Rightarrow y$  im Ableitungsbaum, da es keine Regel  $A \mapsto A$  geben kann. Weiterhin gilt auch  $|vwy| \leq l$  für  $A \Rightarrow w$  im Ableitungsbaum, da in den letzten  $m+1$  Ableitungen maximal  $2^m = l$  Symbole nach Lemma 3.50 im Teilbaum ab  $A_i$  entstehen können.

Da im Ableitungsbaum der Teil  $A \Rightarrow \mathbf{VAY}$  und dann  $A \Rightarrow w$  auftritt, kann dieser Teilbaum auch komplett durch  $A \Rightarrow w$  ersetzt werden, um einen gültigen Ableitungsbaum der Grammatik für das Wort  $uwz = uv^0wy^0z \in L$  zu erzeugen. Entsprechend kann aber auch  $A \Rightarrow w$  induktiv durch  $A \Rightarrow \mathbf{VAY}$  für, was gültige Ableitungsbäume der Grammatik für die Worte  $uv^iwy^iz \in L$  für  $i > 1$  erzeugt.  $\square$

Damit kann eine weitere Sprache aus Beispiel 1.11 jetzt eindeutig charakterisiert werden:

**SATZ 3.52.** *Die Sprache  $M_1 = \{0^n 1^n 2^n \mid n \in \mathbb{N}\}$  ist keine Typ-2 Sprache also nicht kontextfrei.*

**BEWEIS.** Angenommen sie wäre kontextfrei, so gäbe es nach dem Pumping-Lemma für kontextfreie Sprachen ein  $l$ , so dass jedes Wort  $|x| \geq l$  zerlegbar ist. Sei  $n \geq \frac{l}{3}$ , dann ist das Wort  $x = 0^n 1^n 2^n \in M_1$  und  $|x| \geq l$ . Damit muss es eine Zerlegung  $x = uvwyz$  geben. Dann gibt es zwei Möglichkeiten:

- (1) Die Teile  $v$  und  $y$  bestehen zusammen nur aus einer Art von Symbolen. Dann hat  $uwz$  nicht mehr eine identische Anzahl von 0, 1 und 2. Damit kann dieser Fall nicht auftreten.
- (2) Mindestens einer der Teile  $v$  oder  $y$  besteht aus mehr als einer Art von Symbolen. Dann liegt  $uv^2wy^2z$  nicht mehr in der sortierten Reihenfolge  $0^m 1^m 2^m$  für ein  $m > n$  vor. Damit ist dieser Fall auch ausgeschlossen.

Da beide Fälle nicht auftreten können und es keine weiteren Möglichkeiten gibt, muss die Annahme, dass die Sprache  $M_1$  kontextfrei wäre, verworfen werden.  $\square$

Im Gegensatz zu endlichen Automaten oder folgend eingeführten Turingmaschinen werden Kellerautomaten seltener in physischen Rechnermodellen umgesetzt. Trotzdem beschreibt die Kellermaschine genau das erforderliche und praktische Vorgehen in der Analyse, Behandlung von kontextfreien Sprachen in der Softwareentwicklung. Dies wird nach Definition einer kontextfreien Grammatik wie in Beispielen 2.31 und 2.32 beschrieben mit Hilfe von durch Lexer und Parser oder mit einem Parsergenerator generierten Programmteilen oder Bibliotheken in einer höheren Programmiersprache realisiert.

### 3.3. Turingmaschinen

Soweit wurden Rechnermodelle vorgestellt, die den regulären und kontextfreien Sprachen entsprechen. Sowohl für kontextsensitive und allgemeine Chomsky-Sprachen wird das allgemeinere Rechnermodell der Turingmaschinen benötigt, die im Gegensatz zum gesteuerten LIFO-Zugriff einer Kellermaschine einen wahlfreien Zugriff auf einen Speicher ermöglichen. Die Einschränkung auf kontextsensitive Sprachen wird sich nur in der Beschränkung des verfügbaren Speichers im Gegensatz zur beliebigen Speichergröße bei allgemeinen Chomsky-Sprachen ausgestalten.

**DEFINITION 3.53.** Eine **deterministische Turingmaschine (TM)** ist ein Tupel  $M = (Q, \Sigma, \delta, q_s, q_e)$  mit einer endlichen Menge  $Q$  der Zustände,  $\Sigma$  einem Alphabet von Symbolen, einer Übergangsfunktion

$$\delta : (Q \setminus \{q_f\}) \times (\Sigma \cup \{\sqcup, \varepsilon\}) \rightarrow Q \times (\Sigma \cup \{\sqcup\}) \times \{\mathbf{L}, \mathbf{R}\},$$

einem Startzustand  $q_s \in Q$  und einem Endzustand  $q_e \in Q$ .

Eine **Konfiguration einer Turingmaschine** ist ein Tupel

$$(q, w, a, u) \in Q \times (\Sigma \cup \{\sqcup\})^* \times (\Sigma \cup \{\sqcup, \varepsilon\}) \times (\Sigma \cup \{\sqcup\})^*$$

geschrieben  $(q, w \underline{a} u)$ , wo die Vorstellung ist, dass die Turingmaschine in diesem Moment ihren Schreib- und Lesekopf auf  $a$  hat. Die initiale Konfiguration für ein Wort  $w \in \Sigma^*$  ist  $(q_s, \sqcup w_1 w_2 \dots w_n \sqcup)$ . Die Konfiguration  $(q, w \underline{a} u)$  geht abhängig von  $\delta(q, a) = (p, b, c)$  in die nächste Konfiguration über, geschrieben  $(q, w \underline{a} u) \xrightarrow{\delta} (p, \tilde{w} \tilde{a} \tilde{u})$  oder die TM endet akzeptierend nach folgenden Regeln:

- (1) Ist  $p = q_e$ , so akzeptiert die Turingmaschine das Wort und endet.
- (2) Ist  $c = \mathbf{L}$  und  $w = w_1 \dots w_n$ , so ist die nächste Konfiguration  $(p, w_1 \dots w_{n-1} \underline{w_n} b u)$ . Die **beschränkte Turingmaschine** erlaubt dies nur für  $|w| \geq 1$ , sonst ist  $w = \varepsilon$  mit nächster Konfiguration  $(p, \varepsilon \underline{\varepsilon} b u)$  zugelassen.
- (3) Ist  $c = \mathbf{R}$  und  $u = u_1 \dots u_n$ , so ist die nächste Konfiguration  $(p, w b \underline{u_1} u_2 \dots u_n)$ . Die **beschränkte Turingmaschine** erlaubt dies nur für  $|u| \geq 1$ , sonst ist  $u = \varepsilon$  mit nächster Konfiguration  $(p, w b \underline{\varepsilon} \varepsilon)$  zugelassen.

Die **Sprache einer Turingmaschine**  $M$  ist

$$L(M) = \{w \in \Sigma^* \mid \exists \left( (q_s, \sqcup w_1 w_2 \dots w_n \sqcup) \xrightarrow{\delta} \dots \xrightarrow{\delta} (q_e, \tilde{w} \tilde{a} \tilde{u}) \right) \text{ für beliebiges } \tilde{w} \tilde{a} \tilde{u} \in (\Sigma \cup \{\sqcup\})^* \}.$$

**BEISPIEL 3.54.** Eine Turingmaschine kann die kontextsensitive Sprache  $M_1 = \{0^n 1^n 2^n \mid n \in \mathbb{N}\}$  akzeptieren. Dies kann beispielsweise über eine Maschine realisiert werden, die ein Wort  $0^n 1^n 2^n$  schrittweise auf  $0^{n-1} 1^{n-1} 2^{n-1}$  reduziert und akzeptiert, wenn im letzten Schritt kein Symbol mehr übrig ist. Dazu wird zunächst eine 0 mit einem  $\sqcup$  gelöscht, dann sucht der Schreib- und Lesekopf die erste 2 und überschreibt dann die letzte 1 davor mit einer 2. Dann sucht es die letzte 2 vor dem Ende und löscht die letzten beiden 2 mit  $\sqcup$ . Jetzt wurde das Wort auf  $0^{n-1} 1^{n-1} 2^{n-1}$  reduziert. Jetzt läuft der Schreib- und Lesekopf zurück zur ersten 0, wenn es noch eine Zahl gibt, sonst endet die TM akzeptierend. Im Detail hat die Maschine  $M = (Q, \Sigma, \delta, q_s, q_e)$  die Zustände  $Q = \{q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9\}$ , das

Alphabet  $\Sigma = \{0, 1, 2\}$ , den Startzustand  $q_s = q_1$ , den Endzustand  $q_e = q_9$  und die Übergangsfunktion hat diese Zuordnungen (ohne Komma und Klammern der Tupel):

$q_1 0 \mapsto q_2 \text{R}$ ,  $q_2 0 \mapsto q_2 0 \text{R}$ ,  $q_3 1 \mapsto q_4 2 \text{R}$ ,  $q_4 2 \mapsto q_4 2 \text{R}$ ,  $q_5 2 \mapsto q_6 \text{L}$ ,  $q_6 2 \mapsto q_7 \text{L}$ ,  $q_7 \text{ } \mapsto q_9 \text{R}$ ,  $q_8 2 \mapsto q_8 2 \text{L}$ ,  
 $q_2 1 \mapsto q_2 1 \text{R}$ ,  $q_4 \text{ } \mapsto q_5 \text{L}$ ,  $q_7 2 \mapsto q_8 2 \text{L}$ ,  $q_8 1 \mapsto q_8 1 \text{L}$ ,  
 $q_2 2 \mapsto q_3 2 \text{L}$ ,  $q_8 0 \mapsto q_8 0 \text{L}$ ,  
 $q_8 \text{ } \mapsto q_1 \text{R}$ .

Damit wird das Wort  $001122 \in M_1$  so erkannt (ohne Klammern und teilweise Komma der Tupel):

| Konfiguration                                            | Konfiguration                                                       | Zuordnung                           | Erklärung                              |
|----------------------------------------------------------|---------------------------------------------------------------------|-------------------------------------|----------------------------------------|
| $q_1, \text{ } \underline{0} 01122 \text{ } \Rightarrow$ | $q_2, \text{ } \text{ } \underline{0} 1122 \text{ } \Rightarrow$    | $q_1 0 \mapsto q_2 \text{R}$        | Löschen der ersten 0 und rechts.       |
| $\Rightarrow$                                            | $q_2, \text{ } \text{ } \underline{0} 1 122 \text{ } \Rightarrow$   | $q_2 0 \mapsto q_2 0 \text{R}$      | Zur ersten 2 laufen.                   |
| $\Rightarrow$                                            | $q_2, \text{ } \text{ } \underline{0} 1 1 22 \text{ } \Rightarrow$  | $q_2 1 \mapsto q_2 1 \text{R}$      | Zur ersten 2 laufen.                   |
| $\Rightarrow$                                            | $q_2, \text{ } \text{ } \underline{0} 1 1 2 2 \text{ } \Rightarrow$ | $q_2 1 \mapsto q_2 1 \text{R}$      | Zur ersten 2 laufen.                   |
| $\Rightarrow$                                            | $q_3, \text{ } \text{ } \underline{0} 1 1 22 \text{ } \Rightarrow$  | $q_2 2 \mapsto q_3 2 \text{L}$      | Zurück zur letzten 1.                  |
| $\Rightarrow$                                            | $q_4, \text{ } \text{ } \underline{0} 1 2 2 2 \text{ } \Rightarrow$ | $q_3 1 \mapsto q_4 2 \text{R}$      | Die 1 zur 2 wandeln und rechts.        |
| $\Rightarrow$                                            | $q_4, \text{ } \text{ } \underline{0} 1 2 2 2 \text{ } \Rightarrow$ | $q_4 2 \mapsto q_4 2 \text{R}$      | Hinter die letzte 2 laufen.            |
| $\Rightarrow$                                            | $q_4, \text{ } \text{ } \underline{0} 1 2 2 2 \text{ } \Rightarrow$ | $q_4 2 \mapsto q_4 2 \text{R}$      | Hinter die letzte 2 laufen.            |
| $\Rightarrow$                                            | $q_5, \text{ } \text{ } \underline{0} 1 2 2 2 \text{ } \Rightarrow$ | $q_4 \text{ } \mapsto q_5 \text{L}$ | Zurück zur letzten 2.                  |
| $\Rightarrow$                                            | $q_6, \text{ } \text{ } \underline{0} 1 2 2 \text{ } \Rightarrow$   | $q_5 2 \mapsto q_6 \text{L}$        | Letzte 2 löschen und links.            |
| $\Rightarrow$                                            | $q_7, \text{ } \text{ } \underline{0} 1 2 \text{ } \Rightarrow$     | $q_6 2 \mapsto q_7 \text{L}$        | Vorletzte 2 löschen und links.         |
| $\Rightarrow$                                            | $q_8, \text{ } \text{ } \underline{0} 1 2 \text{ } \Rightarrow$     | $q_7 2 \mapsto q_8 2 \text{L}$      | Noch nicht fertig, also wieder zurück. |
| $\Rightarrow$                                            | $q_8, \text{ } \text{ } \underline{0} 1 2 \text{ } \Rightarrow$     | $q_8 1 \mapsto q_8 1 \text{L}$      | Zurück vor das erste Zeichen.          |
| $\Rightarrow$                                            | $q_8, \text{ } \text{ } \underline{0} 1 2 \text{ } \Rightarrow$     | $q_8 0 \mapsto q_8 0 \text{L}$      | Zurück vor das erste Zeichen.          |
| $\Rightarrow$                                            | $q_1, \text{ } \text{ } \underline{0} 1 2 \text{ } \Rightarrow$     | $q_8 \text{ } \mapsto q_1 \text{R}$ | Neustart.                              |
| $\Rightarrow$                                            | $q_2, \text{ } \text{ } \underline{1} 2 \text{ } \Rightarrow$       | $q_1 0 \mapsto q_2 \text{R}$        | Löschen der ersten 0 und rechts.       |
| $\Rightarrow$                                            | $q_2, \text{ } \text{ } \underline{1} 2 \text{ } \Rightarrow$       | $q_2 1 \mapsto q_2 1 \text{R}$      | Zur ersten 2 laufen.                   |
| $\Rightarrow$                                            | $q_3, \text{ } \text{ } \underline{1} 2 \text{ } \Rightarrow$       | $q_2 2 \mapsto q_3 2 \text{L}$      | Zurück zur letzten 1.                  |
| $\Rightarrow$                                            | $q_4, \text{ } \text{ } \underline{2} 2 \text{ } \Rightarrow$       | $q_3 1 \mapsto q_4 2 \text{R}$      | Die 1 zur 2 wandeln und rechts.        |
| $\Rightarrow$                                            | $q_4, \text{ } \text{ } \underline{2} 2 \text{ } \Rightarrow$       | $q_4 2 \mapsto q_4 2 \text{R}$      | Hinter die letzte 2 laufen.            |
| $\Rightarrow$                                            | $q_5, \text{ } \text{ } \underline{2} 2 \text{ } \Rightarrow$       | $q_4 \text{ } \mapsto q_5 \text{L}$ | Zurück zur letzten 2.                  |
| $\Rightarrow$                                            | $q_6, \text{ } \text{ } \underline{2} \text{ } \Rightarrow$         | $q_5 2 \mapsto q_6 \text{L}$        | Letzte 2 löschen und links.            |
| $\Rightarrow$                                            | $q_7, \text{ } \text{ } \underline{\text{ }} \text{ } \Rightarrow$  | $q_6 2 \mapsto q_7 \text{L}$        | Vorletzte 2 löschen und links.         |
| $\Rightarrow$                                            | $q_9, \text{ } \text{ } \underline{\text{ }} \text{ } \Rightarrow$  | $q_7 \text{ } \mapsto q_9 \text{R}$ | Da fertig in den Endzustand.           |

Dieses Beispiel definiert eine Turingmaschine, die am Beispiel die Erkennung eines Wortes der Sprache  $M_1$  demonstriert. Offensichtlich benötigt diese Turingmaschine keinen zusätzlichen Speicher, sondern arbeitet alleine auf dem beschränkten Speicher, der durch die Länge des zu analysierenden Wortes gegeben ist. Entsprechend der bisherigen Ausführungen ist das auch zu erwarten, da die Sprache  $M_1$  vom Chomsky Typ-1 sein soll und daher durch eine beschränkte Turingmaschine erkannt werden können soll. Obwohl die grundlegenden Eigenschaften schon vorliegen, sind wie bei den Kellermaschinen deterministische beschränkte Turingmaschinen etwas zu beschränkt, um alle kontextsensitiven Sprachen zu erkennen. Mit nichtdeterministischen beschränkten Turingmaschinen können aber genau die Typ-1 Sprachen abgebildet werden:

**SATZ 3.55.** Für jede nichtdeterministische beschränkte Turingmaschine  $M$ , das ist eine beschränkte Turingmaschine mit einer Übergangsrelation

$$\delta : (Q \setminus \{q_f\}) \times (\Sigma \cup \{\text{ }, \varepsilon\}) \rightarrow \mathcal{P}(Q \times (\Sigma \cup \{\text{ }, \text{L}, \text{R}\}))$$

statt einer Übergangsfunktion, gibt es eine Typ-1 Grammatik  $G$ , die die akzeptierte Sprache  $L(M) = L(G)$  beschreibt, und für jede Typ-1 Grammatik  $G$  gibt es eine nichtdeterministische beschränkte Turingmaschine  $M$ , für die  $L(G) = L(M)$  ist. Eine nichtdeterministische beschränkte Turingmaschine wird auch **linear beschränkter Automat (LBA)** genannt.

BEWEIS. Die Beweise für beide Aussagen finden sich beispielsweise in [4].

□

Eine interessante Kenntnis aus dem Satz ist, dass nichtdeterministische beschränkte Turingmaschinen auch die Sprachen aller beschränkten Chomsky Grammatiken erkennen können. Solche **beschränkte Chomsky-Grammatiken** sind allgemeine Chomsky Grammatiken mit der zusätzlichen Anforderung, dass alle Regeln die Länge mindestens erhalten müssen oder die Wörter verlängern. Da alle kontextsensitiven Grammatiken sowieso beschränkte Chomsky Grammatiken sind, ist die erstaunliche Erkenntnis, dass es für alle beschränkten Grammatiken auch kontextsensitive Grammatiken geben muss. Genau so eine Konstruktion einer kontextsensitiven Grammatik aus einer beschränkten Grammatik für  $M_1$  in Beispiel 2.4 wurde in Beispiel 2.9 demonstriert. Die Anforderung eine Erhaltung von Prefix oder Postfix hat sich damit in Bezug auf den Umfang der Sprache in theoretischer Betrachtung als nicht relevant herausgestellt sondern alleine die „Verlängerungsbeschränkung“, aus der auch die Entscheidbarkeit folgt, gibt den Ausschlag. Trotzdem ist die Berücksichtigung von Pre- und Postfix für eine effizientere Analyse von Typ-1 Sprachen natürlich relevant und sollte in einer tatsächlichen Implementation nach Möglichkeit berücksichtigt werden.

Ohne Beschränkung des Speichers gibt es im Bezug auf die akzeptierten Sprachen keinen Unterschied zwischen deterministischen und nichtdeterministischen Turingmaschinen. Dies liegt alleine schon daran, dass für jede nichtdeterministische Turingmaschine eine deterministische Turingmaschine angegeben werden kann, die die gleiche Sprache akzeptiert. Das wird daraus ersichtlich, dass es in jedem Schritt einer nichtdeterministischen Turingmaschine wegen endlich vieler Zustände und endlicher Alphabete höchstens endlich viele Verzweigungen  $m$  gibt. Damit gibt es auch nach einer Anzahl von  $n$  Schritten höchstens  $m^n$  viele unterschiedliche Zweige. Akzeptiert nun eine nichtdeterministische Turingmaschine ein Wort, so muss dies nach endlich vielen Schritten, beispielsweise  $n$  Schritten, erfolgen. Die Simulation der nichtdeterministischen Turingmaschine durch eine deterministische Turingmaschine würde nach Simulation der  $m^n$  Varianten, wenn hintereinander jede Variante immer nur bis gleicher Schrittlänge simuliert wird, ebenso nach endlicher Zeit zum identischen Ergebnis kommen. In [4] sind die Details zur Konstruktion einer solchen simulierenden deterministischen Turingmaschine dargestellt. Die deterministischen Turingmaschinen ohne Beschränkung des Speichers sind nun tatsächlich geeignet, um allgemeine Chomsky Typ-0 Sprachen zu erkennen, und sind zu ihnen sogar äquivalent:

**SATZ 3.56.** *Für jede allgemeine Chomsky-Grammatik  $G$  gibt es eine Turingmaschine  $M$ , für die  $L(G) = L(M)$  gilt, und für jede Turingmaschine  $M$  gibt es eine allgemeine Chomsky Grammatik  $G$ , für die  $L(M) = L(G)$  gilt.*

**BEWEIS.** Die Beweise für beide Aussagen finden sich beispielsweise in [4]. □

**SATZ 3.57.** *Die Menge der möglichen Turingmaschinen ist abzählbar.*

**BEWEIS.** Der Satz 3.56 liefert hier noch nicht einmal eine Aussage über die Abzählbarkeit, da es ja überabzählbar viele Turingmaschinen geben könnte, die einer Grammatik zugeordnet werden könnte. Das ist natürlich nicht der Fall, aber für die Abzählbarkeit muss eine berechenbare Methode angegeben werden. Der Nachweis dieser Abzählbarkeit wird auch **Gödelisierung** genannt, obwohl nicht unbedingt dafür eine Gödelnummer zum Einsatz kommen muss. Zunächst reicht es für eine Turingmaschine  $M = (Q, \Sigma, \delta, q_s, q_e)$  neben Start- und Endzustand die Übergangsfunktion zu beschreiben. Die Zustände und das verwendete Alphabet inklusive der zusätzlichen Befehle  $\Sigma \cup \{\_, \mathbf{L}, \mathbf{R}\}$  werden jeweils durchnummeriert mit  $n : Q \rightarrow \mathbb{N}$  und  $m : \Sigma \cup \{\_, \mathbf{L}, \mathbf{R}\} \rightarrow \mathbb{N}$  und im Folgenden durch ihre Zahlen repräsentiert. Jede Zustandsänderung (von endlich vielen) hat die Form

$$r_k : q_k, x_k \mapsto s_k, y_k$$

und kann daher über vier Zahlen  $n(q_k), m(x_k), n(s_k), m(y_k)$  repräsentiert werden. Bei einer Darstellung mit Gödelnummer könnte die Maschine damit so in eine Zahl gewandelt werden:

$$c(M) = p_1^{n(q_s)} \cdot p_2^{n(q_e)} \cdot \prod_k p_{4k-1}^{n(q_k)} \cdot p_{4k}^{m(x_k)} \cdot p_{4k+1}^{n(s_k)} \cdot p_{4k+2}^{m(y_k)}$$

Dabei kann einer Turingmaschine durch unterschiedliche Reihenfolgen der aufgezählten Änderungen mehrere, aber endliche viele, Gödelnummern zugeordnet werden. Über eine Faktorisierung können die einzelnen Ziffern wieder rekonstruiert werden, und eindeutig eine Turingmaschine rekonstruiert werden, soweit sie gültig ist.

Eine sonst häufig in der Literatur verwendete Darstellung ist die Erzeugung eines Worts aus Binärziffern mit einer Funktion  $b : \mathbb{N} \rightarrow \{0, 1\}^*$  und einem Trennzeichen  $\#$ , der in einem zweiten Schritt dann über eine weitere Kodierung in eine Zahl gewandelt wird. Das Wort hätte dann die Darstellung

$$\#b(n(q_s))\#b(n(q_e))\underbrace{\#b(n(q_1))\#b(m(x_1))\#b(n(s_1))\#b(m(y_1))}_{r_1}\underbrace{\#b(n(q_2))}_{r_2}\# \dots$$

und mit der Kodierung  $0 \mapsto 00$ ,  $1 \mapsto 01$ ,  $\# \mapsto 10$  kann eine lange Binärzahl und eine Zahl mit dem Stellenwertsystem erzeugt werden, die entsprechend leichter als mit einer Faktorisierung wieder zu einer Turingmaschine eindeutig rekonstruiert werden kann.  $\square$

Im Grunde entspricht das Ergebnis der früheren Erkenntnis aus Satz 1.21, die schon als zur Aufzählbarkeit ergänzbaren Abzählbarkeit von Programmen gedeutet wurde. Nur ist bis hier noch nicht klar, wann und ob eine Programmiersprache und deren Ausführungsmodell überhaupt mit einer Turingmaschine verglichen werden kann, deren Fähigkeiten nun als Basis genommen werden kann. Um genau diese Frage der Berechenbarkeit und der dafür erforderliche Aufwand ist das Thema des folgenden Kapitels.

Es gibt auch weitere Rechnermodelle, wie beispielsweise reversible Rechenmodelle, wo jede Operation umkehrbar sein muss. Diese Einschränkung ist beispielsweise eine wichtige Eigenschaft bei der Entwicklung von Algorithmen für Quantencomputer. Zwar hat das Rechenmodell für Quantencomputer die Menge der damit berechenbaren Funktionen im Vergleich zur Turingmaschine bisher nicht erweitert, doch können bisher nur mit großem Aufwand zu lösende Probleme wie einer Faktorisierung plötzlich auf einer andere Weise gelöst werden, die deutlich frühere Ergebnisse verspricht.

### 3.4. Aufgaben

**AUFGABE 47.** Gegeben sei der deterministische endliche Automat  $M = (Q, \Sigma, \delta, q_s, E)$  mit  $\Sigma = \{a, b\}$ ,  $q_s = q_1$  und  $E = \{q_1, q_4\}$  mit dieser tabellarisch definierten Übergangsfunktion:

| $\delta$ | $q_1$ | $q_2$ | $q_3$ | $q_4$ | $q_5$ |
|----------|-------|-------|-------|-------|-------|
| a        | $q_2$ | $q_3$ | $q_5$ | $q_2$ | $q_5$ |
| b        | $q_5$ | $q_4$ | $q_4$ | $q_5$ | $q_5$ |

Bestimmen Sie das Zustandsdiagramm und die akzeptierte Sprache  $L(M)$ .

**AUFGABE 48.** Gegeben sei der deterministische endliche Automat  $M = (Q, \Sigma, \delta, q_s, E)$  mit  $\Sigma = \{a, b\}$ ,  $q_s = q_1$  und  $E = \{q_3\}$  mit dieser tabellarisch definierten Übergangsfunktion:

| $\delta$ | $q_1$ | $q_2$ | $q_3$ | $q_4$ | $q_5$ |
|----------|-------|-------|-------|-------|-------|
| a        | $q_2$ | $q_5$ | $q_2$ | $q_3$ | $q_5$ |
| b        | $q_4$ | $q_3$ | $q_4$ | $q_5$ | $q_5$ |

Bestimmen Sie das Zustandsdiagramm und die akzeptierte Sprache  $L(M)$ .

**AUFGABE 49.** Bestimmen Sie für diesen Mealy-Automaten  $M = (Q, \Sigma, \delta, q_s, \Gamma, \omega)$  mit  $Q = \{q_1, q_2\}$ ,  $\Sigma = \{a, b\}$ ,  $\Gamma = \{x, y\}$ ,  $q_s = q_1$  und diesen Übergangs- und Ausgabefunktionen ein Zustandsdiagramm, bestimmen Sie die Ausgabe zum Wort **baaba** und leiten Sie einen äquivalenten Moore-Automaten als Zustandsdiagramm ab:

| $\delta / \omega$ | $q_1$   | $q_2$   | $q_3$   |
|-------------------|---------|---------|---------|
| a                 | $q_2/x$ | $q_3/x$ | $q_1/x$ |
| b                 | $q_3/y$ | $q_2/x$ | $q_3/y$ |

**AUFGABE 50.** Bestimmen Sie für diesen Mealy-Automaten  $M = (Q, \Sigma, \delta, q_s, \Gamma, \omega)$  mit  $Q = \{q_1, q_2\}$ ,  $\Sigma = \{a, b\}$ ,  $\Gamma = \{x, y\}$ ,  $q_s = q_1$  und diesen Übergangs- und Ausgabefunktionen ein Zustandsdiagramm, bestimmen Sie die Ausgabe zum Wort **babba** und leiten Sie einen äquivalenten Moore-Automaten als Zustandsdiagramm ab:

| $\delta / \omega$ | $q_1$   | $q_2$   | $q_3$   |
|-------------------|---------|---------|---------|
| a                 | $q_2/x$ | $q_2/x$ | $q_3/y$ |
| b                 | $q_1/y$ | $q_3/y$ | $q_2/y$ |

**AUFGABE 51.** Bestimmen Sie gemäß behandeltem Beispiel 3.17 das Zustandsdiagramm und Kodierung eines Moore-Automaten, der die Rock-Paper-Scissors Strategie verfolgt, dass immer die Folge **RPSRPSRPS**... unabhängig von den Zügen des Gegners ausgegeben wird.

**AUFGABE 52.** Bestimmen Sie gemäß behandeltem Beispiel 3.17 das Zustandsdiagramm und Kodierung eines Moore-Automaten, der die Rock-Paper-Scissors Strategie verfolgt, dass immer nach einem ersten **R** das Zeichen wählt, das gegen den letzten Zug des Gegners gewinnt.

**AUFGABE 53.** Gegeben sei der nichtdeterministische endliche Automaten  $M = (Q, \Sigma, \delta, q_s, E)$  mit  $Q = \{q_1, q_2, q_3, q_4\}$ ,  $\Sigma = \{a, b\}$ ,  $q_s = q_1$ ,  $E = \{q_1\}$  mit der tabellarischen Übergangsrelation:

| $\delta$ | $q_1$       | $q_2$     | $q_3$       | $q_4$       |
|----------|-------------|-----------|-------------|-------------|
| a        | $\{q_2\}$   | $\{q_3\}$ | $\emptyset$ | $\{q_1\}$   |
| b        | $\emptyset$ | $\{q_4\}$ | $\{q_1\}$   | $\emptyset$ |

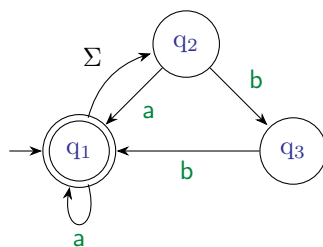
Bestimmen Sie das Zustandsdiagramm und die akzeptierte Sprache  $L(M)$ .

**AUFGABE 54.** Gegeben sei der nichtdeterministische endliche Automaten  $M = (Q, \Sigma, \delta, q_s, E)$  mit  $Q = \{q_1, q_2, q_3, q_4\}$ ,  $\Sigma = \{a, b\}$ ,  $q_s = q_1$ ,  $E = \{q_1\}$  mit der tabellarischen Übergangsrelation:

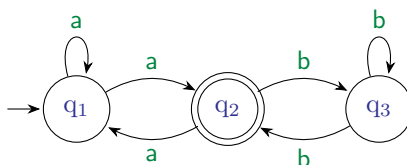
| $\delta$ | $q_1$          | $q_2$       | $q_3$       | $q_4$       |
|----------|----------------|-------------|-------------|-------------|
| a        | $\{q_1, q_2\}$ | $\emptyset$ | $\{q_4\}$   | $\emptyset$ |
| b        | $\{q_3\}$      | $\{q_4\}$   | $\emptyset$ | $\{q_1\}$   |

Bestimmen Sie das Zustandsdiagramm und die akzeptierte Sprache  $L(M)$ .

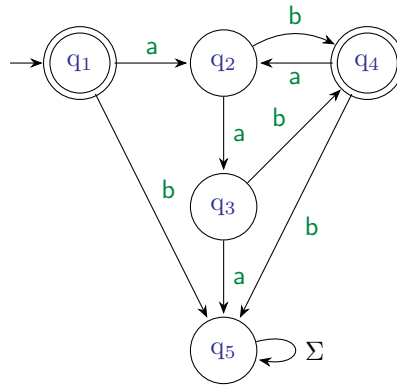
**AUFGABE 55.** Bestimmen Sie einen äquivalenten deterministischen endlichen Automaten samt Zustandsdiagramm für den nichtdeterministischen endlichen Automaten mit diesem Zustandsdiagramm:



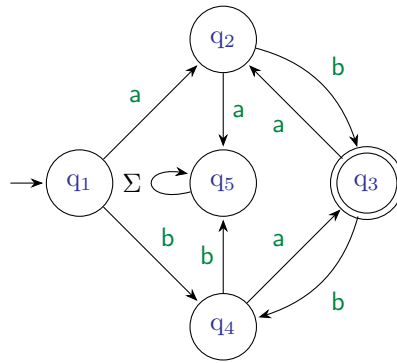
**AUFGABE 56.** Bestimmen Sie einen äquivalenten deterministischen endlichen Automaten samt Zustandsdiagramm für den nichtdeterministischen endlichen Automaten mit diesem Zustandsdiagramm:



**AUFGABE 57.** Geben Sie zu diesem als Zustandsdiagramm gegebenen deterministischen endlichen Automaten  $M$  eine reguläre Grammatik  $G$  an, so dass  $L(M) = L(G)$  gilt:



**AUFGABE 58.** Geben Sie zu diesem als Zustandsdiagramm gegebenen deterministischen endlichen Automaten  $M$  eine reguläre Grammatik  $G$  an, so dass  $L(M) = L(G)$  gilt:



**AUFGABE 59.** Geben Sie für die Grammatik  $G = (V, T, R, S)$

$$\begin{aligned}
 V &= \{\text{w, x, y, z}\}, \\
 T &= \{\text{a, b}\}, \\
 R &= \{r_1, r_2, r_3, r_4\}, \\
 &\quad r_1 : \text{w} \mapsto \text{ax} \mid \text{bw}, \\
 &\quad r_2 : \text{x} \mapsto \text{ax} \mid \text{by} \mid \text{b}, \\
 &\quad r_3 : \text{y} \mapsto \text{bz}, \\
 &\quad r_4 : \text{z} \mapsto \text{aw} \mid \text{bx} \mid \text{a}, \\
 S &= \text{w}.
 \end{aligned}$$

einen nichtdeterministischen endlichen Automaten  $M$  an, für den  $L(G) = L(M)$  gilt.

**AUFGABE 60.** Geben Sie für die Grammatik  $G = (V, T, R, S)$

$$\begin{aligned}
 V &= \{\text{w, x, y, z}\}, \\
 T &= \{\text{a, b}\}, \\
 R &= \{r_1, r_2, r_3, r_4\}, \\
 &\quad r_1 : \text{w} \mapsto \text{ax} \mid \text{by}, \\
 &\quad r_2 : \text{x} \mapsto \text{ay} \mid \text{bz} \mid \text{b}, \\
 &\quad r_3 : \text{y} \mapsto \text{aw}, \\
 &\quad r_4 : \text{z} \mapsto \text{bw} \mid \text{a}, \\
 S &= \text{w}.
 \end{aligned}$$

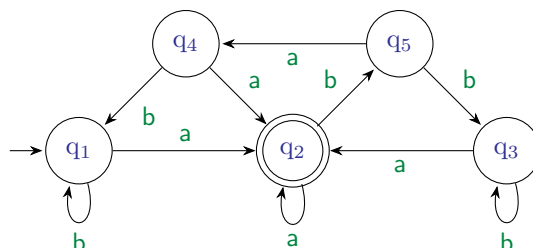
einen nichtdeterministischen endlichen Automaten  $M$  an, für den  $L(G) = L(M)$  gilt.



**AUFGABE 61.** Zeigen Sie, dass  $L = \{ww \mid w \in \{0,1\}^*\}$  keine reguläre Sprache ist.

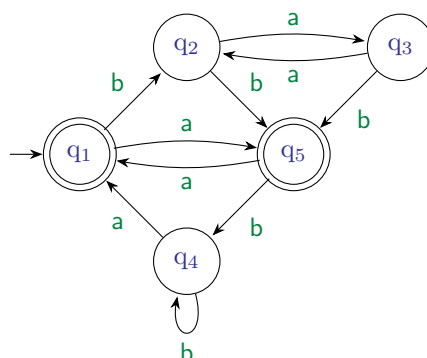
**AUFGABE 62.** Zeigen Sie, dass  $L = \{1^{(n^2)} \mid n \in \mathbb{N}\}$  keine reguläre Sprache ist.

**AUFGABE 63.** Vereinfachen Sie diesen deterministischen endlichen Automaten:



Geben Sie das Zustandsdiagramm des vereinfachten deterministischen Automaten an.

**AUFGABE 64.** Vereinfachen Sie diesen deterministischen endlichen Automaten:



Geben Sie das Zustandsdiagramm des vereinfachten deterministischen Automaten an.

**AUFGABE 65.** Konstruieren Sie einen Kellerautomaten für die **Sprache der ungeraden Palindrome**

$$L = \{xyx^R \mid x \in \Sigma^*, y \in \Sigma\}$$

über  $\Sigma = \{a, b\}$  und bestimmen Sie alle erforderlichen Übergänge für das Wort  $ababa \in L$ .

**AUFGABE 66.** Konstruieren Sie einen Kellerautomaten für die **Dyck-Sprache** oder **Klammersprache**  $D_1 = L(G_1)$ , die durch die folgende Grammatik  $G_1 = (V, T, R, S)$  definiert wird:

$$\begin{aligned} V &= \{S, A\}, \\ T &= \{(\, , )\}, \\ R &= \{r_1, r_2\}, \\ r_1 : S &\mapsto \varepsilon \mid A, \\ r_2 : A &\mapsto (\, \mid (A) \mid AA, \\ S &= S. \end{aligned}$$

und bestimmen Sie alle Übergänge für das Wort  $((())) \in L$ . (Die weiteren Dyck-Sprachen wie  $D_2$  oder  $D_3$  erlauben mehr, also hier zwei oder drei, Klammertypen.)

AUFGABE 67. Zeigen Sie, dass  $L = \{a^n b^n c^m \mid n, m \in \mathbb{N}, m \leq n\}$  keine kontextfreie Sprache ist.

AUFGABE 68. Zeigen Sie, dass  $L = \{ww \mid w \in \{0, 1\}^*\}$  keine kontextfreie Sprache ist.

AUFGABE 69. Bestimmen Sie eine Turing-Maschine, die die Sprache  $L = \{0^{2^k} \mid k \in \mathbb{N}_0\}$  akzeptiert.

AUFGABE 70. Bestimmen Sie eine Turing-Maschine, die die Sprache  $L = \{0^{(2^k)} \mid k \in \mathbb{N}_0\}$  akzeptiert.

AUFGABE 71. Bestimmen Sie die Turing-Maschine aus zwei Zuständen, die auf einem Band mit lauter 0 möglichst viele 1 erzeugt und terminiert.

AUFGABE 72. Bestimmen Sie die Turing-Maschine aus drei Zuständen, die auf einem Band mit lauter 0 möglichst viele 1 erzeugt und terminiert.

## Berechenbarkeit und Komplexität

### 4.1. Berechenbarkeit

Aus den bisherigen Erkenntnissen an die Konzepte der Entscheidbarkeit und der Rechnermodellen schließt sich die bisher nicht vollständig beantwortete Frage der Berechenbarkeit einer Chomsky-Sprache aus Definition 1.26 und Bemerkung 1.27 an: Wenn Turingmaschinen mächtig genug sind, um diese Sprachen zu erkennen, so kann auch eine Berechenbarkeit auch über Turingmaschinen definiert werden.

DEFINITION 4.1. Eine  $n$ -stellige Funktion

$$f : (\Sigma^+)^n \rightarrow \Sigma^*, n \geq 1$$

ist **Turing-berechenbar** oder **berechenbar**, wenn es eine Turingmaschine  $M = (Q, \Sigma, \delta, q_s, q_e)$  gibt, die zur Berechnung des Funktionswertes von  $f(x_1, \dots, x_n)$  für die Funktion zulässige  $x_1, \dots, x_n$  nach Erkennung des Wortes  $w = x_1 \sqcup \dots \sqcup x_n$  die Konfiguration am Ende  $(q_e, \sqcup y \sqcup)$  für  $y = f(x_1, \dots, x_n)$  lautet.

BEMERKUNG 4.2. Vor so einer Definition steht natürlich die Frage, welche weitere sinnvolle Definitionen der „Berechenbarkeit“ es gibt, und in welchem Zusammenhang diese miteinander stehen. Das Grundprinzip zur Nachweis von vergleichbaren Möglichkeiten und auch der Äquivalenz haben wir schon in Satz 3.22 kennen gelernt: Ein Modell ist mindestens so mächtig, wie ein anderes, wenn ein „sinnvoller“ Simulator in einem Modell für das andere Modell angegeben werden kann. Wenn sie sich sogar gegenseitig „sinnvoll“ simulieren können, so werden die beiden Modelle zueinander äquivalent bezeichnet. Mit einer „sinnvollen“ Simulation ist dabei häufig gemeint, dass der Aufwand im Sinne der im folgenden Abschnitt definierten Komplexität maximal polynomial (in Zeit oder Speicheraufwand) abgeschätzt werden kann, ansonsten wäre die Simulation bei größeren Problemen schwer durchführbar.

Für eine Simulation von Quantencomputern mit Hilfe von Turingmaschinen ist beispielsweise noch keine „sinnvolle“ Simulation in polynomialer Zeit bekannt. Auch wenn Quantencomputer nichts berechnen können, was nicht auch mit einer Turingmaschine berechnet werden kann, so konnte inzwischen der [Shor-Algorithmus](#) in polynomialer Zeit tatsächlich (bisher nur relativ kleine) Zahlen auf real existierenden Quantencomputern faktorisieren, jedoch ist entsprechend auf Turingmaschinen bisher kein Algorithmus bekannt ist, der diese Aufgabe in polynomialer Zeit lösen kann.

Ein wichtiger Schritt zur Simulation anderer Rechnermodelle ist der Nachweis, dass eine Turingmaschine sich überhaupt selbst simulieren kann:

SATZ 4.3. (**Universelle Turingmaschine**) Für jede Eingabe einer Turingmaschine gibt es eine Turingmaschine, die die Ausführung simuliert.

BEWEIS. Den Beweis führte Turing schon 1936, siehe beispielsweise aber auch [7, 1]. □

Eine universelle Turingmaschine beinhaltet also auf dem Band sowohl die Definition der Turingmaschine als auch die Daten, ein Modell, dass in der **Von-Neumann-Architektur** dann auch zu einem Modell für tatsächlich aufgebaute Computer wurde.

Damit können ganz praktisch auch Mengen von Turingmaschinen auch als Sprachen interpretiert werden, die von Turingmaschinen akzeptiert werden. Es gibt also auch eine Menge oder eine Sprache der korrekt formulierten Turingmaschinen, die jeweils für sich unterschiedliche Eigenschaften haben, wie beispielsweise, dass sie terminieren oder nicht. Es kann keine Turingmaschine geben, die in endlicher Zeit für alle Turingmaschinen entscheidet, welche Maschinen terminieren oder nicht. Genau genommen gilt das aber grundsätzlich für **alle nicht-triviale Eigenschaften**, das sind Eigenschaften, für die es jeweils mindestens eine Turingmaschine gibt, die sie erfüllen oder nicht erfüllen. Das hat unmittelbare Auswirkungen auf Aussagen, die wir auf Programmen treffen können:

**SATZ 4.4. (Satz von Rice)** *Jede nicht triviale Eigenschaft der rekursiv aufzählbaren Sprachen ist unentscheidbar.*

**BEWEIS.** Siehe beispielsweise [7, 1]. Die Idee ist, dass die Repräsentanten für die Eigenschaften zu nicht-entscheidbaren Turingmaschinen erweitert werden.  $\square$

Damit ist beispielsweise vorab grundsätzlich nicht allgemein prüfbar, ob ein Programm ein erwartetes Ergebnis liefert. Daraus darf aber nicht gefolgert werden, dass es keinen Sinn macht Softwaretests oder Codescreening durchzuführen, denn die Verfahren können in sehr vielen oder vielleicht auch in fast allen relevanten Fällen helfen: Die Aussage des Satzes ist nur, dass es immer Fälle gibt, wo eine Prüfung nicht erfolgreich sein kann.

Um weitere Rechnermodelle simulieren zu können, gibt es erweiterte Turingmaschinen, wie die **Mehrband-Turingmaschine**, die auf mehreren unabhängigen unendlichen Bändern gleichzeitig operieren kann:

**DEFINITION 4.5.** Eine  **$k$ -Band-Turingmaschine** ist ein Tupel  $M = (Q, \Sigma, \delta, q_s, q_e)$  mit einer endlichen Menge  $Q$  der Zustände,  $\Sigma$  einem Alphabet von Symbolen, im Falle einer deterministischen Maschine mit einer Übergangsfunktion

$$\delta : (Q \setminus \{q_f\}) \times (\Sigma \cup \{\_, \varepsilon\})^k \rightarrow Q \times (\Sigma \cup \{\_, \mathbf{L}, \mathbf{R}\})^k ,$$

im Falle einer nichtdeterministischen Maschine mit einer Übergangsrelation

$$\delta : (Q \setminus \{q_f\}) \times (\Sigma \cup \{\_, \varepsilon\})^k \rightarrow \mathcal{P}(Q \times (\Sigma \cup \{\_, \mathbf{L}, \mathbf{R}\})^k) ,$$

einem Startzustand  $q_s \in Q$  und einem Endzustand  $q_e \in Q$ . Eine Konfiguration einer  $k$ -Band-Turingmaschine ist ein Tupel

$$(q, w, a, u) \in Q \times ((\Sigma \cup \{\_\})^*)^k \times (\Sigma \cup \{\_, \varepsilon\})^k \times ((\Sigma \cup \{\_\})^*)^k ,$$

das aus einer initialen Belegung der Bänder mit der Übergangsfunktion bzw. -relation in jeweils den nächsten Zustand oder der nächsten Zustandsmenge übergeht.

Da deterministische  $k$ -Band-Turingmaschinen durch einfache Turingmaschinen simuliert werden können, ergeben sich aus diesen keine anderen Ergebnisse in Bezug auf die Berechenbarkeit, doch wird es deutlich einfacher eine Assembler-nahe **Registermaschine** zu beschreiben, die durch eine  $k$ -Band-Turingmaschine simuliert werden kann.

Diese besteht beispielsweise aus einem Programm ( $P_B$ ) mit einem beschränkten Befehlssatz, einem Akkumulator  $A$ , einem Registerspeicher ( $R_n$ ) und einem Programmzähler  $B$ , der die aktuelle Position im Programm mit Eingabe ( $x_i$ ) und Ausgabe ( $y_j$ ) beschreibt. Der Befehlssatz kann beispielsweise Inhalte im Speicher in den Akkumulator laden, speichern, damit rechnen oder bedingte Sprünge im Programm ausführen. Die Tabelle 2 zeigt einen Befehlssatz einer Registermaschine. Da Registermaschinen dem Rechenmodell moderner Prozessoren entsprechen, sind damit alle höheren Programmiersprachen,

| Befehl in $P_B$ | Beschreibung                                          | Aktion                                                                                       |
|-----------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>INP</b>      | Wert in Akkumulator $A$ lesen                         | $A := x_i$<br>$i := i + 1$<br>$B := B + 1$                                                   |
| <b>OUT</b>      | Wert aus $A$ ausgeben                                 | $z_j := A$<br>$j := j + 1$<br>$B := B + 1$                                                   |
| <b>LDA #n</b>   | Wert $n$ in $A$ laden                                 | $A := n$<br>$B := B + 1$                                                                     |
| <b>LDA n</b>    | Wert vom Register $R_n$ in $A$ laden                  | $A := R_n$<br>$B := B + 1$                                                                   |
| <b>LDA (n)</b>  | Wert vom Register $R_{R_n}$ indirekt laden            | $A := R_{R_n}$<br>$B := B + 1$                                                               |
| <b>STA n</b>    | Wert von $A$ in das Register $n$ speichern            | $R_n := A$<br>$B := B + 1$                                                                   |
| <b>STA (n)</b>  | Wert von $A$ indirekt in Register $R_{R_n}$ speichern | $R_{R_n} := A$<br>$B := B + 1$                                                               |
| <b>ADD #n</b>   | Wert $n$ zu $A$ addieren                              | $A := A + n$<br>$B := B + 1$                                                                 |
| <b>SUB #n</b>   | Wert $n$ von $A$ abziehen                             | $A := \max(0, A - n)$<br>$B := B + 1$                                                        |
| <b>JMP n</b>    | Direkter Sprung zur Programmposition $n$              | $B := n$                                                                                     |
| <b>BEQ #i,n</b> | Sprung zur Programmposition $n$ , wenn $A = i$        | $B := \begin{cases} n, & \text{falls } A = i \\ B + 1, & \text{falls } A \neq i \end{cases}$ |

TABELLE 2. Befehlssatz einer Registermaschine für Programm ( $P_B$ ) mit Eingabe ( $x_i$ ), Ausgabe ( $y_j$ ), Akkumulator  $A$ , Befehlszähler  $B$  und Register ( $R_n$ ) nach [1]

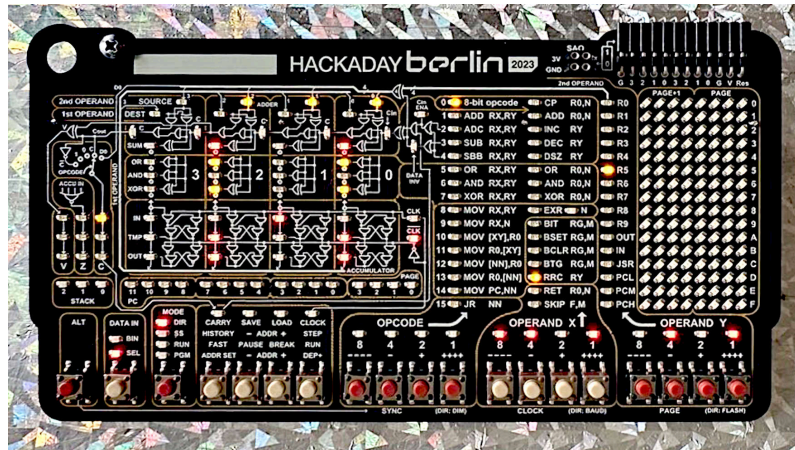


ABBILDUNG 4.1.1. Registermaschine als Badge der HackADay Berlin 2023, CC-BY L. Hohl

die auf modernen Prozessoren ausgeführt werden können, unmittelbar auch von einer Turingmaschine ausführbar, nur eventuell mit vielen Schritten selbst für kleinste Programme.

DEFINITION 4.6. Eine Programmiersprache oder ein Rechenmodell wird **Turing-vollständig** bezeichnet, wenn dieses durch eine Turingmaschine simuliert, als auch damit eine Turingmaschine simuliert werden kann.

Damit kann jedes Turing-vollständige System die gleichen Berechnungen wie eine Turingmaschine durchführen, und die Definition 4.1 ist auf alle Turing-vollständigen Systeme anwendbar. Und für jede auf

einem klassischen Computer ausführbaren Programmiersprache muss nur ein Simulator einer Turingmaschine umsetzbar sein, damit eine Turing-Vollständigkeit nachgewiesen ist.

Tatsächlich sind klassische hohen Programmiersprachen auf klassischen Computern wie Java, Python und C samt Derivate alle Turing-vollständig, da in ihnen ein Simulator für die Turingmaschine umgesetzt werden kann. Offenbar spielt die Möglichkeit für Schleifen oder Rekursion eine unmittelbar wichtige Rolle für eine Turing-Vollständigkeit, doch was bedeutet das genau?

DEFINITION 4.7. Die folgenden Funktionen sind **primitiv rekursiv**:

- (1) Die Nullfunktion  $\text{null}(n) = 0$ ,
- (2) die Nachfolgefunktion  $\text{succ}(n) = n + 1$ ,
- (3) die Projektion  $\text{proj}_i(x_1, \dots, x_n) = x_i$ ,
- (4) Verknüpfungen  $h(g_1(x_1, \dots, x_n), \dots, g_m(x_1, \dots, x_n))$  primitiv rekursiver  $h, g_1, \dots, g_m$ ,
- (5) und primitive Rekursionen von primitiv rekursiven Funktionen  $g$  und  $h$ :

$$f(m, n) = \begin{cases} g(n) & \text{für } m = 0, \\ h(f(m-1, n), m-1, n) & \text{für } m > 0. \end{cases}$$

BEISPIEL 4.8. Die Addition  $\underbrace{1 + \dots + 1}_{m \text{ mal}} + n = m + n$  ist primitiv rekursiv:

$$\text{add}(m, n) = \begin{cases} \text{proj}_1(n) = n & \text{für } m = 0, \\ \text{succ}(\text{add}(m-1, n)) & \text{für } m > 0. \end{cases}$$

Ebenso sind die Multiplikation  $\text{mul}(m, n)$  und die Potenzierung  $\text{pow}(m, n)$  primitiv rekursiv.

Um einen Eindruck von der Mächtigkeit der primitiv rekursiven Funktionen zu erhalten, hilft der Vergleich mit der Loop-Programmiersprache:

DEFINITION 4.9. Die **Loop-Sprache** besteht aus Variablen ( $x_i$ ) als Platzhalter für nicht-negative ganze Zahlen, der Zahl 0, den Funktionen **SUCC**( $n$ ) :=  $n + 1$  und **PRED**( $n$ ) =  $\max(n - 1, 0)$ , sowie einer Schleife **LOOP**...**DO**...**END**, deren Inhalt entsprechend des Inhalts einer Variable oft ausgeführt wird. Eine mögliche PEG-Beschreibung der Loop-Sprache lautet:

```
start:      w* command w*
command:    command ";" w* command | assignment | loop
assignment: variable w* "!=" w* expression w*
expression: "0" | variable | function
variable:   "x" number
function:   "PRED(" w* expression w* ")" | "SUCC(" w* expression w* ")"
loop:       "LOOP" w* variable w* "DO" w* command w* "END" w*
number:     ( "1" .. "9" ) ( "0" .. "9" ) *
w:         "\u" | "\t" | "\n"
```

BEISPIEL 4.10. Eine Addition  $x_3 = x_1 + x_2$  sieht in der Loop-Sprache so aus, analog in Python:

|                               |                                     |
|-------------------------------|-------------------------------------|
| <code>x3 := x1;</code>        | <code>x3 = x1</code>                |
| <code>LOOP x2 DO</code>       | <code>for _ in range( x2 ) :</code> |
| <code>x3 := SUCC( x3 )</code> | <code>x3 += 1</code>                |
| <code>END</code>              |                                     |

Das folgende Programm berechnet eine Multiplikation  $x_3 = x_1 \cdot x_2$ :

```

x3 := 0;
LOOP x1 DO
  LOOP x2 DO
    x3 := SUCC( x3 )
  END
END

```

```

x3 = 0
for _ in range( x1 ) :
  for _ in range( x2 ) :
    x3 += 1

```

**SATZ 4.11.** *Alle primitiv rekursiven Funktionen sind durch ein Programm in der Loop-Sprache berechenbar und alle mit einem Loop-Programm berechenbaren Funktionen sind primitiv rekursiv.*

**BEWEIS.** Die beiden Aussagen sind beispielsweise in [1] bewiesen. □

Das Konzept der primitiven Rekursion ist für sich schon sehr mächtig, so dass der berühmte Mathematiker David Hilbert 1926 vermutete, dass jede berechenbare Funktion primitiv rekursiv sei. Von seinem Schüler Wilhelm Ackermann wurde dazu im gleichen Jahr diese Funktion erdacht:

**DEFINITION 4.12.** Die (ursprüngliche) **Ackermann-Funktion** lautet:

$$\varphi(a, b, n) = \begin{cases} a + b, & \text{wenn } n = 0, \\ 0, & \text{wenn } n = 1 \text{ und } b = 0, \\ 1, & \text{wenn } n = 2 \text{ und } b = 0, \\ a, & \text{wenn } n > 2 \text{ und } b = 0, \\ \varphi(a, \varphi(a, b - 1, n), n - 1) & \text{sonst.} \end{cases}$$

Die Ackermann-Funktion hat diese überraschende Eigenschaft:

$$\begin{aligned} \varphi(a, b, 0) &= \text{add}(b, a) = a + b \\ \varphi(a, b, 1) &= \text{mul}(b, a) = a \cdot b \\ \varphi(a, b, 2) &= \text{pow}(b, a) = a^b \\ &\vdots \end{aligned}$$

Das liegt daran, dass die letzte Zeile der Ackermannfunktion genau den primitiven Rekursionsschritt entsprechender Definitionen der Funktionen durchführt.

**SATZ 4.13.** *Die Ackermann-Funktion ist nicht primitiv-rekursiv.*

**BEWEIS.** Diese Aussage ist naheliegend, da zur Berechnung von  $\varphi(a, b, n)$  für beliebiges  $b > 0$  anzunehmen ist, dass mindestens  $n$  primitiv rekursive Schleifen benötigt werden. Das wären mehr, als jedes formulierbare Loop-Programm darstellen kann. Details finden sich beispielsweise auch in [1]. □

**BEMERKUNG 4.14.** Später wurde von Péter und Robinson eine kürzere Definition einer Funktion mit entsprechendem Wachstumsverhalten vorgestellt, die von den meisten Quellen als „die“ **Ackermann-Funktion** bezeichnet wird:

$$A(m, n) = \begin{cases} n + 1 & , \text{ wenn } m = 0, \\ A(m, 1) & , \text{ wenn } n = 0, \\ A(m, A(m + 1, n)) & , \text{ sonst.} \end{cases}$$

Da diese Funktionen aber berechenbar sind, beispielsweise mit einer Turingmaschine oder einer höheren Programmiersprache, zeigt, dass die einfache Schleifen der Loop-Sprache oder die primitiven Rekursionen nicht ausreichen, um die Mächtigkeit der Turingmaschine zu erreichen. Die erforderliche Verallgemeinerung der primitiven Rekursion ist die  $\mu$ -Rekursion:

DEFINITION 4.15. Die folgenden Funktionen sind  $\mu$ -rekursiv:

- (1) Alle primitiv rekursiven Funktionen  $f$ ,
- (2) Verknüpfungen  $h(g_1(x_1, \dots, x_n), \dots, g_m(x_1, \dots, x_n))$   $\mu$ -rekursiver  $h, g_1, \dots, g_m$ ,
- (3) primitive Rekursionen von  $\mu$ -rekursiven Funktionen  $g$  und  $h$ :

$$f(m, n) = \begin{cases} g(n) & \text{für } m = 0, \\ h(f(m-1, n), m-1, n) & \text{für } m > 0, \end{cases}$$

- (4) die Anwendungen des  $\mu$ -Operators auf  $\mu$ -rekursive Funktionen  $f$  mit möglichem Ergebnis  $\perp$  für undefinierte Werte:

$$\mu f(x_1, \dots, x_n) = \begin{cases} \min\{k : f(x_1, \dots, x_{n-1}, k) = 0 \wedge f(f(x_1, \dots, x_{n-1}, l) \neq \perp \text{ für } l < k\}, \\ \perp, \text{ wenn die Menge leer ist.} \end{cases}$$

Die  $\mu$ -Rekursion erweitert die primitive Rekursion mit durch den Wert von  $m$  linear beschränktem Anteil durch die Suche nach einem Minimum über alle mögliche Werte in einer Komponente. Wie sich diese erweiterte Rekursion auf Schleifen auswirkt, zeigt die While-Sprache:

DEFINITION 4.16. Die **While-Sprache** entspricht der Loop-Sprache aus Definition 4.9 zuzüglich der Schleifenkonstruktion **WHILE** ... **DO** ... **END**, die so lange ausgeführt wird, bis die Schleifen-Variable den Wert 0 hat. Eine mögliche EBNF-Beschreibung der While-Sprache lautet:

```
start:      w* command w*
command:    command ";" w* command | assignment | loop | while
assignment: variable w* "!=" w* expression w*
expression: "0" | variable | function
variable:   "x" number
function:   "PRED(" w* expression w* ")" | "SUCC(" w* expression w* ")"
loop:       "LOOP" w* variable w* "DO" w* command w* "END" w*
while:      "WHILE" w* variable w* "DO" w* command w* "END" w*
number:     ( "1" .. "9" ) ( "0" .. "9" ) *
w:          "\u0020" | "\t" | "\n"
```

BEMERKUNG 4.17. Die **LOOP**-Schleife ist bezogen auf die Berechenbarkeit in der While-Sprache eigentlich nicht erforderlich, da jede **LOOP**-Schleife durch eine entsprechende **WHILE**-Konstruktion ersetzt werden kann. Doch ist so mit dieser Ergänzung jedes Loop-Programm automatisch auch ein korrektes While-Programm.

BEISPIEL 4.18. Dieses Programm in der While-Sprache berechnet ganzzahlig  $x_2 := \log_2(x_1)$ . Es zieht so lange Zweier-Potenzen von der untersuchten Zahl ab, bis 0 erreicht ist. Dazu wird für die äußerste Schleife zur Suche des kleinsten Zweierexponenten, der in der Größe ganzzahlig  $x_1$  entspricht, die **WHILE**-Konstruktion verwendet.

|                                                                                                                                                                  |                                                                                                                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>x1 := PRED( x1 ); x2 := 0; x3 := SUCC( SUCC( 0 ) ); WHILE x1 DO   x2 := SUCC( x2 );   LOOP x3 DO     x1 := PRED( x1 );     x3 := SUCC( x3 )   END END</pre> | <pre>x1 = max( x1 - 1 , 0 ) x2 = 0 x3 = 0 + 1 + 1 while x1 :   x2 += 1   for _ in range( x3 ) :     x1 = max( x1 - 1 , 0 )     x3 += 1</pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|



**SATZ 4.19.** *Alle  $\mu$ -rekursiven Funktionen sind durch ein Programm in der While-Sprache berechenbar und alle mit einem While-Programm berechenbaren Funktionen sind  $\mu$ -rekursiv. Außerdem ist die While-Sprache Turing-vollständig.*

**BEWEIS.** Die Aussagen sind beispielsweise in [1] bewiesen. □

Damit beschreiben die  $\mu$ -Rekursion und auch die While-Sprache, welche Art von Rekursion oder Schleifen für eine Turing-Vollständigkeit erforderlich sind. Auch weitere reduzierte Sprachen, wie die **GOTO-Sprache**, die neben Addition und Subtraktion auf Variablen einmal unbedingte und bedingte Sprünge besitzt, sind Turing-vollständig.

Einen komplett anderen Ansatz verfolgt das sehr elementare **Lambda-Kalkül**, das vollständig und allein auf der Definition und Anwendung von Funktionen basiert. Darin können abstrakte Entsprechungen für logische Aussagen und Zahlen wie auch das Konzept der Rekursion definiert werden und daraus ebenso eine Turing-Vollständigkeit hergeleitet werden.

Die Beschreibungen über unbeschränkte Rekursionen oder bedingten und damit nicht vorab begrenzten Schleifen sind damit sehr gute Indikatoren, ob oder wie eine Sprache als Turing-vollständig erkannt oder umgesetzt werden kann. Ein weiterer möglicher Ansatz ist aber auch die Konstruktion eines Rechenmodells aus Logikbausteinen:

**BEISPIEL 4.20.** Im Dezember 2021 wurde durch das Project Zero Team bei Google eine Erklärung Einbrüche in Smartphones unter anderem von Journalisten mit dem [NSO zero-click iMessage exploit](#) veröffentlicht. Dazu wurde eine vermeintliche GIF-Datei mitgesendet, die tatsächlich ein speziell erstelltes PDF enthielt und als solches verarbeitet wurde. Neben der LZW-Kompression dort auch die JBIG2-Kompression enthalten ist, deren Implementation aber einen Fehler enthielt. Dieser Fehler ermöglichte die Konstruktion beliebiger Logiken, die am Ende zu einem Turing-vollständigen Rechnermodell führen sollen. Zumindest haben die Fähigkeiten gereicht, um letztlich die Sandbox zu verlassen und in die Mobilgeräte einzubrechen.

Eine Vielzahl weiterer Modelle zeigten sich durch die Nachweise der Äquivalenz von Turingmaschinen ebenso mächtig, dass eine Definition der Berechenbarkeit über andere Modelle im Grunde höchstens wieder auf die Turing-Berechenbarkeit führt. Aus dieser Erfahrung wurde um 1936 die folgende, soweit nicht beweisbare (was bedeutet „intuitiv berechenbar“?) aber dennoch sehr wichtige These formuliert, die den vermuteten Zusammenhang zwischen dem menschlichen Verständnis für Berechenbarkeit und der abstrakten Definition herstellt:

**BEHAUPTUNG 4.21. (Church-Turing-These)** Die Klasse der Turing-berechenbaren Funktionen stimmt mit der Klasse der intuitiv berechenbaren Funktionen überein.

**BEISPIEL 4.22.** Teilweise sollen Sprachen absichtlich weniger mächtig sein: Beispielsweise ist die Programmiersprache **Bitcoin-Skript** absichtlich nicht Turing-vollständig. In dieser Sprache wird eine Transaktion beschrieben und auf der Bitcoin-Blockchain validiert und umgesetzt. Die Sprache ist zentral dadurch beschränkt, dass nur Vorwärtssprünge erlaubt sind und damit Schleifen unmöglich sind. Dies engt die Möglichkeiten für damit umsetzbare Logiken zwar stark ein, verhindert aber grundsätzlich Denial-of-Service-Attacken durch nicht endende Programme. Denn so kann die Laufzeit durch die Anzahl der Bytes in der Transaktion abgeschätzt werden, und die Transaktion wird nur mit einer der Laufzeit entsprechend hohen Transaktionsgebühr in die Blockchain aufgenommen.

Diese Einschränkung führte zur Entwicklung der Ethereum-Blockchain, bei der der Bytecode der **Ethereum Virtual Machine** auf Turing-Vollständigkeit ausgelegt wurde, und die ausgeführte Laufzeit sowie

ermöglichter Speicherzugriff oder weitere Kommunikation nicht durch Programmlänge sondern durch einen mitgesendeten Geldbetrag, dem Sprit oder englisch „gas“, beschränkt wird.

## 4.2. Komplexitätsklassen

Während sich die Berechenbarkeit mit der grundsätzlichen Frage beschäftigt, ob der Wert einer Funktion oder beispielsweise auch die Zugehörigkeit eines Wortes zu einer Sprache in einem Rechenmodell bestimmt werden kann, geht es bei der Frage der Komplexität darum, welcher Aufwand zur Beantwortung erforderlich ist.

Hier gibt es zwei grundsätzlich wichtige und unterschiedliche Sichtweisen: Einerseits die Frage, welche theoretisch relevante Komplexitätsklassen es überhaupt gibt, und andererseits die Frage, wie Komplexität sich in der Praxis auswirkt.

**DEFINITION 4.23.** Sei  $L$  eine formale Sprache, mit  $M$  eine deterministische Mehrband-Turingmaschine und mit  $N$  eine nichtdeterministische Mehrband-Turingmaschine nach Definition 4.5, die  $L$  erkennen. Dann ist

- $L \in \text{time}(f)$ , wenn es  $M$  gibt, die für  $w \in L$  nach  $\max. f(|w|)$  Schritten terminieren,
- $L \in \text{ntime}(f)$ , wenn es  $N$  gibt, die für  $w \in L$  nach  $\max. f(|w|)$  Schritten terminieren kann,
- $L \in \text{space}(f)$ , wenn es  $M$  gibt, die für  $w \in L$  maximal  $f(|w|)$  Bandpositionen nutzt,
- $L \in \text{nspace}(f)$ , wenn es  $N$  gibt, die für  $w \in L$  optimal maximal  $f(|w|)$  Bandpositionen nutzt.

Diese Einteilungen sind auf diesem Niveau natürlich sehr grob, weil weder einem Schritt eine Zeiteinheit zugeordnet werden kann, noch eine Bandposition eine bezifferbare Speichermenge beschreibt. Außerdem wird keine Zeit oder kein Raum für die Initialisierung oder Finalisierung berücksichtigt. Es können auch durch Vergrößerung der Alphabete  $\Sigma$  zu Vektoren  $\Sigma^2$  die Anzahl erforderlicher Programmschritte und Bandpositionen in einer entsprechend modifizierten Turingmaschine fast halbiert werden, da immer zwei Informationseinheiten auf einmal verarbeitet und pro Position doppelt so viel gespeichert werden kann. Damit sind diese Zuordnungen asymptotisch im Sinne des Wachstumsverhalten für große Wörter zu verstehen und jeweils mit beliebigem Vorfaktor und optionalen Konstanten. Mit diesem Verständnis können die wichtigsten Komplexitätsklassen definiert werden:

**DEFINITION 4.24.** Die Komplexitätsklassen  $P$ ,  $NP$ ,  $PSPACE$ , und  $NPSPACE$  sind definiert durch:

$$\begin{aligned} P &:= \bigcup_{k \in \mathbb{N}} \text{time}(n^k) \\ NP &:= \bigcup_{k \in \mathbb{N}} \text{ntime}(n^k) \\ PSPACE &:= \bigcup_{k \in \mathbb{N}} \text{space}(n^k) \\ NPSPACE &:= \bigcup_{k \in \mathbb{N}} \text{nspace}(n^k) \end{aligned}$$

Es ist leicht Probleme oder Sprachen anzugeben, die in  $P$  oder  $PSPACE$  liegen, und ebenso sind die Zusammenhänge  $P \subseteq NP$  sowie  $PSPACE \subseteq NPSPACE$  offensichtlich, da alle deterministischen Turingmaschinen auch immer nichtdeterministisch sind. Es ist aber klar, dass ein Algorithmus in  $P$  bzw.  $NP$  nur maximal  $PSPACE$  bzw.  $NPSPACE$  liegen kann, da in jedem Schritt maximal ein neue Bandposition benutzt werden kann. Durch Walter Savitch wurde 1970 festgestellt [5], dass jede nicht-deterministische Turingmaschine durch eine deterministische Turingmaschine mit maximal quadratisch höherem Speicheraufwand simuliert werden kann, damit ist  $PSPACE = NPSPACE$  und sowohl  $P$  als



ABBILDUNG 4.2.1. Fahrradschlösser mit unterschiedlich vielen Ziffern

auch NP liegen darin, doch ist es bisher ein ungelöstes Problem der Informatik, ob  $P \neq NP$  oder  $P = NP$  oder sogar  $P = NP = PSPACE$  gilt- bisher deutet aber viel dafür, dass alles echte Teilmengen sind. Insgesamt ergeben sich damit diese Aussagen:

SATZ 4.25. *Es gilt diese **Klassenhierarchie für Komplexitätsklassen**:*

$$P \subseteq NP \subseteq PSPACE = NPSPACE$$

Ist ein Entscheidungsproblem in der Komplexitätsklasse P, so gilt das Problem als relativ effizient lösbar, verglichen zu Problemen, die beispielsweise mit wachsender Problemstellung ein exponentielles Verhalten zeigen: Soll beispielsweise ein Schloss wie in Abbildung 4.2.1 mit einer etwas größeren Anzahl von  $n$  Zahlen zerstörungsfrei ohne Kenntnis der Kombination geöffnet werden, so braucht es eine feste maximale Zeit  $t$  pro Ziffer, diese einzustellen und die Kombination auszuprobieren. Da es bei 10 Ziffern  $10^n$  Möglichkeiten auszuprobieren gibt, würde ein deterministischer endlicher Automat zum Ausprobieren aller Kombinationen im schlimmsten Fall eine Zeit von  $n \cdot t \cdot 10^n$  benötigen. Eine nichtdeterministische Herangehensweise könnte hingegen alle Kombinationen parallel ausprobieren und wäre nach der Zeit  $n \cdot t$  bei der Lösung.

Oder anders formuliert zählt für die Zeitkomplexität der nichtdeterministischen Turingmaschine nur der Aufwand zum Prüfen einer korrekt erratenen Lösung, hingegen muss die deterministische Turingmaschine diese Lösung erst mit viel Aufwand finden.

Mit der Brute-Force Strategie ist mit Sicherheit kein Ergebnis nach einer polynomialen Abschätzung aus Anzahl von Ziffern für die erforderlichen Schritte zu erwarten. Es ist aber unklar, ob es in Zukunft nicht vielleicht mit einem Röntgenblick oder einer besonders strategischen Drehweise möglich wird auch mit einem deterministischen Ansatz die Lösung schneller zu erhalten. Es kann hier damit nicht mit Sicherheit gesagt werden, dass die Lösung sicher nicht in P sei, obwohl wir stark davon ausgehen. Umgekehrt ist es aber gesichert, dass das Problem sicher in NP liegt, da ein Algorithmus die Überprüfung in polynomialer Zeit, sogar linearer Zeit, in Abhängigkeit von der Anzahl der Ziffern sichert.

Das ist ein grundsätzliches Problem für einen Nachweis, dass ein Problem nicht schnell lösbar ist, also nicht in P liegt, dass es immer so sein kann, dass die Idee für eine schnelle Lösung schlicht noch nicht gefunden wurde. Es hilft dazu auch nicht, dass noch immer nicht bewiesen werden konnte, dass P und NP sich überhaupt unterscheiden. Hier hilft der Ansatz, nachzuweisen, das untersuchte Problem mindestens so schwierig wie andere schwierige Probleme ist– genau wie auch durch Simulation nachgewiesen wurde, dann ein Modell mindestens so mächtig ist:

DEFINITION 4.26. Seien zwei Sprachen  $L \subseteq \Sigma^*$  und  $N \subseteq \Gamma^*$  gegeben.  $N$  ist genau dann auf  $L$  **polynomiell reduzierbar**, geschrieben  $N \leq_{\text{poly}} L$ , wenn es eine in polynomiell beschränktem Aufwand berechenbare Funktion  $f : \Gamma^* \rightarrow \Sigma^*$  gibt mit

$$\forall w \in \Gamma^* : w \in N \Leftrightarrow f(w) \in L.$$

Da jedes Problem auf  $L$  in ein Problem in  $N$  äquivalent umgeformt werden kann, so muss  $N$  mindestens so schwer sein wie  $L$ . Damit können die wichtigsten Begriffe zur Klasse NP beschrieben werden:

DEFINITION 4.27. Eine Sprache  $L$  ist

- NP-schwer, wenn für alle  $N \in \text{NP}$  gilt, dass  $N \leq_{\text{poly}} L$ .
- NP-vollständig oder  $L \in \text{NPC}$  (NP-complete), wenn  $L$  NP-schwer und  $L \in \text{NP}$  ist.

Diese Definition stellt einerseits klar, dass entsprechende NP-Probleme tatsächlich schwierig sind, da sie auf alle anderen entsprechenden Probleme abgebildet werden. Doch ist die Eigenschaft NP-schwer wirklich schwer nachzuweisen, wenn überhaupt nicht alle NP-Probleme bekannt sind. Den ersten Durchbruch gab es mit dem SAT-Problem:

DEFINITION 4.28. Das **Erfüllbarkeitsproblem der Aussagenlogik** oder **SAT-Problem** für satisfiability bezeichnet die Frage, ob es für einen gegebenen geklammerten logischen Ausdruck aus booleschen Variablen mit den logischen Operationen von Konjunktion, Disjunktion und Negation in konjunktiver Normalform wie  $X_1 \wedge (\overline{X_1} \vee X_2)$  eine Belegung der Variablen gibt, so dass der Term wahr wird. Die entsprechende Sprache lautet:

$$\text{SAT} = \{F \mid F \text{ ist boolescher Ausdruck und erfüllbar}\}$$

Ein bedeutendes Ergebnis war der Satz von Cook, der sich mit dem Erfüllbarkeitsproblem der 1971 nachweisen konnte, dass alle NP-Probleme auf das SAT-Problem zurückgeführt werden können, und zwar durch die Definition der nichtdeterministischen Turingmaschinen. Dadurch ist jedes Problem, für das ein polynomialer Algorithmus für eine nichtdeterministische Turingmaschine angegeben werden kann, unmittelbar darstellbar als ein SAT-Problem, und damit ist das SAT-Problem unmittelbar NP-schwer! Da es selbst in polynomialer Zeit auf einer nichtdeterministischen Turingmaschine gelöst werden kann, einfach durch Ausprobieren einer korrekten Lösung, ist es auch NP-vollständig.

Jetzt wurde es immer einfacher, da alle anderen NP-Probleme ihre erforderliche NP-Härte durch polynomieller Ableitung aus dem SAT-Problem (oder einem anderen NP-schweren Problem) erhalten können, und damit sofort ebenso gegenüber allen weiteren Problemen transitiv abgeleitet werden können.

Entsprechend gibt es heute wortwörtlich einen Zoo weiterer Komplexitätsklassen, die im [complexity-zoo.net](http://complexity-zoo.net) bewundert werden können, doch es bleibt eine Kernfragen der Informatik und für großer Relevanz für die Anwendungen beispielsweise in der Kryptographie, warum noch immer nicht bewiesen werden konnte, dass  $P \neq \text{NP}$  gilt.

BEISPIEL 4.29. Ein typisches SAT-Problem ist die Frage, ob der Ausdruck

$$f(x_1, x_2, x_3, x_4) = \overline{x_3} \wedge (x_1 \vee x_2) \wedge (\overline{x_2} \vee x_3 \vee \overline{x_2} \vee x_3) \wedge (x_4 \vee \overline{x_1})$$

erfüllbar ist. Diese Frage kann mit einer Wertetabelle beantwortet werden:

|                         |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|-------------------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $x_1$                   | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| $x_2$                   | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| $x_3$                   | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| $x_4$                   | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| $f(x_1, x_2, x_3, x_4)$ | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |

Hier ist also die Antwort ja, denn mit  $x = (1, 0, 0, 1)$  ist der Ausdruck erfüllt.

DEFINITION 4.30. Das **3SAT-Problem** bezeichnet die Frage, ob logische Termine in konjunktiver Normalform mit maximal (oder genau) drei Literalen erfüllbar sind.

SATZ 4.31. *Das 3SAT-Problem ist NP-vollständig.*

BEWEIS. Das Problem liegt in NP, da eine Belegung direkt geprüft werden kann. Das Problem ist NP-schwer, da das SAT-Problem polynomiell auf das 3SAT-Problem zurückgeführt werden kann, also

$$\text{SAT} \leq_{\text{poly}} \text{3SAT}$$

gilt: Dazu muss gezeigt werden, dass eine Formel in konjunktiver Normalform eines SAT-Problems in ein äquivalentes 3SAT-Problem überführt werden kann. Sei  $x = (x_1, \dots, x_n)$  und  $f(x)$  ein gegebenes SAT-Problem in konjunktiver Normalform,

$$f(x) = g_1(x) \wedge \dots \wedge g_m(x)$$

wobei  $g_1(x), \dots, g_m(x)$  jeweils Terme von Literalen oder Negation von Literalen in Disjunktion. Besteht ein Term  $g_k(x)$  aus genau drei Termen, so kann dieser übernommen werden. Besteht ein Term aus weniger als drei Termen, so können Literale neutral vervielfältigt werden, z.B.  $x_4$  zu  $x_4 \vee x_4$  erweitert werden, um auf genau drei Terme zu kommen, und vergrößert damit Terme maximal auf das Dreifache. Es verbleiben Terme, die aus mehr als drei Literalen besteht. Hat  $g_k(x)$  gerade  $r$  Terme, so werden  $r - 2$  neue Literale  $y_v, \dots, y_{v+r-3}$  erzeugt: Hat  $g_k(x)$  die Form

$$g_k(x) = l_1 \vee \dots \vee l_r$$

so wird  $g_k(x)$  ersetzt wird durch

$$(l_1 \vee l_2 \vee y_v) \wedge (\overline{y_v} \vee l_3 \vee y_{v+1}) \wedge \dots \wedge (\overline{y_{v+r-4}} \vee l_{r-2} \vee y_{v+r-3}) \wedge (\overline{y_{v+r-3}} \vee l_{r-1} \vee l_r).$$

Damit verlängert sich maximal um das dreifache (in den mittleren Termen). Der entstehende Ausdruck ist genau dann erfüllbar, wenn der ursprüngliche Ausdruck erfüllbar ist und maximal dreimal länger. Damit ist jedes SAT-Problem auf 3SAT-Problem mit maximal dreifacher Länger zurückgeführt werden.

Damit ist das 3SAT-Problem mit dem SAT-Problem auch NP-schwer und somit NP-vollständig.  $\square$

BEISPIEL 4.32. Das Problem aus Beispiel 4.29 ist in dieses 3SAT-Problem überführbar:

$$g(x, y) = (\overline{x_3} \vee \overline{x_3} \vee \overline{x_3}) \wedge (x_1 \vee x_2 \vee x_2) \wedge (\overline{x_2} \vee x_3 \vee y_1) \wedge (\overline{y_1} \vee \overline{x_2} \vee x_3) \wedge (x_4 \vee \overline{x_1} \vee \overline{x_1})$$

DEFINITION 4.33. Das **Teilsommenproblem** oder **SSP-Problem** (subset sum problem) bezeichnet die Fragestellung, ob bei einer gegebenen Menge von natürlichen Zahlen  $M = \{m_1, \dots, m_n\}$ , ob eine Summe

$$0 \leq T \leq \sum_{k=1}^n m_k$$

durch eine Teilmenge  $N \subseteq M$  exakt erreicht werden kann:

$$T = \sum_{m \in N} m$$

BEISPIEL 4.34. Beispiele für ein Teilsommenprobleme sind die Fragen bei der gegebenen Menge  $M = \{2, 4, 5, 11, 20\}$ , welche Teilsommen  $T \in \{25, 34, 37\}$  aus dieser Menge erreicht werden können. Offensichtlich ist  $20 + 5 = 25$ , also ist diese Zahl erreichbar. Für die Zahl 34 muss etwas mehr überlegt werden: Da die Summe gerade ist, müssen die beiden ungeraden Zahlen zusammen auftreten, wenn sie überhaupt auftauchen.

$$20 + 11 + 5 = 36$$

$$2 + 4 + 11 + 5 = 22$$

Dazwischen kann nichts mit  $11 + 5$  erreicht werden, also geht es nur ohne die beiden, doch damit kann die gewünschte Zahl nicht mehr erreicht werden:

$$2 + 4 + 20 = 26$$

Also ist 34 nicht erreichbar. Die Zahl 37 ist ungerade, muss also entweder 5 oder 11 enthalten. Da  $20 + 5 + 4 + 2 = 31$  nicht reicht bleibt

$$20 + 11 + 4 + 2 = 37$$

und damit ist die Zahl 37 erreichbar.

Der Eindruck, dass das Teilsummenproblem sehr aufwendig werden kann, täuscht nicht:

**SATZ 4.35.** *Das Teilsummenproblem ist NP-vollständig.*

**BEWEIS.** Das Problem liegt in NP, da eine Summe direkt geprüft werden kann. Das Problem ist NP-schwer, da das 3SAT-Problem polynomiell auf das SSP-Problem zurückgeführt werden kann, also

$$3\text{SAT} \leq_{\text{poly}} \text{SSP}$$

gilt: Dazu muss gezeigt werden, dass eine Formel in konjunktiver Normalform eines 3SAT-Problems in ein äquivalentes SSP-Problem überführt werden kann. Sei  $x = (x_1, \dots, x_n)$  und  $f(x)$  ein gegebenes 3SAT-Problem mit  $n$  Variablen und  $m$  Termen in konjunktiver Normalform,

$$f(x) = (l_{1,1} \vee l_{1,2} \vee l_{1,3}) \wedge (l_{2,1} \vee l_{2,2} \vee l_{2,3}) \wedge \dots \wedge (l_{m,1} \vee l_{m,2} \vee l_{m,3})$$

wobei  $l_{k,p}$  jeweils negierte oder nicht-negierte Literale sind. Es werden jetzt Zahlen konstruiert, die zu einer bestimmten Zielsumme summiert werden können, wenn das 3SAT-Problem lösbar ist. Die erzeugten  $2n + 2m$  Zahlen haben insgesamt  $n + m$  Ziffern, wobei die vorderen  $n$  Ziffern garantieren, dass eine Besetzung wie 0 oder 1 für jede Variable gewählt wird, und die hinteren  $m$  Ziffern garantieren, dass alle Terme damit erfüllt werden. Zunächst die  $2n$  Zahlen für jede Variable  $x_k$  mit  $a_{k,l} = 1$ , wenn  $x_k$  nicht negiert im  $k$ -ten Term auftritt, sonst  $a_{k,l} = 0$ , und  $b_{k,l} = 1$ , wenn  $x_k$  negiert im  $k$ -ten Term auftritt, sonst  $b_{k,l} = 0$ .

$$\begin{array}{ccccccc} & & k & & & & k \\ & & \downarrow & & & & \downarrow \\ 0 & \dots & 0 & 1 & 0 & \dots & 0 & a_{k,1} & \dots & a_{k,m} & \text{und} & 0 & \dots & 0 & 1 & 0 & \dots & 0 & b_{k,1} & \dots & b_{k,m} \\ \underbrace{\hspace{10em}}_{n \text{ Variablen}} & & \underbrace{\hspace{10em}}_{m \text{ Terme}} & & \underbrace{\hspace{10em}}_{n \text{ Variablen}} & & \underbrace{\hspace{10em}}_{m \text{ Terme}} \end{array}$$

Zusätzlich werden für jeden Term  $l$  diese beiden Zahlen hinzugefügt:

$$\begin{array}{ccccccc} & & l & & & & l \\ & & \downarrow & & & & \downarrow \\ 0 & \dots & 0 & 0 & \dots & 0 & 1 & 0 & \dots & 0 & \text{und} & 0 & \dots & 0 & 0 & \dots & 0 & 2 & 0 & \dots & 0 \\ \underbrace{\hspace{10em}}_{n \text{ Variablen}} & & \underbrace{\hspace{10em}}_{m \text{ Terme}} & & \underbrace{\hspace{10em}}_{n \text{ Variablen}} & & \underbrace{\hspace{10em}}_{m \text{ Terme}} \end{array}$$

Die erreichbare Zielsumme ist dabei:

$$T = \underbrace{1 \dots 1}_{n \text{ Variablen}} \underbrace{4 \dots 4}_{m \text{ Terme}}$$

Die Rechnung muss dabei in einer hinreichend großen Basis erfolgen, so dass es nicht zu einem unerwünschten Übertrag in der Addition kommen kann. Bei dieser Konstruktion wird links erzwungen, dass zu jeder Variable eine Besetzung von 0 oder 1 gewählt wird, und auf der Rechten Seite muss in jedem Term mindestens eine positive Besetzung 1, aber auch 2 oder 3 sein, die nur bei 1 oder 2 auf 3 mit den weiteren  $2m$  Zahlen aufgefüllt werden kann. Nur genau wenn das gerade konstruierte SSP-Problem lösbar ist, ist auch das 3SAT-Problem lösbar, und die Problemgröße hat sich (nur) in etwa verdoppelt.  $\square$

BEISPIEL 4.36. Das 3SAT-Problem aus Beispiel 4.32 kann in dieses Teilsummenproblem überführt werden: Mit  $n = 5$  Variablen und  $m = 5$  Termen ergibt sich die Zielsumme:

$$T = 1111144444$$

Die möglichen Summanden für die  $n = 5$  Variablen lauten:

|       | normal / ... = 1 | negiert / ... = 0 |
|-------|------------------|-------------------|
| $x_1$ | 10000 01000      | 10000 00001       |
| $x_2$ | 01000 01000      | 01000 00110       |
| $x_3$ | 00100 00110      | 00100 10000       |
| $x_4$ | 00010 00001      | 00010 00000       |
| $y_1$ | 00001 00100      | 00001 00010       |

Die optionalen Summanden zum Auffüllen für die  $m = 5$  Terme lauten:

|             |             |
|-------------|-------------|
| 00000 10000 | 00000 20000 |
| 00000 01000 | 00000 02000 |
| 00000 00100 | 00000 00200 |
| 00000 00010 | 00000 00020 |
| 00000 00001 | 00000 00002 |

Das Problem wird gelöst durch  $x_1 = 1$ ,  $x_2 = 0$ ,  $x_3 = 0$ ,  $x_4 = 1$  und  $y_1 = 0$ , und genau das ist hier dann auch die Lösung in der Summation:

|             |             |
|-------------|-------------|
| $x_1 = 1 :$ | 10000 01000 |
| $x_2 = 0 :$ | 01000 00110 |
| $x_3 = 0 :$ | 00100 10000 |
| $x_4 = 1 :$ | 00010 00001 |
| $y_1 = 0 :$ | 00001 00010 |
|             | 11111 11121 |
|             | 00000 10000 |
|             | 00000 20000 |
|             | $\vdots$    |
|             | 00000 00200 |
|             | 00000 00020 |
|             | 00000 00001 |
|             | 00000 00002 |
|             | 11111 44444 |

Da fast jede Variable hier zufällig genau für die Erfüllung genau eines Terms außer dem vierten verantwortlich ist, muss schließlich zu allen Termen außer dem vierten noch die 1 und 2 addiert werden, beim vierten nur die 2, um die Zielsumme  $T$  zu erreichen.

Damit sind SAT, 3SAT und SSP Beispiele für viele bekannte NP-vollständige Probleme, jedoch gibt es auch Probleme, von denen unter anderem mit dramatischen Folgen vermutet wird, dass sie nicht NP-vollständig sind:

SATZ 4.37. Das Problem der **Faktorisierung** (FACTORIZE) einer ganzen Zahl  $n = pq \in \mathbb{N}$  in zwei ganzzahlige Primfaktoren  $p, q \in \mathbb{N}$  liegt in NP.

BEWEIS. Sind zwei Primfaktoren  $p$  und  $q$  gegeben, so ist die Multiplikation in polynomialer Zeit bezogen auf die Länge des Produkts durchführbar. Damit kann ein nichtdeterministisches Rechnermodell alle in Frage kommenden Faktoren parallel multiplizieren und in einem Pfad das korrekte Ergebnis in polynomialer Zeit bestimmen.  $\square$

Es ist bisher weder gelungen für die Faktorisierung oder noch für das ebenso wichtige diskrete Logarithmusproblem eine NP-Schwere nachzuweisen. Der Shor-Algorithmus hat bewiesen, dass diese Probleme auf einem Quantencomputer in polylogarithmischer Zeit gelöst werden können. Da dies ist noch für kein einziges NP-schweres Problem gelungen ist, wird vermutet, dass diese beiden Probleme nie NP-vollständig sein werden.

Da die Entwicklung der Quantencomputer immer weiter fortschreitet, müssen daher Alternativen für die am häufigsten verwendeten Public-Key-Kryptoverfahren RSA und ECC gefunden werden, die beide genau auf der Schwierigkeit dieser Problemklasse basieren. Aus den Erkenntnissen der Berechnungskomplexität erscheint es daher sinnvoll, auf Verfahren zu setzen, die auf Problemen basieren, die mindestens NP-vollständig sind. Tatsächlich war dies der Ansatz eines der ersten asymmetrischen Verschlüsselungsverfahren von Merkle und Hellman. Hier wurden Nachrichten mit dem Teilsummenproblem kodiert, das mit Hilfe eines Tricks in linearer Zeit gelöst werden konnte:

**DEFINITION 4.38.** Ein **vereinfachtes Teilsummenproblem** zur Bildung der Summe  $T$  liegt vor, wenn die Summanden  $M = \{m_1, \dots, m_n\}$  streng stark anwachsend sind, also größer als die Summe der vorangehenden Summanden sind:

$$m_k > \sum_{i=1}^{k-1} m_i$$

Dieses Problem wird auch „**super-increasing knapsack**“ genannt.

**BEISPIEL 4.39.** Beispiele für ein vereinfachtes Teilsummenprobleme sind die Fragen bei der gegebenen Menge  $M = \{1, 2, 6, 10, 20\}$ , welche Teilsummen  $T \in \{22, 24, 27\}$  aus dieser Menge erreicht werden können. Es kann nun jeweils der Größe nach probiert werden, ob die Zahlen abgezogen werden können:

$$\begin{array}{lll} 22 - 1 \cdot 20 = 2 & 24 - 1 \cdot 20 = 4 & 27 - 1 \cdot 20 = 7 \\ 2 - 0 \cdot 10 = 2 & 4 - 0 \cdot 10 = 4 & 7 - 0 \cdot 10 = 7 \\ 2 - 0 \cdot 6 = 2 & 4 - 0 \cdot 6 = 4 & 7 - 1 \cdot 6 = 1 \\ 2 - 1 \cdot 2 = 0 & 4 - 1 \cdot 2 = 2 & 1 - 0 \cdot 2 = 1 \\ 0 - 0 \cdot 1 = 0 & 2 - 1 \cdot 1 = 1 & 1 - 1 \cdot 1 = 0 \end{array}$$

Damit sind  $22 = 20 + 2$  und  $27 = 20 + 6 + 1$  und  $24$  ist nicht darstellbar. Es ist auch unmittelbar ersichtlich, wie die Binärzahl  $01001_2$  in der Zahl  $22$  und die Binärzahl  $10101_2$  in der Zahl  $27$  eindeutig kodiert wird. Diese einfache Lösung des Teilsummenproblems sollte natürlich nicht allen zur Verfügung stehen.

**DEFINITION 4.40. Merkle-Hellman-Kryptosystem, 1978.** Sei  $M = \{m_1, \dots, m_n\}$  die Menge eines vereinfachten Teilsummenproblems, sowie ein zufällige natürliche Zahlen  $C, e \in \mathbb{N}$  mit  $C > \sum m_k$  und  $\text{ggT}(C, e) = 1$ . Mit dem erweiterten euklidischen Algorithmus gibt es dann ein  $d \in \mathbb{N}$  mit  $e \cdot d \equiv 1 \pmod{C}$ . Sei nun  $w_k \equiv e \cdot m_k \pmod{C}$ . Dann ist  $K_{\text{pub}} = (w_1, \dots, w_n)$  der öffentliche Schlüssel, sowie  $K_{\text{priv}} = (m_1, \dots, m_n, d, C)$  der private Schlüssel des Kryptosystems. Sind  $b = (b_1, \dots, b_n)$  die Bits einer zu verschlüsselnden Nachricht, so ist  $u = E(b) = \sum b_k w_k$  die Chiffre der Nachricht. Diese kann mit dem privaten Schlüssel durch  $v \equiv d \cdot u \pmod{C}$  und Lösen des Teilsummenproblems von  $M$  zu  $v$  entschlüsselt werden.

**BEISPIEL 4.41.** Die Menge  $M = \{1, 2, 6, 10, 20\}$  ist die Menge eines vereinfachten Teilsummenproblems mit der Gesamtsumme  $39$ . Sei  $C = 41$  und  $e = 17$ , dann ist  $\text{ggT}(41, 17) = 1$  und  $17 \cdot 29 \equiv 1 \pmod{41}$ . Als öffentlichen Schlüssel ergibt sich damit aus  $1 \cdot 17 \equiv 17 \pmod{41}, \dots$

$$K_{\text{pub}} = (17, 34, 20, 6, 12)$$

und der private Schlüssel ist

$$K_{\text{priv}} = (1, 2, 6, 10, 20, 29, 41).$$



Soll nun die Nachricht  $21 = 10101_2$  verschlüsselt werden, so ergibt sich

$$u = E(21) = 1 \cdot 17 + 0 \cdot 34 + 1 \cdot 20 + 0 \cdot 6 + 1 \cdot 12 = 49.$$

Die Nachricht kann dann mit Hilfe des privaten Schlüssels mit

$$v = 49 \cdot 29 \equiv 27 \pmod{41}$$

und dann wie im Beispiel zuvor

$$\begin{array}{rcl} 27 - 1 \cdot 20 & = & 7 \\ 7 - 0 \cdot 10 & = & 7 \\ 7 - 1 \cdot 6 & = & 1 \\ 1 - 0 \cdot 2 & = & 1 \\ 1 - 1 \cdot 1 & = & 0 \end{array}$$

zu  $10101_2 = 21$  dechiffriert werden.

Das Merkle-Hellman-Kryptosystem galt eine zeitlang als sicher und als Alternative zu beispielsweise RSA. 1984 veröffentlichte Adi Shamir jedoch eine Arbeit, die nachwies, dass aus dem öffentlichen Schlüssel effizient passende Zahlen  $\tilde{d}$  und  $\tilde{C}$  gefunden werden können (nicht erforderlicherweise  $d, C$  aus dem privaten Schlüssel), die das nur scheinbar sichere Teilsummenproblem wieder in ein vereinfachtes Teilsummenproblem gewandelt werden konnte. Damit war das Verfahren gebrochen, da das verschleierte vereinfachte Teilsummenproblem kein NP-vollständiges Teilsummen-Problem war. Verbleibende Probleme für sicherere Kandidatenverfahren basieren beispielsweise auf Gittern oder fehlerkorrigierenden Codes wie dem McEliece-Verfahren.

### 4.3. Algorithmische Komplexität

Wie nun tatsächlich Algorithmen bezüglich ihrer Komplexität bewertet werden, wird hier an Hand eines Beispiels dargestellt:

**DEFINITION 4.42.** Das **Rucksackproblem**: Gegeben seien Wertepaare  $\{(g_1, w_1), \dots, (g_m, w_m)\}$  mit  $g_i, w_i \in \mathbb{N}$ , die das Gewicht  $g_i$  und den Wert  $w_i$  eines Teils  $i$  darstellen. Gesucht sind die Anzahlen  $a_i \in \mathbb{N}_0$  der jeweiligen Teile, so dass

$$\sum_{i=1}^m a_i g_i \leq n \quad \text{und} \quad \sum_{i=1}^m a_i w_i \text{ maximal wird,}$$

also für gegebene maximale Last  $n$  des Rucksacks der aufsummierte Wert maximal wird.

**BEISPIEL 4.43.** Die Abbildung 4.3.1 zeigt zwei Algorithmen, die das Problem einmal rekursiv und einmal iterativ lösen.

Die erste Variante ist rekursiv und prüft für jedes mögliche Teil  $i$ , ob es mit Gewicht  $g_i$  noch in einen Rucksack mit gegebener Restkapazität  $n$  passt, also  $n \geq g_i$  ist. Wenn ja, so berechnet er testweise, ob der Wert mit diesem Teil  $w_i$  plus dem besten Wert eines Rucksacks mit Kapazität  $n - g_i$  besser als der bisherige Bestwert ist, wenn ja, so haben wir einen neuen Bestwert gefunden, die am Ende nach dem Test mit allen Teilen zurückgegeben wird.

Die zweite Variante ist iterativ und sie berechnet jeweils das beste Ergebnis für alle Kapazitäten bis inklusive  $n$  jeweils bei Hinzunahme eines neuen Teils  $i$ . Ohne Teile ist der beste Wert für alle Kapazitäten 0. Ab einer Kapazität von  $j = g_i$  kann testweise iterativ bis  $j = n$  geprüft werden, ob das bisher beste Ergebnis bei einer Kapazität von  $j - g_i$  zuzüglich dem Wert  $w_i$  eines Teils  $i$  besser als der bisherige Bestwert für die aktuelle Kapazität  $j$  ist. Wenn alle Teile und Kapazitäten durchlaufen sind, so wird der Bestwert für die geforderte Kapazität  $n$  zurückgegeben. Die Tabelle `best[]` durchläuft für  $n = 13$  diese Werte:

```

static int gW[][] =
    {{1,1},{3,4},{5,8},{2,3}};

static int bestWertRek(int n)
{
    int best=0;
    for(int i=0;i<gW.length;i++)
    {
        int gewt=gW[i][0],
            wert=gW[i][1];
        if(n>=gewt)
        {
            int test=wert
                +bestWertRek(n-gewt);
            if(test>best)
                best=test;
        }
    }
    return best;
}

static int gW[][] =
    {{1,1},{3,4},{5,8},{2,3}};

static int bestWertIte(int n)
{
    int[] best=new int[n+1];
    for(int i=0;i<gW.length;i++)
    {
        int gewt=gW[i][0],
            wert=gW[i][1];
        for(int j=gewt;j<=n;j++)
        {
            int test=best[j-gewt]
                +wert;
            if(test>best[j])
                best[j]=test;
        }
    }
    return best[n];
}

```

ABBILDUNG 4.3.1. Zwei Java-Algorithmen zur Lösung eines Rucksackproblems aus Definition 4.42

| $j \setminus i$    | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8  | 9  | 10 | 11 | 12 | 13 |    |
|--------------------|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|
| $g_1 = 1, w_1 = 1$ | 0 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7  | 8  | 9  | 10 | 11 | 12 | 13 |
| $g_2 = 3, w_2 = 4$ | 1 | 0 | 1 | 2 | 4 | 5 | 6 | 8 | 9  | 10 | 12 | 13 | 14 | 16 | 17 |
| $g_3 = 5, w_3 = 8$ | 2 | 0 | 1 | 2 | 4 | 5 | 8 | 9 | 10 | 12 | 13 | 16 | 17 | 18 | 20 |
| $g_4 = 2, w_4 = 3$ | 3 | 0 | 1 | 3 | 4 | 6 | 8 | 9 | 11 | 12 | 14 | 16 | 17 | 19 | 20 |

Bei der Berechnung der algorithmischen Komplexität wird die Anzahl der erforderlichen Operationen in Abhängigkeit von der Problemgröße, hier typischerweise die Kapazität  $n$ , berechnet, die zur Durchführung und damit Lösung des Verfahrens erforderlich sind. Da unterschiedliche Operationen unterschiedlichen Aufwand besitzen, erst recht auch auf unterschiedlichen Architekturen, wird die Untersuchung meist auf relevante Operationen eingeschränkt und diese gezählt. Hier könnte Anzahl der Summationen der `test`-Variable oder auch der Vergleich `test>best` oder `test>best[j]` gewählt werden. Natürlich erfordern auch die Schleifen Additionen und Vergleiche, und je nach Fall muss untersucht werden, ob diese eventuell einen Einfluss auf das Ergebnis haben.

Werden hier die Summationen  $c_{\text{Add}}(n)$  in Abhängigkeit von der Kapazität  $n$  gezählt, und die Anzahl der Aufrufe bei einer Kapazität von  $n$  jeweils im schlimmsten Fall mit allen  $m$  Objekten mit Gewicht  $g_i = 1$ , so wird die rekursive Variante in erster Ebene  $m$  Additionen benötigen und sich jeweils  $m$  mal selbst aufrufen, darin entstehen dann  $m \cdot m = m^2$  Additionen und insgesamt mit der geometrischen Summenformel:

$$c_{\text{Add}}^{\text{Rek}}(n) = \underbrace{m + m^2 + \dots + m^n}_{n \text{ Summanden}} = m \cdot (1 + m + \dots + m^{n-1}) = m \cdot \frac{m^n - 1}{m - 1} = \frac{m}{m - 1} \cdot (m^n - 1)$$

Hier also ist  $c_{\text{Add}}^{\text{Rek}}(n) = \frac{4}{3}(4^n - 1)$ . Die Iterative Variante ist leichter zu bewerten, da es hier genau zwei Schleifen über  $n$  und  $m$  gibt:

$$c_{\text{Add}}^{\text{Ite}}(n) = n \cdot m$$

Für die iterative Variante liegt der algorithmische Aufwand in Additionen hier damit bei  $c_{\text{Add}}^{\text{Ite}}(n) = 4n$ .

**BEMERKUNG 4.44.** Selbst für kleine Werte von  $n$  scheint hier die iterative Variante deutlich im Vorteil zu sein, da der (für  $m > 1$ ) eigentlich nur relevante Term  $m^n$  für steigendes  $n$  deutlich stärker steigt, als  $m \cdot n$ . Der Grund dafür, dass die iterative Variante so sehr viel schneller ist, liegt daran, dass in der rekursiven Variante frühere Ergebnisse von parallelen Ästen nicht gespeichert werden und daher sehr oft erneut berechnet werden. Die iterative Variante speichert die Zwischenwerte und ist daher so viel schneller. Dieses Vorgehen der Zwischenspeicherung nennt sich auch **dynamische Programmierung**, wobei der Begriff „Programmierung“ hier die Lösung eines Optimierungsproblems und nicht allgemeine Programmerstellungen meint. In der rekursiven Variante würde ein über die Äste hinweg persistenter Cache für frühere Ergebnisse auf das entsprechende Ergebnis kommen.

**BEMERKUNG 4.45.** Das Rucksackproblem ist ein NP-vollständiges Problem, liegt also in NPC, was bei einem linearem Aufwand von  $c_{\text{Add}}^{\text{Ite}}(n) = n \cdot m$  mit dynamischer Programmierung zunächst etwas überraschend erscheint. Der Grund liegt darin, dass  $n$  nicht der Wortlänge entspricht, sondern z.B. Binärwörter der Länge  $k$  Werte bis zu  $n = 2^k - 1$  erreichen können. Damit ist der Aufwand, selbst mit dynamischer Programmierung in Abhängigkeit der Wortlänge „ $|n|$ “ tatsächlich  $(2^k - 1) \cdot m$  und damit keineswegs mehr linear. Bei dem Beispiel der Schlösser hatten wir auf natürliche Art von der Anzahl der Stellen gesprochen, in anderen Problemstellungen muss daher geprüft werden, was genau die Wortlänge ist. Jedoch ist für den lokalen Vergleich von zwei alternativen Algorithmen nicht immer die Betrachtung bezüglich der eher globalen Wortlänge erforderlich; im obigen Beispiel war die Kapazität als direkt zugängliche Ausgangsgröße sehr passend, um sich ein Bild im Vergleich machen zu können.

Im Beispiel war der Ausdruck  $\frac{4}{3}(4^n - 1)$  sehr unhandlich, wenn es eigentlich um das Verhalten von  $4^n$  geht. Daher werden für algorithmische Aufwände die Landau-Symbole verwendet:

**DEFINITION 4.46. (Landau-Symbol  $\mathcal{O}$ )** Es gilt  $f \in \mathcal{O}(g)$ , wenn  $\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} < \infty$  gilt.

Damit gilt für die rekursive und iterative Lösung des Rucksackproblems bezüglich der Kapazität  $n$

$$c_{\text{Add}}^{\text{Rek}}(n) = \frac{4}{3}(4^n - 1) \in \mathcal{O}(4^n), \text{ und } c_{\text{Add}}^{\text{Ite}}(n) = 4n \in \mathcal{O}(n),$$

sowie bezüglich der Wortlänge  $n = 2^k - 1$  dann

$$c_{\text{Add}}^{\text{Rek}}(2^k - 1) = \frac{4}{3}(4^{(2^k - 1)} - 1) \in \mathcal{O}(4^{(2^k)}), \text{ und } c_{\text{Add}}^{\text{Ite}}(2^k - 1) = 4 \cdot (2^k - 1) \in \mathcal{O}(2^k).$$

Außer der Laufzeit im schlimmsten Fall kann auch die Laufzeit im besten Fall oder im durchschnittlichen Fall, oder auch entsprechende Aufwandsabschätzungen für die Nutzung des Speichers oder Anzahl an Transaktionen verwendet werden.

**SATZ 4.47.** Für die Simulation erkennender Automaten und Maschinen wurden für effiziente Algorithmen diese Abschätzungen für Wortlängen  $n$  bewiesen:

| Sprache         | Automat                             | Aufwand            |
|-----------------|-------------------------------------|--------------------|
| regulär         | Deterministischer endlicher Automat | $\mathcal{O}(n)$   |
| kontextfrei     | Kellerautomaten                     | $\mathcal{O}(n^3)$ |
| kontextsensitiv | Linear beschränkter Automat         | PSPACE-vollständig |
| Typ-0           | Turingmaschine                      | unentscheidbar     |

Diese Aufwände rechtfertigen zusätzlich sehr deutlich, dass in der Anwendung, wie beispielsweise für Programmiersprachen, weitestgehend kontextfreie Sprachen verwendet werden.

#### 4.4. Aufgaben

**AUFGABE 73.** Zeigen Sie, dass die Multiplikation  $\text{mul}(m, n) = m \cdot n$  primitiv rekursiv ist.

**AUFGABE 74.** Zeigen Sie, dass die Potenzierung  $\text{pow}(m, n) = n^m$  primitiv rekursiv ist. Die Operandenreihenfolge ist aus technischem Grund wegen der Definition der primitiven Rekursivität vertauscht.

**AUFGABE 75.** Formulieren Sie ein Programm in der Loop-Sprache, mit dem  $x_3 = x_1^{x_2}$  berechnet wird.

**AUFGABE 76.** Formulieren Sie ein Programm in der Loop-Sprache, mit dem  $x_2 = x_1!$  berechnet wird.

**AUFGABE 77.**

- (1) Ist die Prüfung ob eine Zahl ein Teiler einer anderen Zahl ist, primitiv rekursiv überprüfbar?
- (2) Ist die Prüfung ob eine Zahl eine Primzahl ist, primitiv rekursiv überprüfbar?

Falls etwaig die Fragen primitiv rekursiv prüfbar sind, können sie mit der WHILE-Sprache effizienter überprüft werden?

**AUFGABE 78.** Für eine gegebene natürliche Zahl größer 2 soll die nächste höhere Primzahl bestimmt werden, wenn sie nicht schon eine Primzahl ist. Ist das eine sinnvoll definierte Funktion? Beweisen Sie, ob diese Frage mit einer primitiven Rekursion bestimmt werden kann. (Hinweis: Der Abstand zwischen zwei Primzahlen kann beliebig groß werden, betrachten Sie z.B. die Zahlen  $n! + 2, \dots, n! + n$ )

Kann ein WHILE-Programm diese Frage beantworten?

**AUFGABE 79.** Prüfen Sie, ob dieser boolesche Ausdruck erfüllbar ist:

$$f(x_1, x_2, x_3, x_4) = (x_4 \vee x_3) \wedge (x_1 \vee \overline{x_4}) \wedge (\overline{x_3} \vee x_4) \wedge (x_3 \vee \overline{x_1}) \wedge (\overline{x_3} \vee \overline{x_1} \vee \overline{x_4})$$

**AUFGABE 80.** Prüfen Sie, ob dieser boolesche Ausdruck erfüllbar ist:

$$f(x_1, x_2, x_3, x_4) = (\overline{x_3} \vee x_1) \wedge (x_3 \vee \overline{x_1} \vee x_4) \wedge (x_3 \vee x_1) \wedge (\overline{x_2} \vee \overline{x_1} \vee x_4) \wedge (\overline{x_4} \vee \overline{x_1})$$

**AUFGABE 81.** Wandeln Sie dieses Problem in ein 3SAT-Problem:

$$f(x_1, x_2, x_3, x_4) = (x_4 \vee \overline{x_3}) \wedge (x_3 \vee x_2 \vee x_1 \vee x_4) \wedge (\overline{x_2} \vee x_3 \vee x_4 \vee x_1) \wedge (x_2 \vee \overline{x_4})$$

**AUFGABE 82.** Wandeln Sie dieses Problem in ein 3SAT-Problem:

$$f(x_1, x_2, x_3, x_4) = (\overline{x_1} \vee x_3 \vee x_2 \vee \overline{x_4}) \wedge (x_4 \vee x_3) \wedge (\overline{x_3} \vee \overline{x_2}) \wedge (\overline{x_2} \vee \overline{x_1} \vee \overline{x_4} \vee x_3)$$

**AUFGABE 83.** Sei  $M = \{6, 7, 10, 14, 17, 19\}$ . Ist das Teilsummenproblem für  $T \in \{45, 53\}$  lösbar?

**AUFGABE 84.** Sei  $M = \{8, 9, 10, 15, 19, 20\}$ . Ist das Teilsummenproblem für  $T \in \{57, 65\}$  lösbar?

**AUFGABE 85.** Wandeln Sie dieses 3SAT-Problem in ein Teilsummenproblem um:

$$f(x_1, x_2, x_3) = (x_3 \vee \overline{x_2} \vee x_1) \wedge (x_3 \vee x_2 \vee \overline{x_1}) \wedge (\overline{x_1} \vee \overline{x_3} \vee \overline{x_2}) \wedge (\overline{x_2} \vee \overline{x_1} \vee x_3)$$

**AUFGABE 86.** Wandeln Sie dieses 3SAT-Problem in ein Teilsummenproblem um:

$$f(x_1, x_2, x_3) = (x_2 \vee x_1 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_3} \vee x_2) \wedge (x_1 \vee \overline{x_2} \vee x_3)$$

**AUFGABE 87.** Sei  $M = \{3, 4, 10, 18, 37, 76\}$ . Ist das Teilsummenproblem für  $T \in \{107, 112\}$  lösbar?

**AUFGABE 88.** Sei  $M = \{3, 4, 8, 16, 33, 72\}$ . Ist das Teilsummenproblem für  $T \in \{104, 108\}$  lösbar?

**AUFGABE 89.** Erstellen Sie für  $M = \{5, 7, 15, 28\}$ ,  $C = 61$  und  $e = 30$  das Merkle-Hellman-Kryptosystem Schlüsselpaar.

**AUFGABE 90.** Erstellen Sie für  $M = \{4, 9, 25, 40\}$ ,  $C = 100$  und  $e = 7$  das Merkle-Hellman-Kryptosystem Schlüsselpaar.

**AUFGABE 91.** Ein Merkle-Hellman Kryptosystem hat das Schlüsselpaar

$$k_{\text{pub}} = (50, 75, 76, 29, 61), \quad k_{\text{sec}} = (2, 3, 7, 17, 46, 4, 99).$$

Welche Dezimalzahl wurde mit der Zahl 212 verschlüsselt?

**AUFGABE 92.** Ein Merkle-Hellman Kryptosystem hat das Schlüsselpaar

$$k_{\text{pub}} = (34, 68, 69, 40, 16), \quad k_{\text{sec}} = (2, 4, 7, 20, 48, 3, 100).$$

Welche Dezimalzahl wurde mit der Zahl 143 verschlüsselt?

**AUFGABE 93.** Dieser Algorithmus in Java soll durch lineare Suche den Wert `value` in dem aufsteigend sortierten Array finden und den Index zurückgeben, sowie eine `-1`, falls der Wert nicht gefunden wurde:

```
static int linSearch( int[] sorted , int value )
{
    for( int i = 0 ; i < sorted.length ; i++ )
    {
        if( sorted[ i ] == value )
            return i;
    }
    return -1;
}
```

Bestimmen Sie den Aufwand und Aufwandsabschätzung in O-Notation in Anzahl Elementzugriffe `sorted[ i ]` in Abhängigkeit von Arraylänge `sorted.length = 2k` im schlimmsten Fall, im besten Fall und im durchschnittlichen Fall.

**AUFGABE 94.** Dieser Algorithmus in Java soll durch lineare Suche den Wert `value` in dem aufsteigend sortierten Array finden und den Index zurückgeben, sowie eine `-1`, falls der Wert nicht gefunden wurde:

```
static int binSearch( int[] sorted , int value )
{
    int low  = 0;
    int high = sorted.length;
    while( low < high )
    {
        int p = ( low + high ) >> 1;
        int q = sorted[ p ];
        if( q < value )
            low = p + 1;
        else if( q > value )
            high = p;
        else
            return p;
    }
    return -1;
}
```

Bestimmen Sie den Aufwand und Aufwandsabschätzung in O-Notation in Anzahl Elementzugriffe `sorted[ i ]` in Abhängigkeit von Arraylänge `sorted.length = 2k` im schlimmsten Fall, im besten Fall und schätzen Sie den durchschnittlichen Fall. Vergleichen Sie das Resultat zur vorherigen Aufgabe.

## KAPITEL 5

# Lösungen

## Literaturverzeichnis

- [1] HOFFMANN, DIRK W: *Theoretische Informatik*. Carl Hanser Verlag GmbH Co KG, 2022.
- [2] HUNT III, HARRY B: *On the time and tape complexity of languages I*. In: *Proceedings of the fifth annual ACM symposium on Theory of computing*, Seiten 10–19, 1973.
- [3] KEARNS, MICHAEL J: *The computational complexity of machine learning*. MIT press, 1990.
- [4] PRIESE, LUTZ und KATRIN ERK: *Theoretische Informatik: Eine umfassende Einführung*. Springer-Verlag, 2018.
- [5] SAVITCH, WALTER J: *Relationships between nondeterministic and deterministic tape complexities*. *Journal of computer and system sciences*, 4(2):177–192, 1970.
- [6] WEGENER, INGO: *Kompendium Theoretische Informatik-eine Ideensammlung*. Springer-Verlag, 2013.
- [7] WEGENER, INGO: *Theoretische Informatik: Eine algorithmenorientierte Einführung*. Springer-Verlag, 2013.





# Index

3SAT-Problem, 85

$\mu$ -rekursiv, 80

## A

Ableitung, 18

Abschluss, 6

abzählbar, 7

Ackermann-Funktion, 79

Akzeptor, 46

Akzeptoren, 41

allgemeinen Chomsky-Grammatik, 20

Alphabet, 5

aufzählbar, 11

Aufzählung, 11

## B

Backus-Naur-Form (BNF), 28

berechenbar, 75

beschränkte Chomsky-Grammatik, 68

beschränkte Turingmaschine, 66

Bitcoin-Skript, 81

## C

Chomsky Normalform, 25

Chomsky-Typ, 20

Church-Turing-These, 81

Collatz-Problem, 12

CYK-Algorithmus, 27

## D

determiniert kontextfreien Sprachen, 64

deterministische kontextfreie Grammatik, 24

deterministische Turingmaschine (TM), 66

deterministischer endlicher Automat (DEA), 41

Diagonalargument von Cantor, 9

Diagonalisierungsverfahren von Cantor, 7

domänenspezifische Sprachen (DSL), 29

Dyck-Sprache, 73

dynamische Programmierung, 91

## E

entscheidbar, 11

entscheidbare Sprache, 12

Entscheidungsproblem, 11

Erfüllbarkeitsproblem der Aussagenlogik, 84

Erweiterte Backus-Naur-Form (EBNF), 28

Ethereum Virtual Machine, 81

externe DSL, 30

## F

Faktorisierung, 87

Field Programmable Gate Array (FPGA), 58

formale Sprache, 6

## G

Gleichheit von Sprachen, 44

Gödelisierung, 68

Gödelnummer, 8

GOTO-Sprache, 81

Grammatik, 17

Grammatik für boolesche Terme, 18

Graphics Interchange Format (GIF), 30

## H

Halteproblem, 13, 22

$\epsilon$ -Hülle, 51

## I

inhärent mehrdeutig, 23

interne DSL, 30

Internet-of-Things (IoT), 58

iterierte Übergangsfunktion, 41

iterierte Übergangsrelation, 49

iterierte  $\epsilon$ -Übergangsrelation, 51

## K

kartesisches Produkt, 5

$k$ -Band-Turingmaschine, 76

Klammersprache, 73

Klammerung regulärer Ausdrücke, 31

Klassenhierarchie für Komplexitätsklassen, 83

Kleene-\*-Abschluss, 5

Kleene-+-Abschluss, 5

Konfiguration einer Turingmaschine, 66

Konfiguration eines Kellerautomaten, 59

Konkatenation, 6

Konstruktion äquivalenter regulärer Ausdrücke aus  
DEA, 45

konstruktivistische Logik, 11

kontextfreien Grammatik, 20

kontextsensitiven Grammatik, 20

## L

Lambda-Kalkül, 81

Landau-Symbol  $\mathcal{O}$ , 91

Länge, 5

leerenden nichtdeterministischen Kellerautomaten, 60

leeres Wort, 5

Leerheitsproblem, 13

Lemma von Arden, 44

Lempel-Ziv-Welch (LZW)-Algorithmus, 30

linear beschränkter Automat (LBA), 67

Linksableitung, 23

links-lineare Grammatiken, 20

Loop-Sprache, 78

Low-Code-Entwicklungsplattform, 29

## M

matcht, 31

Mealy-Automat, 46

Mehrband-Turingmaschine, 76

mehrdeutig, 23

Merkle-Hellman-Kryptosystem, 88

Moore-Automat, 46

## N

nicht entscheidbar, 11

nichtdeterministischer endlicher Automat (NEA), 48

nicht-deterministischer endlicher Epsilon-Automat ( $\epsilon$ -NEA), 50

nichtdeterministischer Kellerautomat, 59

nicht-triviale Eigenschaften, 76

NKA, 59

## P

Parsing Expression Grammar (PEG), 29

partielle Zustandsdiagramm, 52

passt, 31

polynomiell reduzierbar, 84

Portable Document Format (PDF), 30

Potenz, 6

primitiv rekursiv, 78

Produkt, 6

Produktion, 18

Programmable Input/Output (PIO), 58

Pumping-Lemma für kontextfreie Sprachen, 65

Pumping-Lemma für reguläre Sprachen, 54

Pumping-Zahl, 54

## R

Rechtsableitung, 23

rechts-lineare Grammatiken, 20

Regel, 17

Registermaschine, 76

Regular Expressions, 33, 34

regulären Grammatik, 20

regulärer Ausdruck, 31

rekursiv aufzählbar, 11

rekursive Sprache, 12

Rucksackproblem: , 89

## S

SAT-Problem, 84

Satz von Rice, 76

semi-entscheidbar, 11

Sprache der geraden Palindrome, 61

Sprache der ungeraden Palindrome, 73

Sprache des deterministischen endlichen Automaten, 42

Sprache des nichtdeterministischen endlichen Automaten, 49

Sprache des nichtdeterministischen endlichen Epsilon-Automaten, 51

Sprache einer Grammatik, 18

Sprache einer Turingmaschine, 66

Sprache eines nichtdeterministischen Kellerautomaten, 59

Sprachhierarchie, 22

SSP-Problem, 85

Startsymbol, 17

Stellenwertsystem, 8

super-increasing knapsack, 88

Symbol, 5

Syntaxbaum, 23

## T

Teilsummenproblem, 85

Terminale, 17

tertium non datur, 11

Transduktor, 46

Transduktoren, 41

transitive Ableitung, 18

Tupel, 5

Turing-berechenbar, 75

Turing-vollständig, 77

Typ-0 Grammatik, 20

Typ-1 Grammatik, 20

Typ-2 Grammatik, 20

Typ-3 Grammatik, 20

## U

überabzählbar unendlich, 7

Universelle Turingmaschine, 75

## V

Variable, 17

vereinfachtes Teilsummenproblem, 88

Von-Neumann-Architektur, 75

## W

While-Sprache, 80

Wort, 5

Wortmenge, 5

Wortproblem, 12

**Z**

Zeichen, [5](#)

Zustandsdiagramm , [42](#), [48](#)

zustandslose Kellermaschine, [62](#)

Zustandsmaschine, [58](#)